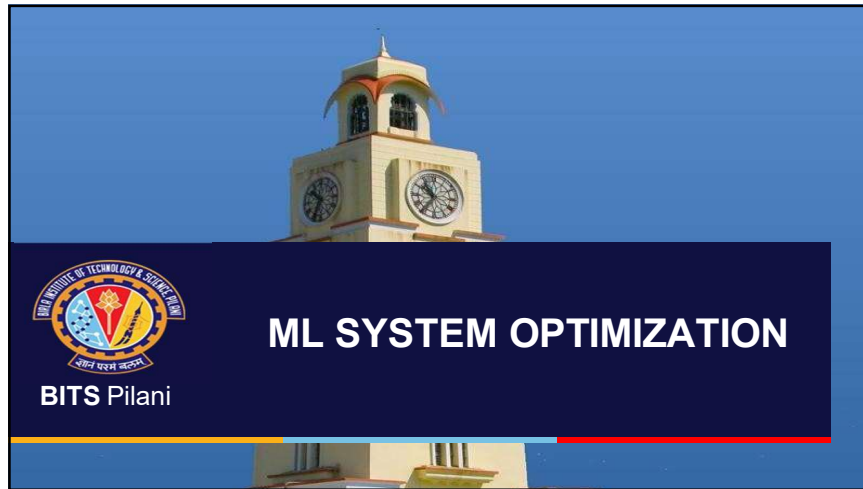




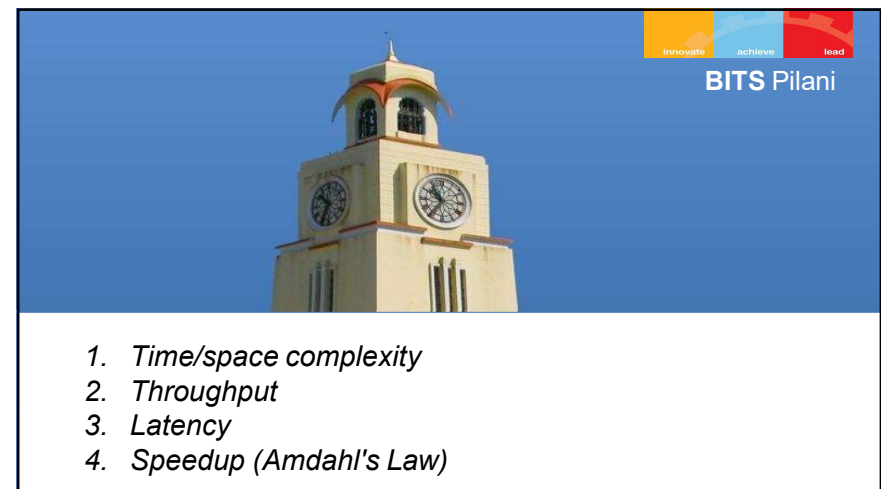
1



3



2



4

Time Complexity

- Time complexity measures how the **execution time of an algorithm** grows with the size of input n . It is independent of hardware but depends on the algorithm's structure.

$T(n)$ = time taken to process input of size n

- Helps **predict performance** for large inputs.
- Allows comparison between **different algorithms** solving the same problem.
- Guides **optimization** in high-performance computing.

Asymptotic Notations

Big O (O): Upper bound, worst-case time

$T(n) = O(f(n)) \Rightarrow$ Algorithm will take at most $c \cdot f(n)$ time for large n

Omega (Ω): Lower bound, best-case time

$T(n) = \Omega(f(n)) \Rightarrow$ Algorithm will take at least $c \cdot f(n)$ time for large n

Theta (Θ): Tight bound, average-case time

$T(n) = \Theta(f(n)) \Rightarrow$ Algorithm grows exactly like $f(n)$

5

Time Complexity

Linear Search: Search for element in array of size n

$T(n)$ = comparisons made = n (worst case)

$T(n) = O(n)$

Binary Search: Search in sorted array

$T(n) = 1 + \log_2 n = O(\log n)$

Recursive Algorithms

Time complexity can also be **recursively defined**. Example: Merge Sort

$T(n) = 2 \cdot T\left(\frac{n}{2}\right) + O(n)$

$T(n) = 2 \cdot T(2n) + O(n)$

Using Master Theorem: $T(n) = O(n \log n)$

7

Time Complexity

Algorithm	Best Case	Average Case	Worst Case
Linear Search	$\Omega(1)$	$\Theta(n)$	$O(n)$
Binary Search	$\Omega(1)$	$\Theta(\log n)$	$O(\log n)$
Merge Sort	$\Omega(n \log n)$	$\Theta(n \log n)$	$O(n \log n)$
Bubble Sort	$\Omega(n)$	$\Theta(n^2)$	$O(n^2)$

Common Growth Rates

- $O(1)$:** Constant time – independent of n
- $O(\log n)$:** Logarithmic – e.g., binary search
- $O(n)$:** Linear – e.g., linear search, sum of array
- $O(n \log n)$:** Log-linear – e.g., merge sort, quicksort average case
- $O(n^2)$:** Quadratic – e.g., bubble sort, insertion sort
- $O(2^n)$, $O(n!)$:** Exponential / factorial – impractical for large n

6

Space Complexity

Space complexity measures **how much memory an algorithm requires** relative to input size n . It includes:

- Input space:** Memory needed to store inputs
- Auxiliary space:** Temporary space used by algorithm
- Function/recursion stack:** Memory for recursion calls

$S(n) = S_{\text{input}} + S_{\text{auxiliary}} + S_{\text{stack}}$

Why Space Complexity Matters

- Helps predict **memory requirements** for large datasets.
- Prevents **memory overflows** in HPC and parallel systems.
- Guides algorithm selection for memory-constrained systems.

8

Space Complexity

Linear Search: Only a few variables used for index and element comparison.

$$S(n) = O(1) \text{ (constant space)}$$

Merge Sort: Uses an auxiliary array for merging.

$$S(n) = O(n) \text{ (extra array of size } n\text{)}$$

Quick Sort (in-place): Recursion stack used: log n depth on average

$$S(n) = O(\log n) \text{ (average case)}$$

Matrix Multiplication: For two $n \times n$ matrices:

$$S(n) = O(n^2) \text{ (stores output matrix)}$$

9

Throughput

Throughput is the **number of tasks or operations completed per unit time**. It measures **system-wide productivity**.

$$\text{Throughput} = \frac{\text{Number of Tasks Completed}}{\text{Total Time}}$$

Example (Sequential System):

- CPU processes 1 task every 0.25 sec
- For 100 tasks:

$$\text{Throughput} = \frac{100}{100 \times 0.25} = 4 \text{ tasks/sec}$$

In Parallel Systems:

- Throughput can increase with **more cores/processors**
- Ideal case (linear scaling):

$$\text{Throughput}_{\text{parallel}} = N \times \text{Throughput}_{\text{single core}}$$

where N = number of cores

11

Space Complexity

If an algorithm uses:

- k scalar variables $\rightarrow$ each takes constant space $\rightarrow$ total $O(k)$
- n input elements $\rightarrow$ space grows with input size $\rightarrow O(n)$
- Recursion depth $d \rightarrow$ each call uses stack space $\rightarrow$ total $O(d)$

Then:

$$S(n) = O(n + k + d)$$

All these memory usages happen together, so we **add them**. In Big-O, we keep the dominant growing term. If **k is constant** and **$d \leq n$** , then:

$$S(n) = O(n)$$

because **n grows fastest and dominates the total space**.

10

Latency

Latency is the **time taken to complete a single task from start to finish**.

$$\text{Latency} = \text{Finish Time} - \text{Start Time}$$

Example:

- Task A starts at 0 sec, finishes at 2 sec $\rightarrow$ Latency = 2 sec
- Task B starts at 1 sec, finishes at 3 sec $\rightarrow$ Latency = 2 sec

Important Note:

- Low latency $\neq$ high throughput (especially in parallel systems)
- Example: Batch processing can increase throughput but individual task latency may remain high

12

Relationship Between Throughput and Latency

Little's Law (Queueing Theory)

$$L = \lambda \cdot W$$

Where:

- L = Average number of tasks in the system (system load)
- λ = Arrival rate (tasks/sec) → closely related to **throughput**
- W = Average time spent in the system → **latency**

Rearranging:

$$W = \frac{L}{\lambda}$$

- If arrival rate λ increases
- But system resources (CPU, memory, cores) remain fixed

As tasks begin to queue, the load (L) increases, which in turn increases the latency (W).

13

Core Performance Metrics

$$\text{Throughput} = \frac{\text{Tasks Completed}}{\text{Total Time}} \quad (\text{tasks/sec})$$

$$\text{Latency} = \text{Task Completion Time} - \text{Task Arrival Time} \quad (\text{seconds})$$

$$\text{Efficiency} = \frac{\text{Throughput}_{\text{actual}}}{\text{Throughput}_{\text{ideal}}}$$

In parallel systems:

$$\text{Efficiency} = \frac{\text{Speedup}}{p}$$

Where:

$$\text{Speedup} = \frac{T_1}{T_p}$$

- T_1 = execution time on 1 processor
- T_p = execution time on p processors
- p = number of processors

15

Relationship Between Throughput and Latency

Increasing throughput does **not automatically** reduce latency.

If system utilization approaches 100%:

- Queue length grows rapidly
- Waiting time increases
- Latency increases nonlinearly

In many queue models (e.g., M/M/1):

$$W = \frac{1}{\mu - \lambda}$$

Where:

- μ = service rate
- As $\lambda \rightarrow \mu$

$$W \rightarrow \infty$$

So latency grows dramatically near saturation.

14

Speedup

Speedup measures how much faster a parallel system performs compared to serial execution.

$$S = \frac{T_{\text{serial}}}{T_{\text{parallel}}}$$

Where:

- T_{serial} = execution time on **one processor**
- T_{parallel} = execution time on **N processors**

Example

- Single-core time: $T_{\text{serial}} = 100$ sec
- 4-core time: $T_{\text{parallel}} = 30$ sec

16

Speedup

- The system is **3.33 times faster** using 4 processors.
- Ideal speedup for 4 processors would be:

$$S_{ideal} = 4$$

Since $3.33 < 4$, the system has some overhead (communication, synchronization, imbalance).

Important Insight

$$S \leq N$$

- If $S = N \rightarrow$ Perfect linear speedup
- If $S < N \rightarrow$ Parallel overhead exists
- If $S > N \rightarrow$ Superlinear speedup (rare, due to cache effects)

17

Amdahl's Law

- The **sequential part (1 - P)** must always execute on a single processor. It cannot be reduced by adding more processors.

Example:

If:

- $P = 0.9$ (90% parallelizable)

Then:

$$S_{max} = 1 / 0.1 = 10$$

Even with unlimited processors, speedup cannot exceed 10.

- The **parallel part (P)** gets divided across N processors, so its time reduces as N increases.

- However, even if $N \rightarrow \infty$:

$$S_{max} = \frac{1}{1 - P}$$

This means the **sequential fraction limits the maximum speedup**.

19

Amdahl's Law

Amdahl's Law predicts the **maximum theoretical speedup** of a program when only part of it can be parallelized.

Parallel portion time (divided among N processors):

$$\frac{PT_{serial}}{N}$$

Assumptions

- P = fraction of program that can run in parallel
- $1 - P$ = fraction that must run sequentially
- N = number of processors

So total parallel time:

$$T_{parallel} = T_{serial} \left[(1 - P) + \frac{P}{N} \right]$$

Parallel Execution Time

Total serial time = T_{serial}

Sequential portion time:

$$(1 - P)T_{serial}$$

Speedup

$$S(N) = \frac{T_{serial}}{T_{parallel}} = \frac{1}{(1 - P) + \frac{P}{N}}$$

18

Amdahl's Law

Given:

- Parallel fraction: $P = 0.8$
- Sequential fraction: $1 - P = 0.2$
- Number of processors: $N = 4$

Even though 4 processors are used, the speedup is only **2.5x**, not 4x

Apply Amdahl's Law

Why?

$$S(N) = \frac{1}{(1 - P) + \frac{P}{N}}$$

Substitute values:

$$S(4) = \frac{1}{0.2 + \frac{0.8}{4}}$$

$$S(4) = \frac{1}{0.2 + 0.2}$$

$$S(4) = \frac{1}{0.4}$$

$$S(4) = 2.5$$

- The 20% sequential portion must still run on one processor.
- Only the 80% portion benefits from parallel execution.
- This sequential part becomes a limiting factor.

20

Amdahl's Law

If total serial time = 100 sec:

- Sequential part (20%) = 20 sec
- Parallel part (80%) = 80 sec

With 4 processors:

$$\text{Parallel time: } 80 / 4 = 20 \text{ sec}$$

$$\text{Total parallel execution time: } 20 + 20 = 40 \text{ sec}$$

$$\text{Speedup: } 100 / 40 = 2.5$$

The sequential portion $(1-P)$ restricts scalability. Adding more processors improves performance, but the improvement is limited by the non-parallelizable part of the program.

21

Amdahl's Law

Original time = 200 minutes

Speedup ≈ 2.58

New execution time:

$$T_{\text{parallel}} = \frac{200}{2.58} \approx 77.5 \text{ minutes}$$

Why Not 8x Speedup?

Even though 8 processors are used:

- The 70% rendering part is divided across 8 cores.
- But the 30% encoding part must still run sequentially.

23

Amdahl's Law

Suppose a video rendering task takes **200 minutes** on a single processor.

Apply Amdahl's Law

- 70% of the work (frame rendering) can be parallelized

$$P = 0.7$$

- 30% (final encoding + synchronization) is sequential

$$1 - P = 0.3$$

- Number of processors:

$$N = 8$$

$$S(8) = \frac{1}{(1 - P) + \frac{P}{N}}$$

$$S(8) = \frac{1}{0.3 + \frac{0.7}{8}}$$

$$S(8) = \frac{1}{0.3 + 0.0875}$$

$$S(8) = \frac{1}{0.3875}$$

$$S(8) \approx 2.58$$

22

Amdahl's Law

Sequential time:

$$0.3 \times 200 = 60 \text{ minutes}$$

That 60 minutes cannot be reduced by adding more processors.

In real systems like video rendering:

- Parallel tasks (frame processing) scale well.
- Sequential tasks (final merge, encoding) limit total speedup.

So even with powerful HPC clusters, performance is bounded by the sequential bottleneck.

This is exactly what Amdahl's Law predicts.

24

Maximum Speedup

We know:

$$S(N) = \frac{1}{(1-P) + \frac{P}{N}}$$

To find the **maximum possible speedup**, take the limit as the number of processors becomes very large:

$$\lim_{N \rightarrow \infty} S(N)$$

Apply the Limit

As:

$$N \rightarrow \infty$$

The term:

$$\frac{P}{N} \rightarrow 0$$

So the speedup formula becomes:

$$S_{max} = \frac{1}{(1-P)}$$

25

Efficiency

Efficiency measures how well the processors are utilized during parallel execution.

Using Amdahl's Law

$$E(N) = \frac{S(N)}{N}$$

Where:

- $S(N)$ = speedup with N processors
- N = number of processors

We know:

$$S(N) = \frac{1}{(1-P) + \frac{P}{N}}$$

Substitute into efficiency:

$$E(N) = \frac{1}{N[(1-P) + \frac{P}{N}]}$$

Simplify:

$$E(N) = \frac{1}{N(1-P) + P}$$

It tells us what fraction of each processor's capacity is effectively used.

27

Maximum Speedup

Suppose total serial execution time = 100 seconds.

- Parallel part (80%) = 80 sec
- Sequential part (20%) = 20 sec

If we use infinite processors:

- Parallel part becomes ≈ 0 sec
- Sequential part remains 20 sec

So total time ≈ 20 sec.

Speedup: $100 / 20 = 5$

Even with unlimited processors, the execution time **cannot go below 20 seconds**, because that portion cannot be parallelized.

26

Efficiency

Given:

- $P = 0.8$
- $N = 4$

Previously calculated speedup:

$$S(4) = 2.5$$

Efficiency:

- $E(4) = 2.54 / 4$
- $E(4) = 0.625$
- $E(4) = 62.5\%$

Although 4 processors are used:

- The system achieves only $2.5\times$ speedup.
- So each processor is effectively utilized at 62.5%.

Although 4 processors are used:

- The system achieves only $2.5\times$ speedup.
- So each processor is effectively utilized at 62.5%.
- The remaining 37.5% is wasted due to the sequential part $(1-P)$ of the code.

Why Efficiency < 1

- Sequential fraction of the program $(1-P = 0.2)$ cannot be parallelized.
- Adding more processors **does not speed up sequential tasks**, so efficiency drops.
- With more processors, efficiency **keeps decreasing**, illustrating **diminishing returns**.

28

Efficiency

Why Efficiency Decreases as N Increases

From:

$$E(N) = \frac{1}{N(1-P) + P}$$

As N increases:

- The term $N(1-P)$ increases.
- Denominator increases.
- So efficiency decreases.

Unless:

$P \rightarrow 1$

Meaning the program is almost fully parallelizable

29



BITS Pilani

1. *Data-level parallelism*
2. *Task-level parallelism*
3. *Algorithm-specific parallelism*

31



BITS Pilani

Parallelization Paradigms

30

Parallelization

Parallelization is the decomposition of a computational workload into independent components that can execute concurrently on N processors.

Let T_{serial} be the total execution time of a task on a single processor.

If a fraction P of the task is parallelizable and is distributed across N processors, then the parallel execution time is:

$$T_{\text{parallel}} = (1-P)T_{\text{serial}} + \frac{PT_{\text{serial}}}{N}$$

Ideal Case (No Sequential Part, No Overhead)

If the entire task is parallelizable ($P=1$): $T_{\text{parallel}} = \frac{T_{\text{serial}}}{N}$

Speedup: $S(N) = \frac{T_{\text{serial}}}{T_{\text{parallel}}} = N$

This is called linear speedup

32

Parallelization

General Case (With Overhead)

Including communication and synchronization overhead T_{overhead} :

$$T_{\text{parallel}} = (1 - P)T_{\text{serial}} + \frac{PT_{\text{serial}}}{N} + T_{\text{overhead}}$$

Objective of Parallelization

Minimize:

$$T_{\text{parallel}}$$

Maximize:

$$S(N) = \frac{T_{\text{serial}}}{T_{\text{parallel}}}$$

and maintain high efficiency:

$$E(N) = \frac{S(N)}{N}$$

33

Data-Level Parallelism (DLP)

Data-Level Parallelism refers to performing the **same operation** on multiple data elements simultaneously.

It is applicable when:

- The operation on each data element is **independent**
- No inter-element dependency exists
- Data can be partitioned evenly

Given two arrays A and B of size n:

$$C[i] = A[i] + B[i], i = 1, 2, \dots, n$$

Each computation of $C[i]$ is independent of others. So the total serial execution time:

$$T_{\text{serial}} = n \cdot t_{\text{op}}$$

Where:

- t_{op} = time for one addition

35

Parallelization

Parallelization attempts to transform a computation of cost:

$$O(T)$$

into:

$$O(T/N)$$

subject to limits imposed by sequential fractions and system overhead

34

Data-Level Parallelism (DLP)

Parallel Execution

If N processors are available and work is evenly distributed:

$$T_{\text{parallel}} = \frac{n}{N} \cdot t_{\text{op}}$$

Speedup:

$$S(N) = \frac{T_{\text{serial}}}{T_{\text{parallel}}} = \frac{nt_{\text{op}}}{\frac{n}{N}t_{\text{op}}} = N$$

36

Data-Level Parallelism (DLP)

Maximum Speedup

If $N = n$ (one processor per element):

$$T_{\text{parallel}} = t_{\text{op}}$$

$$S_{\text{max}} = n$$

In real systems:

- There is communication overhead
- Memory access delays occur
- Processors may not be perfectly synchronized

So practical speedup is slightly less than n .

Thus we write:

$$S_{\text{max}} \approx n$$

meaning **approximately linear speedup**.

37

Data-Level Parallelism (DLP)

Image Processing

Applying a filter (e.g., blur, edge detection) to each pixel can be done independently. Each processor/thread handles a subset of pixels simultaneously.

Vector Addition / Matrix Operations

Operations such as:

$$C[i] = A[i] + B[i]$$

can be computed independently for each index i . Similarly, matrix multiplication distributes row-column computations across multiple processors.

GPU Computations

Modern GPUs such as those using **CUDA architecture by NVIDIA** allow thousands of cores to execute threads in parallel.

For example:

- Each CUDA core handles one data element.
- Large-scale computations (deep learning, simulations, rendering) achieve massive speedup.

39

Data-Level Parallelism (DLP)

Computational Complexity View

Serial Complexity

$$\text{Serial Time Complexity} = O(n)$$

Parallel Complexity (Ideal Case, No Overhead)

$$\text{Parallel Time Complexity} = O(n / N)$$

Where:

- n = problem size
- N = number of processors

If $N = n$

$$O(n / n) = 1$$

This represents **constant time complexity** in the ideal case (perfect parallelism, no communication overhead).

38

Task-Level Parallelism (TLP)

Task-Level Parallelism (TLP) refers to executing **multiple independent tasks simultaneously** instead of executing them sequentially.

- Each task performs a different operation.
- Tasks may work on the same dataset or different datasets.
- The goal is to reduce overall completion time by overlapping execution.

Unlike data parallelism (where the same operation is applied to different data elements), TLP focuses on parallelizing **different functions or modules**.

40

Task-Level Parallelism (TLP)

In TLP, we divide the program into **independent tasks**:

$$T_1, T_2, \dots, T_m$$

Each task performs a logically different unit of work.

Unlike data parallelism:

- Tasks may operate on **different data**
- Tasks may even execute **different algorithms**

Sequential Execution Time

If executed sequentially:

$$T_{\text{serial}} = \sum_{i=1}^m t_i$$

- $t_i \rightarrow$ execution time of task T_i
- Summation means tasks execute **one after another**
- Total time is additive

41

Task-Level Parallelism (TLP)

Parallel Execution Time

Now assume we have **m processors**, and each task runs on one processor.

All tasks start at the same time.

The program finishes when the **last task finishes**.

So:

$$T_{\text{parallel}} \approx \max(t_1, t_2, \dots, t_m)$$

Task	Time
T_1	5 sec
T_2	3 sec
T_3	2 sec

All start together.

- T_3 finishes at 2 sec
- T_2 finishes at 3 sec
- T_1 finishes at 5 sec

System completes at 5 sec. That is the **maximum time**.

*This is called the **critical path time**.*

43

Task-Level Parallelism (TLP)

Assume we have m tasks:

$$T_1, T_2, \dots, T_m$$

Each task takes time:

$$t_1, t_2, \dots, t_m$$

Since serial execution runs tasks one by one:

$$T_{\text{serial}} = t_1 + t_2 + \dots + t_m$$

Then,

$$T_{\text{serial}} = \sum_{i=1}^m t_i$$

Because:

- Task 2 starts only after Task 1 finishes.
- Task 3 starts only after Task 2 finishes.
- No overlap happens.

*So total time is the **sum of all task durations**.*

42

Speedup in Task-Level Parallelism (TLP)

Speedup measures how much faster the parallel system is compared to serial execution.

If:

$$S = \frac{T_{\text{serial}}}{T_{\text{parallel}}}$$

$$T_{\text{serial}} = 100 \text{ seconds}$$

$$T_{\text{parallel}} = 20 \text{ seconds}$$

- $T_{\text{serial}} \rightarrow$ Total time taken when tasks are executed one after another.
- $T_{\text{parallel}} \rightarrow$ Total time taken when tasks are executed simultaneously.

Then:

$$S = 100 / 20 = 5$$

Why Serial divided by Parallel?: Because we want to measure improvement.

$$\text{Speedup} = \frac{\text{Old Time}}{\text{New Time}}$$

- Old time $\rightarrow$ Serial time
- New time $\rightarrow$ Parallel time

*The parallel system is **5 times faster**.*

44

Load Balance Ratio in Task-Level Parallelism (TLP)

Average Task Time

$$\bar{t} = \frac{1}{m} \sum_{i=1}^m t_i$$

- $m \rightarrow$ number of tasks
- $t_i \rightarrow$ execution time of task i
- $\bar{t} \rightarrow$ average execution time per task

If total work is divided equally among tasks, each task should ideally take $\bar{t}$ time.

It represents the ideal balanced workload per processor.

45

Task-Level Parallelism (TLP)

When Number of Tasks $\neq$ Number of Processors

m = number of tasks

p = number of processors

If $m > p \rightarrow$ All tasks cannot execute simultaneously, so tasks must be **scheduled**

Parallel Time with Scheduling

$$T_{\text{parallel}} = \max_{j=1}^p \left(\sum t_i \text{ assigned to processor } j \right)$$

- Each processor gets multiple tasks
- Total time per processor = sum of assigned task times
- Overall completion time = maximum workload among processors

When $m > p$, performance depends more on intelligent scheduling than on the number of processors.

47

Load Balance Ratio in Task-Level Parallelism (TLP)

Ideal Parallel Time

$$T_{\text{ideal}} = \bar{t}$$

If:

- All tasks are perfectly balanced
- Each processor gets equal work

Then every task finishes at the same time.

So total completion time = average task time.

This is the **best possible parallel time**
(assuming no overhead).

Actual Parallel Time

$$T_{\text{parallel}} = \max(t_i)$$

Even if most tasks are small:

- The slowest task determines when the system finishes.
- Faster processors must wait for the slowest one.

This is the **bottleneck effect**.

46

Task-Level Parallelism (TLP)

Adding Overhead

$$T_{\text{parallel}} = \max(t_i) + T_{\text{overhead}}$$

- $\max(t_i) \rightarrow$ Longest task (critical path)
- $T_{\text{overhead}} \rightarrow$ Extra system costs

Overhead Includes

- Communication delay
- Synchronization (locks / barriers)
- Task or thread creation
- Context switching

Speedup with Overhead

$$S = \frac{\sum t_i}{\max(t_i) + T_{\text{overhead}}}$$

48

Algorithm-Specific Parallelism (ASP)

Algorithm-Specific Parallelism refers to designing **parallel algorithms tailored to the mathematical structure of a specific problem**.

- Unlike generic data-level or task-level parallelism, ASP **exploits the algorithm's inherent recursion, decomposition, or structure** for concurrency.
- Often used in **scientific computing, numerical methods, and signal processing**.

Instead of simply dividing tasks or data, we analyze:

- Recurrence relations
- Decomposition structure
- Algebraic independence
- Divide-and-conquer properties

*Parallelism is extracted from the **theory of the algorithm itself**.*

49

Parallel Sorting

Suppose $N=2$, array size $n=8$:

- Divide $\rightarrow$ 2 subarrays of size 4, sorted **concurrently**
- Each subarray further divides $\rightarrow$ recursion continues
- Merge step at each level takes $O(n)$

Parallelization reduces the **depth of recursion** execution time

Speedup is limited by the **merge step** (sequential part)

This is an example of **Amdahl's law applied recursively**.

51

Parallel Sorting

Serial Merge Sort

Recurrence relation:

$$T_{\text{serial}}(n) = 2 T(n/2) + O(n)$$

- Divide array in two halves
- Sort each half sequentially
- Merge sorted halves in $O(n)$ time

Parallel Merge Sort

If two halves are sorted **simultaneously on 2 processors**:

$$\text{Where: } T_{\text{parallel}}(n) = \frac{2 T(n/2)}{N} + O(n)$$

- N = number of processors used for the recursive step
- Merge step still requires $O(n)$ time

50

Matrix Multiplication

$$C = A \cdot B$$

Divide A and B into submatrices (blocks)

Compute block products independently:

$$C_{ij} = \sum_k A_{ik} B_{kj}$$

Each block multiplication is independent $\rightarrow$ execute in parallel

Strassen Algorithm (Parallel): Strassen reduces number of multiplications using recursion

$$T_{\text{serial}}(n) = 7 T(n/2) + O(n^2)$$

Parallel form:

$$T_{\text{parallel}}(n) = \frac{7 T(n/2)}{N} + O(n^2)$$

- N processors handle recursive multiplications simultaneously
- Significant speedup, especially on GPUs

52

Algorithm-Specific Parallelism (ASP)

Most algorithm-specific parallel algorithms follow:

$$T_{\text{parallel}}(n) = \frac{\text{Recursive Subproblems}}{N} + T_{\text{combine}}$$

Where:

- Recursive subproblems → can be executed concurrently
- T_{combine} → sequential portion for merging or summing results

Speedup:

$$S = \frac{T_{\text{serial}}}{T_{\text{parallel}}} \leq \frac{T_{\text{serial}}}{T_{\text{combine}} + T_{\text{parallel-subtasks}}}$$

- Limited by **sequential combine step** (Amdahl's Law)

53

Parallelization Paradigms

Paradigm	Concept	Example	Hardware Best Suited
Data-Level Parallelism (DLP)	Same operation on multiple data elements	Vector addition, image filter	GPUs, SIMD units
Task-Level Parallelism (TLP)	Different tasks executed concurrently	Web server requests, pipeline	Multi-core CPUs, clusters
Algorithm-Specific	Parallel algorithms for problem structure	Parallel merge sort, FFT	CPU cores, GPUs, clusters

55

Algorithm-Specific Parallelism (ASP)

Algorithm-specific parallelism **maximizes concurrency** by exploiting the problem's structure

Algorithm	Parallel Strategy
Merge Sort	Sort halves concurrently
FFT	Recursive even-odd decomposition
Matrix Multiplication	Block multiplication / Strassen
Numerical PDE Solvers	Domain decomposition

Efficiency depends on:

- Number of processors N
- Sequential combine time
- Task granularity

54



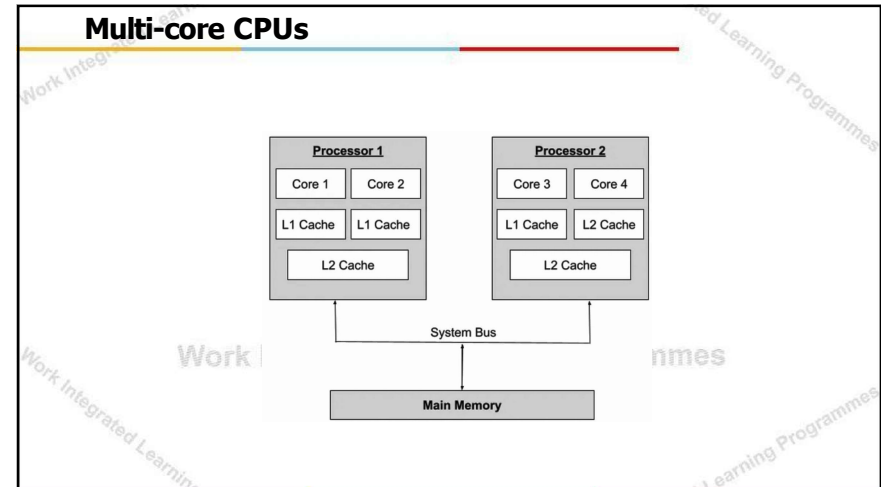
56



BITS Pilani

1. Multi-core CPUs
2. GPGPUs
3. GPU/CPU Clusters
4. Resource Management

57



59

Multi-core CPUs

A multi-core CPU integrates multiple independent processing cores on a single chip.

Each Core Contains:

- Private registers
- ALU (Arithmetic Logic Unit)
- Often private L1 cache
- Shared L2/L3 cache
- Shared main memory (RAM)

Parallel Model → Shared Memory Model

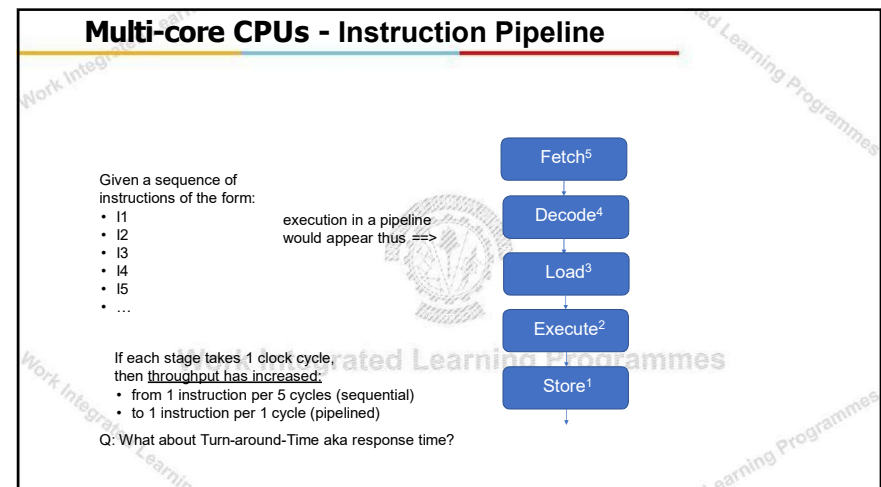
All cores access the same global address space:

$$M_{global} \rightarrow \{Core_1, Core_2, \dots, Core_N\}$$

This enables:

- Thread-based programming (e.g., OpenMP, Pthreads)
- Fast inter-thread communication (via shared variables)

58



60

Executing an Instruction

- Different stages of Instruction Execution:

Fetch Instruction	Move the next instruction (tracked by PC) to a register
Decode Instruction	Identify Operator and (data) addresses
Load (data)	Move data into register (if needed)
Execute	Perform the operation
Store (result)	Move the resulting data to memory

If separate circuitry is designed for each stage - so that the stage take the same amount of time - then a sequence of instructions can be executed in a pipeline.

61

MIMD – Multiple Instruction, Multiple Data

Each processor or core can execute **different instructions** on **different data**

Key Features:

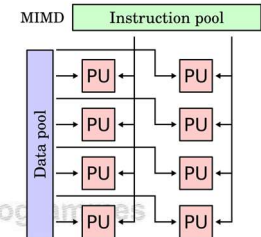
- Flexible and versatile execution model
- Ideal for task-parallel workloads

Examples of Use:

- Operating systems
- Web servers
- Multithreaded applications

Hardware Support:

- Multicore CPUs (Intel, AMD)
- Some heterogeneous compute platforms (e.g., ARM big.LITTLE)



63

SIMD – Single Instruction, Multiple Data

Executes the **same instruction** across multiple data elements

Key Features:

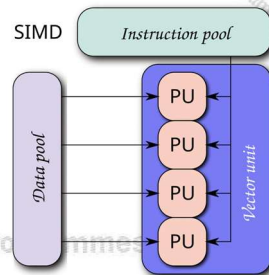
- Suitable for **data-parallel** applications
- Lockstep execution – all processing elements perform same operation simultaneously

Examples of Use:

- Vector and matrix operations
- Image filters, audio processing
- Scientific computation

Hardware Support:

- Intel AVX, ARM NEON, NVIDIA CUDA (warps/wavefronts)



62

SIMD vs. MIMD – Key Differences

Aspect	SIMD	MIMD
Instruction Stream	Single	Multiple
Data Stream	Multiple	Multiple
Synchronization	High (lockstep)	Low (independent threads)
Control Flow	Less flexible	Highly flexible
Hardware Examples	GPUs, Vector Units	Multicore CPUs, Clusters
Use Cases	Matrix ops, signal processing	Web servers, OS-level tasks
Efficiency	High for regular data structures	High for diverse workloads

64

Performance Model

If a program has:

- Parallel fraction = P
- Number of cores = N

Then theoretical speedup:

$$S(N) = \frac{1}{(1-P) + \frac{P}{N}}$$

- Barriers $(1-P) \rightarrow$ Sequential part (cannot be parallelized)
- $P/N \rightarrow$ Parallel part divided across N cores

$$T_{\text{parallel}} = T_{\text{compute}} + T_{\text{synchronization}} + T_{\text{memory contention}}$$

T_{compute} : Actual useful computation time.

$T_{\text{synchronization}}$: Time spent:

- Waiting for other threads
- Barriers
- Locks / coordination

T_{memory} : Time lost due to:

- Shared memory access conflicts
- Cache coherence traffic
- Limited memory bandwidth

65

Multi-core CPUs

Multi-core CPUs are effective for:

- Task-level parallelism
- Moderate data parallelism
- Branch-heavy computations
- General-purpose workloads

Multi-core performance is determined by:

$$\text{Performance} = \text{Parallel Speedup} - \text{Synchronization} + \text{Memory Overhead}$$

Even with many cores, scalability depends on:

- Parallel fraction P
- Synchronization cost
- Memory behavior
- Cache efficiency

67

Memory Hierarchy Impact

CPU performance depends heavily on **cache behavior**.

Access times:

$$T_{L1} < T_{L2} < T_{L3} < T_{RAM}$$

Since RAM access is much slower, cache performance dominates CPU speed

Effective memory access time (EMAT):

$$EMAT = H \cdot T_{\text{cache}} + (1 - H) \cdot T_{\text{memory}}$$

Where:

- H = cache hit rate
- T_{cache} = cache access time
- T_{memory} = RAM access time

Higher $H \rightarrow$ lower average memory delay $\rightarrow$ better CPU performance.

66

GPGPUs

A GPGPU is designed for **massive parallel throughput computing**.

It contains:

- Thousands of lightweight cores
- Organized into **Streaming Multiprocessors (SMs)**
- Each SM executes many threads concurrently

CPU	GPU
Few powerful cores	Thousands of simpler cores
Optimized for low latency	Optimized for high throughput
Complex control logic	Minimal control logic
Large cache	Smaller cache, higher bandwidth

68

Execution Model – SIMT

SIMT - Single Instruction, Multiple Threads

One instruction is executed by many threads simultaneously.
Example:

$$C[i] = A[i] + B[i] \quad C[i]$$

If:

- n elements
- T threads

Each thread computes one index independently. Since operations are independent, they execute in parallel.

This model is ideal when:

- Same operation
- Different data elements

69

GPU Memory Hierarchy

GPU performance strongly depends on memory behavior.

Memory Types:

- **Registers (Fastest):** Private to each thread.
- **Shared Memory (Fast):**
 - Shared among threads within a block.
 - Low latency, programmer-controlled.
- **Global Memory (Slow):**
 - Accessible by all threads.
 - High latency (hundreds of cycles).

71

Memory Hierarchy in Multicore & GPGPU

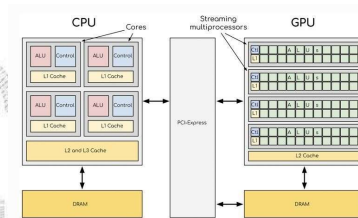
Purpose: Bridge speed gap between fast processors and slow memory

Hierarchy Levels:

- **Registers:** Fastest, smallest, processor-specific
- **L1, L2, L3 Cache:** On-chip, close to core, progressively larger and slower
- **Main Memory (RAM):** Shared DRAM, slower than cache
- **Disk/SSD:** Persistent storage, very high latency

GPU-Specific:

- **Shared Memory:** Fast, limited, user-controlled
- **Global Memory:** Accessible by all threads, high latency
- **Constant/Texture Memory:** Specialized, cached memory for specific use cases



70

GPU Memory Hierarchy

Performance Depends On:

Memory Coalescing

Adjacent threads should access adjacent memory locations:

Coalesced Access ⇒ Fewer memory transactions

Minimizing Global Memory Access

Use shared memory to reduce global access.

Maximizing Occupancy

Occupancy = Active threads / Maximum supported threads

Higher occupancy hides memory latency.

72

GPU Memory Hierarchy

GPU performance is dominated by:

$$\text{Performance} = \text{Massive Thread Parallelism} + \text{High Memory Bandwidth}$$

Unlike CPUs (latency-optimized), GPUs are:

Throughput-Optimized Processors

They perform best when:

- Large data size n
- High parallel fraction P
- Minimal branching
- Memory access is well-structured

Optimizing Memory Performance

For Multicore CPUs:

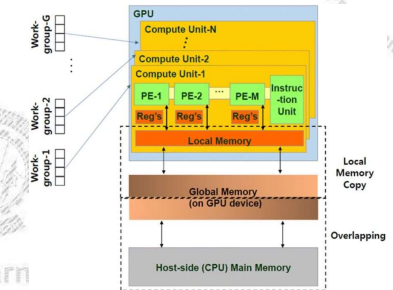
- Use **locality of reference** to benefit from caches
- Avoid **false sharing** – separate data written by different threads
- Use cache-aware data structures

For GPGPU:

- Align memory access for coalescing
- Utilize **shared memory** to avoid global memory latency
- Minimize **bank conflicts** in shared memory

General Strategies:

- Balance **compute vs memory** loads
- Use **pipelining and tiling** techniques



73

75

Memory Bandwidth Considerations

Memory Bandwidth: Maximum rate at which data can be transferred between memory and processors

Why Important?

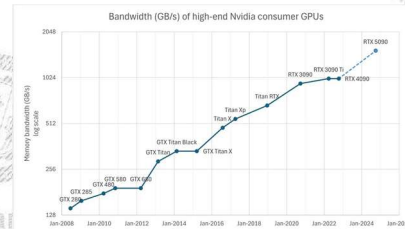
High computation speeds are meaningless without fast data access

Multicore CPUs:

- Shared bandwidth among cores → contention possible
- Cache reduces memory access latency

GPGPUs:

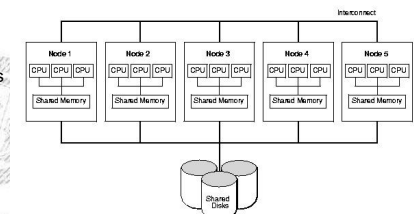
- Very high theoretical memory bandwidth
- Sensitive to **access patterns**
- **Coalesced access** (threads accessing adjacent memory) is critical for performance



74

CPU-only Clusters

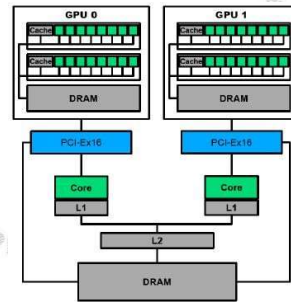
- Comprised solely of **multicore CPU-based nodes**.
- Suitable for:
 - Applications with control-heavy logic
 - Large in-memory databases
 - Distributed file systems and ETL pipelines
- **Benefits:**
 - More memory per node (better for large datasets)
 - Well-supported by traditional distributed frameworks (e.g., Hadoop, Spark, MPI)
 - Easier to manage and deploy
- **Limitations:**
 - Lower computational throughput
 - Slower performance in compute-intensive tasks like deep learning



76

GPU-Accelerated Clusters

- Consist of nodes with both CPUs and **one or more GPUs** (NVIDIA, AMD).
- Ideal for:
 - Training large-scale deep learning models (CNNs, Transformers)
 - Image/video processing, simulations
- Benefits:**
 - Massive parallelism using thousands of CUDA cores
 - Faster matrix operations (e.g., GEMM, convolutions)
 - High floating-point throughput (TFLOPs range)
- Limitations:**
 - Lower memory per GPU
 - Requires GPU-compatible libraries (e.g., TensorFlow GPU, CUDA, NCCL)
 - Higher hardware and maintenance cost



77

GPU/CPU Clusters

A **cluster** is a collection of interconnected computing nodes.

Each node contains:

- One or more **multi-core CPUs**
- Possibly one or more **GPUs**
- Its own **local memory (RAM)**

Nodes are connected using a **high-speed network** (e.g., InfiniBand, Ethernet).

Distributed Memory Model

Unlike shared-memory systems:

Each node has its **own private memory**.
To share data:

Node_i → Node_j
Data must be explicitly communicated over the network.

79

CPU-only vs GPU-Accelerated Clusters

Feature	CPU-only Clusters	GPU-Accelerated Clusters
Compute Units	Multicore CPUs	CPUs + Dedicated GPUs
Memory per Node	High	Lower (per GPU)
Programming Model	Threading, MPI	CUDA, OpenCL, TensorFlow, PyTorch
Power Consumption	Lower	Higher
Ideal Use Cases	Control-heavy tasks, I/O-bound jobs	Deep learning, simulations, HPC
Cost	Lower	Higher (GPUs, cooling systems, etc.)

78

GPU/CPU Clusters

Communication Model

Total execution time:

$$T_{total} = T_{computation} + T_{communication}$$

α - β Communication Model

Communication cost is modeled as:

$$T_{comm} = \alpha + \beta m$$

Where:

- α = latency (startup cost per message)
- β = transfer time per byte
- m = message size (bytes)

80

GPU/CPU Clusters

Scalability Models

Strong Scaling

- Problem size fixed
- Increase number of processors N

Ideal case:

$$T(N) = \frac{T(1)}{N}$$

Speedup:

$$S(N) = N$$

But in reality:

$$T(N) = \frac{T(1)}{N} + T_{comm}$$

Communication overhead prevents perfect linear scaling.

81

GPU/CPU Clusters

Performance Limitation

Cluster efficiency:

$$E(N) = \frac{S(N)}{N}$$

Where:

- $E(N)$ = parallel efficiency
- $S(N)$ = speedup using N nodes
- N = number of processors/nodes

As N increases:

- Communication increases
- Synchronization increases
- Network contention increases

Thus: $E(N) \downarrow$ as $N \uparrow$

83

GPU/CPU Clusters

Scalability Models

Weak Scaling

- Increase processors
- Increase problem size proportionally

If problem size per processor remains constant:

$$T(N) \approx T(1)$$

Ideal weak scaling maintains constant execution time.

However:

$$T(N) = T_{comp} + T_{comm}$$

As N increases, communication may grow and degrade performance.

82

GPU/CPU Clusters

For distributed GPU/CPU clusters, overall performance can be modeled as:

Performance = $f(\text{Compute Capacity, Memory Hierarchy, Communication Cost, Load Balancing})$

This represents the **key determinants of scalability and efficiency** in cluster-based systems.

Total compute power across all nodes:

$$C_{total} = N_{nodes} \times (CPU_{cores} + GPU_{cores})$$

Where:

- N_{nodes} = number of cluster nodes
- CPU_{cores} = processing power per node (CPU)
- GPU_{cores} = GPU compute power per node

Higher compute capacity increases raw throughput.

84

GPU/CPU Clusters

Memory Hierarchy

Each node has:

- Local cache
- Local RAM
- GPU memory

Memory performance impacts:

$$T_{compute} \propto EMAT$$

Effective memory access time:

$$EMAT = H \cdot T_{cache} + (1 - H) \cdot T_{memory}$$

Where:

- EMAT = Effective Memory Access Time
- H = Cache hit rate
- T_{cache} = Cache access time
- T_{memory} = Main memory (RAM) access time

Poor locality increases computation time.

85

GPU/CPU Clusters

Load Balancing

Let:

$$W_i = \text{work assigned to node}$$

Parallel execution time:

$$T_{parallel} = \max(W_1, W_2, \dots, W_N)$$

If workload is uneven:

$$T_{parallel} \uparrow \Rightarrow \text{Efficiency} \downarrow$$

Combined Cluster Performance Model

$$T_{total} = \frac{T_{compute}}{N} + T_{comm} + T_{imbalance}$$

Where:

$T_{imbalance}$ = delay due to uneven work distribution

87

GPU/CPU Clusters

Communication Cost

Since clusters use **distributed memory**, communication cost is critical.

$$T_{comm} = \alpha + \beta m$$

Where:

- α = latency
- β = per-byte transfer cost
- m = message size

Total execution time:

$$T_{total} = T_{compute} + T_{comm}$$

If communication dominates:

$$\text{Scalability} \downarrow$$

86

GPU/CPU Clusters

Performance improves when:

- Compute capacity $\uparrow$
- Memory latency $\downarrow$
- Communication cost $\downarrow$
- Workload evenly distributed

Cluster performance is limited not just by computation, but by **communication and imbalance overhead**, especially at large scale.

88

Resource Management

Modern GPU/CPU clusters must efficiently manage:

- CPU cores → **(Multi-core CPUs)**
- GPU threads → **(GPUs / Accelerators)**
- Memory → **(All systems: CPU, GPU, Clusters)**
- Network bandwidth → **(Clusters / Distributed systems)**
- Storage I/O → **(Data centers / HPC systems)**

Poor resource management directly reduces scalability and efficiency.

89

Resource Management - Scheduling

- Operating Systems (CPU scheduling)
- GPU thread schedulers
- Cluster job schedulers

$$T_{\text{completion}} = \max(\text{Finish time of all tasks})$$

Schedulers aim to:

- Reduce idle processors
- Reduce context switching
- Reduce communication delay (clusters)

91

Resource Management - Load Balancing

- Multi-core CPUs
- GPUs
- CPU/GPU Clusters
- Distributed systems

Goal:

$$\text{Minimize } \max(T_1, T_2, \dots, T_N)$$

Where:

- T_i = execution time on processor i

If one processor takes longer → overall system waits → efficiency drops.

90

Resource Management - Memory

- ❖ Multi-core CPUs → Cache coherence issues
- ❖ GPUs → Shared/global memory optimization
- ❖ Clusters → Distributed memory management

Minimize:

- Memory access latency

Maximize:

- Cache hit rate

Avoid:

- Memory contention
- False sharing (CPUs)
- Bandwidth saturation (GPUs & Clusters)

92

Summary

Feature	Multi-core CPU	GPU	Cluster
Memory Model	Shared	Shared (device memory)	Distributed
Best For	Task parallelism	Data parallelism	Massive distributed workloads
Core Count	Few (4–64)	Thousands	Thousands to millions
Bottleneck	Cache & synchronization	Memory bandwidth	Network communication
Latency	Low	Higher per thread	High (network)
Throughput	Moderate	Very High	Extremely High

93


 BITS Pilani

Parallel ML Algorithms

95


 BITS Pilani

M2: Distributed ML

94


 BITS Pilani

1. CNN
2. Gradient Descent/SGD
3. SVM
4. k-Means
5. kNN,
6. Decision Trees/Random Forests

96

Convolutional Neural Networks (CNN)

A **Convolutional Neural Network (CNN)** is a deep learning architecture designed to process **grid-structured data**, especially images.

An image can be represented as:

$$X \in \mathbb{R}^{H \times W \times C}$$

Where:

- H - Height
- W - Width
- C - Number of channels (e.g., 3 for RGB)

CNNs extract **spatial features** using convolution operations instead of fully connected layers.

97

Structure of CNN

A CNN typically contains:

1. **Convolution Layers:** Applies learnable filters to extract important features (edges, textures, patterns) from the input image.
2. **Activation (ReLU):** Introduces non-linearity by converting negative values to zero, helping the network learn complex patterns.
3. **Pooling:** Reduces the spatial size of feature maps to decrease computation and control overfitting.
4. **Fully Connected Layers:** Combines extracted features to perform final classification or prediction.

Multiple convolution layers extract hierarchical features:

- Early layers → edges
- Middle layers → shapes
- Deep layers → objects

99

Convolutional Neural Networks (CNN)

Instead of connecting every neuron to all pixels, CNNs use:

Local Receptive Fields: Each neuron sees only a small region (e.g., 3×3).

Weight Sharing: Same kernel slides across the image.

$$Y_{i,j} = \sum_{m=0}^{k-1} \sum_{n=0}^{k-1} X_{i+m,j+n} \cdot W_{m,n}$$

Where:

- X = Input
- W = Kernel
- k = Kernel size
- $Y_{i,j}$ = Output feature

This produces **feature maps** detecting edges, textures, patterns, etc.

98

Why Parallelism is Required in CNN

Massive Computational Cost

From complexity:

$$\text{Complexity} = O(HWck^2F)$$

Where:

- H, W = spatial size
- C = input channels
- k = kernel size
- F = number of filters

For large models:

- Millions to billions of parameters
- Billions of floating-point operations (FLOPs)

Sequential CPU execution becomes extremely slow.

100

Why Parallelism is Required in CNN

Independence in Computation

Each output pixel:

$$Y_{ij} \text{ is independent of } Y_{p,q}$$

This means:

- Different pixels can be computed simultaneously.
- Different filters can run in parallel.
- Different images in a batch can run in parallel.

*CNN structure naturally exposes **massive parallelism**.*

101

Why Parallelism is Required in CNN

Memory Bandwidth Requirement

CNN operations involve:

- Large tensor multiplications
- Frequent memory access

Parallel systems (GPUs) provide:

- High memory bandwidth
- Thousands of threads

Which matches CNN workload.

103

Why Parallelism is Required in CNN

Training Requires Repeated Computation

Training involves:

- Forward pass
- Backward pass
- Gradient computation
- Parameter updates

For large datasets (e.g., millions of images):

Training time without parallelism → weeks or months.

Parallel hardware reduces this to hours or days.

102

Why Parallelism is Required in CNN

CNN convolution = structured matrix multiplication

Matrix multiplication is:

- Highly parallelizable
- Optimized on GPUs
- Efficient on distributed clusters

Thus:

CNN efficiency $\propto$ Parallel Compute Resources

104

Single-channel convolution

$$Y_{i,j} = \sum_{m=0}^{k-1} \sum_{n=0}^{k-1} X_{i+m,j+n} \cdot W_{m,n}$$

Where:

- $X \rightarrow$ Input feature map of size $H \times W$
- $W \rightarrow$ Kernel (filter) of size $k \times k$
- $Y_{i,j} \rightarrow$ Output at spatial position (i,j)
- $m,n \rightarrow$ Kernel indices
- $k \rightarrow$ Kernel size

At every output position:

- A **local receptive field** of size $k \times k$ times $k \times k$ is extracted.
- Each element is multiplied by corresponding kernel weight.
- Results are summed $\rightarrow$ dot product.

So convolution = **sliding dot product**.

105

Computational Complexity

Given:

- Input size = $H \times W$
- Kernel size = $k \times k$
- Channels = C
- Filters = F

For one output pixel of one filter:

$$\text{Operations} = C \times k^2$$

For entire feature map:

$$H \times W \times C \times k^2$$

For F filters:

$$O(HWCk^2F)$$

107

Multi-Channel Convolution

Multi-Channel Convolution — Where Channel Summation Appears

$$Y = \sum_{c=1}^C X_c * W_c$$

Where:

- $C \rightarrow$ Number of input channels
- $X_c \rightarrow$ Channel c of input
- $W_c \rightarrow$ Corresponding kernel for channel c
- $*$ $\rightarrow$ Convolution operation

Each channel produces a partial output.

All channels are summed $\rightarrow$ final feature map.

106

Intra-layer Parallelism

Each output pixel:

$$Y_{i,j} \text{ is independent of } Y_{p,q}$$

So:

- Thousands of GPU threads compute different pixels simultaneously.
- No dependency between spatial locations.

This matches GPU architecture.

108

Data Parallelism

If batch size = B

Split across G GPUs:

Each GPU processes $\frac{B}{G}$

Where:

- Every GPU has a **full copy of the model**.
- Each GPU computes gradients on its mini-batch.
- Gradients are **synchronized (averaged)** after the backward pass.
- Widely used in **distributed deep learning training**.
- Improves training speed without changing model structure.

109

Gradient Descent (GD)

Gradient Descent is a fundamental optimization algorithm used to train machine learning models.

Its purpose is to minimize a loss (cost) function by iteratively adjusting the model parameters in the direction that reduces error.

$$J(\theta) = \frac{1}{n} \sum_{i=1}^n L(f(x_i, \theta), y_i)$$

Where:

- $\theta \rightarrow$ model parameters (weights)
- $n \rightarrow$ number of training samples
- $x_i \rightarrow$ input sample
- $y_i \rightarrow$ true label
- $f(x_i, \theta) \rightarrow$ model prediction
- $L(\cdot) \rightarrow$ loss function
- $J(\theta) \rightarrow$ average training error

The objective is:

$$\theta^* = \arg \min_{\theta} J(\theta)$$

This means we want the parameter values that minimize the total error.

111

Model Parallelism

Split filters:

If total filters (or parameters) = F

Split across P processors:

Each processor computes $\frac{F}{P}$ filters

Where:

- Model is **partitioned across devices**.
- Each processor handles part of the model.
- Activations must be communicated between processors.
- Useful when the **model is too large to fit in one GPU memory**.
- Common in very large models (e.g., large transformers).

110

Gradient Descent (GD)

The gradient of the loss function:

$$\nabla J(\theta)$$

gives the direction of steepest increase.

To minimize the function, we move in the opposite direction:

$$\theta_{t+1} = \theta_t - \eta \nabla J(\theta_t)$$

Where:

- $\eta \rightarrow$ learning rate
- $t \rightarrow$ iteration number

This iterative update continues until convergence.

112

Gradient Descent (GD)

- **Almost all modern ML models** are trained using **Gradient Descent** or its variants (SGD, Adam, RMSProp).
- **Deep learning** is entirely based on **gradient-based optimization** to update millions/billions of parameters.

Why Computation Becomes Extremely Large

1. **Large Datasets (N samples)**
 - Gradient must be computed over massive data → high data processing cost.
2. **High-Dimensional Parameter Space (d parameters)**
 - Each update requires computing partial derivatives for all parameters → $\nabla L \in \mathbb{R}^d$
3. **Deep Neural Networks**
 - Multiple layers → forward + backward propagation
 - Complexity increases with depth and width

113

Parallel Computation Model for Gradient Descent

- In **batch gradient descent**, the gradient for each sample is **independent**:

$$\nabla L_1, \nabla L_2, \dots, \nabla L_n$$

- No sample depends on another for its gradient calculation.

Parallelization Strategy

1. **Distribute the dataset** across multiple processors or GPUs.
 - Example: If batch size $B=1024$ and 4 processors → each processes 256 samples.
2. **Compute local gradients** for each subset independently.
3. **Aggregate gradients** after computation using a **parallel reduction**.

115

Why GD is Computationally Heavy

Gradient of loss:

$$\nabla J(\theta) = \frac{1}{n} \sum_{i=1}^n \nabla L_i$$

Computation requires:

1. **Forward pass** for each sample x_i → compute predictions
2. **Backward pass** for each sample → compute ∇L_i
3. **Summation** over all sample gradients

When Cost Becomes Extremely High

- **Large dataset:** n-millions of samples
- **High-dimensional model:** θ -millions of parameters

$$\text{Total cost} \propto n \times \text{model size}$$

114

Parallel Computation Model for Gradient Descent

Assume

- **P processors**
- Dataset split into **P subsets**: $D_1, D_2, \dots, D_P$
- Each processor computes gradients for its subset:

$$\nabla J(\theta) = \frac{1}{n} \sum_{p=1}^P \sum_{i \in D_p} \nabla L_i$$

How It Works

1. Each processor performs:
 - **Forward pass** for samples in D_p
 - **Backward pass** for samples in D_p
2. Compute **local gradient sum** for its subset
3. **Parallel reduction**: sum local gradients across all processors

116

Parallel Computation Model for Gradient Descent

Parallel Time Approximation

$$T_{\text{parallel}} = \frac{T_{\text{compute}}}{P} + T_{\text{communication}}$$

Where:

- T_{compute} → time to compute gradients
- P → number of processors
- $T_{\text{communication}}$ → time to aggregate gradients

Increasing P reduces computation time per processor.

Communication overhead ($T_{\text{communication}}$) may dominate if too many processors are used.

Optimal speedup balances **computation vs communication**.

117

Parallel Stochastic Gradient Descent (SGD)

Communication in Distributed Training

Total Time in a Cluster

$$T_{\text{total}} = T_{\text{compute}} + T_{\text{communication}}$$

Communication Model (α - β Model)

$$T_{\text{comm}} = \alpha + \beta m$$

Where:

- α → **latency cost** (fixed overhead per message)
- β → **transfer time per byte**
- m → **gradient size** (number of parameters)

Large models → large m → **communication dominates training time**

Optimization of communication (e.g., gradient compression, All-Reduce) is crucial for **scalable distributed training**

119

Stochastic Gradient Descent (SGD)

Instead of computing gradients over the **entire dataset**, use a **mini-batch B**.

Update rule:

$$\theta_{t+1} = \theta_t - \eta \frac{1}{|B|} \sum_{i \in B} \nabla L_i$$

Where:

- B → mini-batch of samples
- $|B|$ → size of mini-batch
- η → learning rate

Advantages

- Reduces **computation per iteration**
- Enables **faster updates** and **more frequent learning**
- Works well with **large datasets**

By using only a small subset of data per update, SGD trades exact gradient accuracy for faster, more scalable training.

118

Support Vector Machine (SVM)

Support Vector Machine (SVM) is a supervised learning algorithm used for classification and regression.

Goal: Find a **hyperplane** that separates two classes with **maximum margin** while allowing for some misclassification (ξ_i).

Mathematically, the **optimization problem** is:

$$\min_{w,b} \frac{1}{2} \|w\|^2 + C \sum_{i=1}^n \xi_i$$

Subject to constraints: $y_i(w \cdot x_i + b) \geq 1 - \xi_i$ $\xi_i \geq 0$

Where:

- w = weight vector (hyperplane normal)
- b = bias term
- ξ_i = slack variables (allow margin violations)
- C = regularization parameter (penalty for misclassification)
- x_i, y_i = training samples and labels

120

Computational Complexity – SVM

Kernel Matrix Computation

- **Kernel:** $K(x_i, x_j)$ for all pairs of samples
- **Kernel matrix size:** $K \in \mathbb{R}^{n \times n}$
- **Time Complexity:** $O(n^2)$

Computing the kernel matrix dominates cost for large datasets; **parallelization across kernel entries significantly reduces computation time.**

121

Why Parallelism is Needed in SVM

Challenge

- Large datasets: $n \sim 10^5 - 10^6$
- Kernel matrix size: $n \times n \rightarrow$ **huge memory and computation requirement**
- Sequential computation: **slow and memory-intensive**

Benefits of Parallelism

1. **Faster kernel matrix evaluation**
 - Compute blocks independently on multiple processors/GPUs
2. **Efficient gradient updates**
 - Parallel updates of support vector contributions
3. **Scalable SVM training**
 - Enables training on **clusters** or **GPU farms** for large-scale datasets

Parallelism is essential to make SVM training feasible and efficient on modern, large datasets.

123

Parallelization in SVM

Kernel Computation

- Kernel values between pairs (x_i, x_j) are independent:
 $K(x_i, x_j) \quad \forall i, j$
- Can compute **blocks of the kernel matrix in parallel** across multiple cores or GPUs.
- This reduces the time from $O(n^2)$ sequentially to roughly $O(n^2/P)$ with P processors.

Training Parallelism

- Training methods like **SMO (Sequential Minimal Optimization)** update support vectors iteratively, but parts of computation (gradient updates, kernel block computations) can run concurrently.
- **Mini-batch approaches** or distributed SVMs allow larger parallelization.

***Kernel computation** is the most parallelizable part, while **training updates** have limited but still useful parallel opportunities in large-scale or distributed setups.*

122

SVM Parallelization

Component	Complexity	Parallelization Potential
Kernel matrix computation	$O(n^2)$	High (compute independent blocks)
Training (SMO/QP)	Depends on iterations	Partial (gradient & kernel computations)

SVM training benefits from **data-level parallelism** (kernel computation) and **task-level parallelism** (optimization steps), especially on large datasets or distributed systems.

124

k-Means

k-Means is an unsupervised clustering algorithm that partitions n data points into k clusters by minimizing the **within-cluster sum of squares (WCSS)**:

$$J = \sum_{i=1}^n \min_k \|x_i - \mu_k\|^2$$

Where:

- $x_i \rightarrow$ data point i
- $\mu_k \rightarrow$ centroid of cluster k
- $\|x_i - \mu_k\|^2 \rightarrow$ squared Euclidean distance

125

Computational Complexity of k-Means

Assignment Step

$O(nkd)$

- For each of the n data points
- Compute distance to each of the k centroids
- Each distance computation costs $O(d)$ in d -dimensional space

Update Step

$O(nd)$

- Sum all points belonging to each cluster
- Recompute centroids by averaging
- Total cost proportional to number of points and dimensions

Overall Complexity

$O(nkd \times \text{iterations})$

- Algorithm repeats until convergence
- Runtime depends on number of iterations required

127

k-Means

Algorithm Steps (Iterative Process)

Assignment Step:

Assign each data point x_i to the **nearest centroid** μ_k .

Update Step:

Recompute centroids:

$$\mu_k = \frac{1}{|C_k|} \sum_{x_i \in C_k} x_i$$

Where:

- $C_k \rightarrow$ set of points assigned to cluster k
- $|C_k| \rightarrow$ number of points in cluster k

126

Parallelization in k-Means

Step 1 – Assignment (Data Parallelism)

Distance computations:

$$\|x_i - \mu_k\|^2 \quad \forall i, k$$

can be performed **concurrently**.

Where:

- $x_i \rightarrow$ data point i
- $\mu_k \rightarrow$ centroid of cluster k
- $i=1, 2, \dots, n$
- $k=1, 2, \dots, K$
- $\|x_i - \mu_k\|^2 \rightarrow$ squared Euclidean distance

Since each pair (i, k) is independent, these computations can be executed **in parallel across cores or GPU threads**.

128

Parallelization in k-Means

Step 2 – Centroid Update (Parallel Reduction)

For cluster k: $\mu_k = \frac{1}{|C_k|} \sum_{x_i \in C_k} x_i$

Where:

- $\mu_k \rightarrow$ centroid of cluster k
- $C_k \rightarrow$ set of data points assigned to cluster k
- $|C_k| \rightarrow$ number of points in cluster k
- $x_i \rightarrow$ data point belonging to cluster k

Parallel Strategy

1. Split cluster points among threads
2. Each thread computes a **local sum**
3. Combine partial sums
4. Divide by $|C_k|$ to get updated centroid

Centroid recomputation is efficiently parallelizable using reduction operations on multi-core CPUs and GPUs.

129

k-Nearest Neighbors (kNN)

k. Nearest Neighbors (kNN) is a non-parametric, instance-based learning algorithm used for classification and regression.

Unlike models such as SVM or neural networks:

- **No explicit training phase**
- The entire training dataset is stored
- Prediction is done by comparing a test sample to all training samples

It is called a **lazy learner** because computation is deferred until prediction time.

131

Why Parallelization Helps in k-Means

Challenges

- Large datasets ($n \gg 10^5$) $\rightarrow$ Sequential computation becomes slow
- High-dimensional data ($d \gg 10$) $\rightarrow$ Each distance computation is expensive

How Parallelization Improves Performance

- **Assignment Step** $\rightarrow$ Exploits **data-level parallelism**
 - Distances for multiple points computed simultaneously
- **Update Step** $\rightarrow$ Uses **reduction-level parallelism**
 - Parallel summation of cluster points
- **Distributed Setting (Clusters)**
 - Data batches distributed across nodes
 - Local centroid updates combined globally

130

k-Nearest Neighbors (kNN)

For a test sample $x \in \mathbb{R}^d$, compute its distance to every training sample x_i :

$$d(x, x_i) = \sum_{j=1}^d (x_j - x_{ij})^2$$

Where:

- $x \in \mathbb{R}^d \rightarrow$ test sample in d-dimensional space
- $x_i \rightarrow$ i-th training sample
- $x_j \rightarrow$ j-th feature of test sample
- $x_{ij} \rightarrow$ j-th feature of training sample i
- $d(x, x_i) \rightarrow$ squared Euclidean distance

Prediction Steps

1. Compute distances to all training samples
2. Sort distances
3. Select the k smallest

Final Prediction:

- **Classification** $\rightarrow$ Majority vote among nearest neighbors
- **Regression** $\rightarrow$ Mean value of nearest neighbors

132

Computational Complexity of k-NN

Training Phase

$O(1)$

- No model training or parameter estimation
- Only storing the dataset

If Full Sorting is Performed:

$O(nd + n \log n)$

If Partial Selection is Used (Heap / Quickselect):

$O(nd)$

k-NN has negligible training cost but expensive prediction cost, especially for large n and high-dimensional data.

Prediction Phase

For each test sample:

- Compute distance to all n training samples
- Each distance computation costs $O(d)$

Total Distance Cost:

$O(nd)$

Where:

- $n \rightarrow$ number of training samples
- $d \rightarrow$ number of features

133

Parallelization in k-NN

Each distance:

$$d(x, x_i)$$

is independent of others $\rightarrow$ **k-NN is embarrassingly parallel.**

Where:

- $x \rightarrow$ test sample
- $x_i \rightarrow$ i -th training sample
- $d(x, x_i) \rightarrow$ distance between test and training sample

135

Why k-NN Becomes Expensive

Example Scenario

If:

$$n = 10^6, d = 100$$

Then for **one test sample**, distance computations required:

$$10^6 \times 100 \approx 10^8 \text{ operations}$$

Why This is Heavy

- 100 million arithmetic operations per prediction
- Must repeat for every test sample
- Becomes impractical for real-time systems

k-NN scales poorly with large datasets, making parallelization or approximate methods necessary for efficiency.

134

Parallelization in k-NN

Parallel Strategies

Data-Level Parallelism

If P processors are used:

$$T_{\text{parallel}} \approx \frac{nd}{P}$$

Where:

- $n \rightarrow$ number of training samples
- $d \rightarrow$ feature dimension
- $P \rightarrow$ number of processors
- $T_{\text{parallel}} \rightarrow$ parallel prediction time

Each processor computes distances for n/P training samples

136

Parallelization in k-NN

Parallel Strategies

Parallel Reduction for k Selection

After computing all distances $d(x, x_i)$, we must select the **k smallest distances**.

Why This Matters

- Distance computation is parallel
- But selecting the **top-k nearest neighbors** can become a bottleneck
- Efficient parallel selection is required for scalability.

137

Decision Tree

A **Decision Tree** is a supervised learning algorithm used for **classification and regression**.

It recursively partitions the dataset into subsets based on feature values, aiming to maximize **class purity** at each split.

Choose the feature that provides the **best separation of data** at each node.

How It Works

- Start with the entire dataset at the root node
- Select the best feature to split the data
- Create child nodes based on split condition
- Repeat recursively until stopping criteria is met

139

k-Nearest Neighbors (kNN)

Component	Best Hardware	Reason
Distance computation	GPU	Massive independent arithmetic
Small datasets	Multi-core CPU	Lower overhead
Very large datasets	Cluster	Distribute training data

138

Decision Tree - Splitting Criterion

Entropy

For a dataset S with c classes:

$$H(S) = - \sum_{i=1}^c p_i \log p_i$$

Where:

- p_i = probability of class i in dataset S
- Entropy measures **impurity**
- $H(S) = 0 \rightarrow$ pure node
- Maximum entropy $\rightarrow$ uniform distribution

140

Decision Tree - Splitting Criterion

Information Gain

After splitting dataset SSS using feature A:

$$IG(S, A) = H(S) - \sum_{v \in \text{Values}(A)} \frac{|S_v|}{|S|} H(S_v)$$

Where:

- S_v = Subset where feature A=v
- $|S_v|/|S|$ = Weight of subset
- $H(S_v)$ = Entropy after split

Information Gain measures **reduction in entropy** after splitting.

The feature with **maximum Information Gain** is chosen for the split.

141

Computational Complexity of Decision Trees

Overall Tree Complexity

If the tree depth is h:

$$\text{Total Cost} \approx O(ndh)$$

(Each level evaluates d features across the available n samples.)

For:

- **Balanced tree** $\rightarrow h \approx \log n$

$$O(nd \log n)$$

(Logarithmic depth reduces total split evaluations.)

- **Worst case (skewed tree)** $\rightarrow h \approx n$

$$O(n^2d)$$

(Unbalanced splits repeatedly process nearly the entire dataset.)

143

Computational Complexity of Decision Trees

At Each Node

Operations performed:

1. Evaluate all **d features**
2. For each feature, examine possible split points
3. Compute impurity (Entropy/Gini) for each split

Time Complexity Per Node

If:

- n = number of samples at that node
- d = number of features

Then computational cost per node: $O(nd)$

- For each feature $\rightarrow$ scan all nnn samples
- Total work $\rightarrow d \times n$

142

Random Forest

Random Forest is an ensemble learning method that combines multiple decision trees to improve prediction accuracy and reduce overfitting.

For input x:

$$\hat{y} = \frac{1}{T} \sum_{t=1}^T f_t(x)$$

Where:

- T = number of decision trees in the forest
- $f_t(x)$ = prediction of the t-th tree
- $\hat{y}$ = aggregated output (average for regression, majority vote for classification)

Random Forest uses:

- **Bootstrap sampling** for each tree (different subset of data)
- **Random feature selection** at splits to increase diversit

144

Why Random Forests Are Naturally Parallel

Random Forests are ensembles of multiple decision trees trained on different subsets of data and features. Their structure makes parallelization very efficient:

Independence During Training

Each tree $f_1, f_2, \dots, f_T$ is trained **independently** using:

- **Bootstrap sampling:** a random subset of the training data per tree.
- **Random feature selection:** only a subset of features is considered at each split.

Implication:

- There are **no dependencies between trees**, so each tree can be trained on a separate processor or core without waiting for others.
- This allows **linear scaling** with the number of available processors, up to TTT.

145

Parallel Training Model

Assumptions:

- **Serial training time per tree:** T_{serial}
- **Number of processors:** P

Parallel training time:

$$T_{\text{parallel}} \approx \frac{T_{\text{serial}}}{P}$$

- Each processor trains **one or more trees independently**.
- No synchronization is needed between trees until aggregation of results.

Ideal Speedup:

$$S_{\text{max}} \approx P \text{ (if number of trees } T \geq P \text{)}$$

- Linear scaling is possible when the number of trees is at least the number of processors.
- Perfect for **multi-core CPUs, GPUs, or distributed clusters**.

147

Why Random Forests Are Naturally Parallel

Independence During Prediction

After training, each tree produces a prediction $f_t(x)$ for input x .

- Predictions from all trees can be computed **simultaneously**.
- Final output is obtained via:
 - **Majority vote** for classification
 - **Average** for regression

Implication:

- Tree predictions are independent until aggregation, making the prediction phase **embarrassingly parallel**.
- This is especially useful for real-time or large-scale inference.

146

Parallel Prediction in Random Forests

- Assign each tree (or group of trees) to a processor p , $p=1,2,\dots,P$

$f_t(x)$ computed independently for all $t \in \text{trees on processor } p$

- Aggregation step after all processors finish:

$$\hat{y} = \frac{1}{T} \sum_{p=1}^P \sum_{t \in T_p} f_t(x)$$

Where T_p = set of trees assigned to processor p , $\bigcup_{p=1}^P T_p = \{1, 2, \dots, T\}$

Speedup:

- Ideal parallel time for prediction:

$$T_{\text{parallel}} \approx \frac{T_{\text{serial}}}{P}, \quad S_{\text{max}} \approx P \quad (\text{if } T \geq P)$$

- Only synchronization: aggregation (voting or averaging).

148

Comparison of Parallelization Suitability

Algorithm	Data Parallelism	Task Parallelism	Cluster-Friendly	GPU-Friendly
CNN	Excellent	Moderate	Excellent	Excellent
GD / SGD	Excellent	Moderate	Excellent	Excellent
SVM	Moderate	Limited	Moderate	Limited
k-Means	Excellent	Limited	Good	Excellent
kNN	Excellent	Limited	Good	Excellent
Decision Trees	Moderate	Good	Good	Moderate
Random Forest	Excellent	Excellent	Excellent	Moderate

149


 BITS Pilani

1. Data parallelism
2. Model parallelism
3. Pipeline parallelism
4. gradient checkpointing
5. Mixed precision training

151


 BITS Pilani

Distributed Training Strategies

150

Data Parallelism (DP)

Data Parallelism is a **parallelization strategy** where the **same model** is replicated across multiple devices (CPUs, GPUs, or TPUs) to **process different subsets of the dataset simultaneously**.

Goal:

- Reduce total training time.
- Efficiently utilize hardware resources.
- Keep model parameters synchronized across devices.

Let total batch size = B

Let number of devices = N

Split batch across devices:

$$B_i = \frac{B}{N}, \quad i = 1, 2, \dots, N$$

Each device i processes its mini-batch B_i **independently**.

Computation on each mini-batch does **not depend on other mini-batches**, so it can run concurrently → true parallelism.

152

Local Gradient Computation

For a single device i , compute the gradient of the loss function:

$$\nabla J_i(\theta) = \frac{1}{B_i} \sum_{j \in B_i} \nabla L_j(\theta)$$

Where:

- $L_j(\theta)$ = loss for sample j in the mini-batch.
- θ = model parameters.
- $\nabla J_i(\theta)$ = **local gradient** on device i .

Each device independently computes the contribution of its mini-batch to the overall gradient.

*This is a **data-parallel operation**, since mini-batches are disjoint.*

153

Parameter Update

After global gradient aggregation, model parameters are updated as:

$$\theta_{t+1} = \theta_t - \eta \nabla J(\theta)$$

θ → parameters at iteration t

η → learning rate

$\nabla J(\theta)$ → global gradient aggregated from all devices

How It Works

- Every device applies the **same global gradient** to its local model.
- Ensures all devices **remain synchronized** after the update.

Guarantees **correct convergence** of the training process.

*Even though each device processes different mini-batches, **all model replicas stay identical**.*

155

Global Gradient Aggregation

Once each device i computes its local gradient $\nabla J_i(\theta)$, the **global gradient** is computed as:

$$\nabla J(\theta) = \frac{1}{N} \sum_{i=1}^N \nabla J_i(\theta)$$

N → number of devices

$\nabla J(\theta)$ → gradient used for model update

How It's Done

- Performed using **All-Reduce** in GPU clusters.
- **All-Reduce**: sums the local gradients from all devices and distributes the result back to each device simultaneously.

Prevents parameter divergence: ensures all devices update their model with the same gradient.

Maintains consistency: every device keeps identical parameters θ after the update.

154

Parallelism Benefits

Speedup

- Each device processes **1/N** of the total batch.
- Reduces **per-iteration computation time**.
- Ideal case: speedup $\approx N$ if **communication overhead is negligible**.

Scalability

- Adding more devices enables **larger batch sizes** or **faster training**.
- Training throughput grows **linearly with N** until **communication costs dominate**.

Hardware Utilization

- Maximizes **GPU/CPU cores** and **memory bandwidth**.
- Prevents idle cores during **forward/backward passes**.
- Efficient for large models and high-dimensional data.

156

Communication Overhead

$$T_{\text{comm}} = \alpha + \beta \cdot m$$

$\alpha \rightarrow$ **latency per message** (fixed cost per communication). β

$\rightarrow$ **time per byte** (depends on bandwidth).

$m \rightarrow$ **message size** (size of gradient vector being communicated).

- After **local gradient computation**, devices must communicate to compute the **global gradient**.
- With **too many devices**, communication time can **dominate computation**, reducing overall efficiency.
- Optimization strategies**: gradient compression, overlap of communication and computation, hierarchical aggregation.

157

Model Parallelism (MP)

- Model Parallelism is used when the **model is too large to fit into the memory of a single device** (GPU/CPU).
- Instead of replicating the model on multiple devices (as in DP), **the model itself is split** across devices.
- Each device holds a part of the model and performs computations only for that part.

Typical use case:

- Very deep neural networks (e.g., GPT-like transformers, ResNets with hundreds of layers).
- Extremely large embeddings (e.g., NLP models with billions of parameters).

159

Data Parallelism (DP) - Example

Suppose:

- Batch size $B = 1024$
- Number of GPUs $N = 4$

Step 1 – Split mini-batches:

$$B_i = \frac{1024}{4} = 256 \text{ samples per GPU}$$

Step 2 – Compute local gradients:

Each GPU computes:

$$\nabla J_i(\theta) = \frac{1}{256} \sum_{j=1}^{256} \nabla L_j(\theta)$$

Each GPU only sees 256 samples but contributes to the **same model update** as if 1024 samples were processed sequentially.

Total training time is roughly **1/4 of single GPU**, minus communication overhead.

Step 3 – Aggregate global gradient:

$$\nabla J(\theta) = \frac{1}{4} \sum_{i=1}^4 \nabla J_i(\theta)$$

Step 4 – Update parameters:

$$\theta_{t+1} = \theta_t - \eta \nabla J(\theta)$$

158

Model Parallelism (MP)

Forward Pass

Let the model have m layers:

$$L_1, L_2, \dots, L_m$$

Split across devices:

- GPU1 $\rightarrow$ layers L_1, L_2
- GPU2 $\rightarrow$ layers L_3, L_4
- GPU3 $\rightarrow$ layers $L_5, \dots, L_m$

Input x flows sequentially:

$$y = L_m(\dots(L_2(L_1(x))))$$

- Each device computes only its assigned layers:

$$\text{Device 1: } a_2 = L_2(L_1(x)) \quad \text{Device 2: } a_4 = L_4(L_3(a_2)) \quad \text{Device 3: } y = L_m(a_4)$$

- Activations (a_i) must be **transferred** to the next device in sequence.

160

Model Parallelism (MP)

Backward Pass

- Gradients are computed **layer by layer in reverse order** (standard backpropagation).
- For device i :

$$\delta_i = \frac{\partial L}{\partial a_i}, \quad \theta_i^{\text{new}} = \theta_i - \eta \delta_i$$

$\delta_i \rightarrow$ gradient of loss w.r.t activations of layer i

$\theta_i^{\text{new}} \rightarrow$ updated weights for layer i

$\eta \rightarrow$ learning rate

Device Communication

- Forward pass:** activations are sent to the next device.
- Backward pass:** gradients are sent back to previous devices.
- Ensures **correct weight updates** across split model.

161

Comparison with Data Parallelism

Feature	Data Parallelism (DP)	Model Parallelism (MP)
What is split?	Dataset (mini-batches)	Model (layers, parameters)
Devices hold	Entire model	Part of model
Forward pass	Each device independent	Sequential across devices
Communication	Gradients aggregated (All-Reduce)	Activations and gradients passed sequentially
Best for	Small/medium models with large datasets	Very large models that don't fit on single device

163

Model Parallelism (MP)

- Let total model parameters = P
- Device memory limit = M

Then **number of devices required**:

$$N_{\text{devices}} \geq \frac{P}{M}$$

- If forward computation of layer i takes t_i , total forward time:

$$T_{\text{forward}} = \sum_{i=1}^m t_i + \sum_{i=1}^{m-1} t_{\text{comm}}(a_i)$$

Where t_{comm} = communication time for passing activations between devices.

- Total training time includes **forward + backward + communication**:

$$T_{\text{total}} = T_{\text{forward}} + T_{\text{backward}} + T_{\text{communication}}$$

162

Pipeline Parallelism

Extension of Model Parallelism: Model is split across multiple devices, each handling a stage of the network.

Stage-wise Division: Each device processes a subset of consecutive layers.

Micro-Batch Feeding: Input batch is divided into smaller micro-batches for sequential feeding.

Overlapping Computation & Communication: While GPU1 computes micro-batch 2 at stage 1, GPU2 can compute micro-batch 1 at stage 2 $\rightarrow$ improves throughput.

Reduces Idle Time: Devices are rarely idle compared to standard model parallelism.

164

Pipeline Parallelism

Scalable to Deep Models: Suitable for very deep networks that cannot fit entirely on one device.

Forward & Backward Overlap: Backpropagation for one micro-batch can start before the forward pass of the next micro-batch finishes.

Throughput Optimization: Total training time is reduced even if per-micro-batch latency is similar.

Micro-Batch Size Trade-off: Smaller micro-batches → better overlap, but higher communication overhead; larger micro-batches → more efficient computation per device.

165

Pipeline Parallelism

Throughput Optimization

$$T_{\text{pipeline}} \approx \max(T_1, T_2, \dots, T_n) \cdot \text{number of micro-batches}$$

T_i → time taken by **stage i** to process one micro-batch

n → total number of **stages (devices)** in the pipeline

number of micro-batches → batch size / micro-batch size

Goal:

- Balance T_i across all stages so that no stage becomes a bottleneck.
- Ensures all devices work continuously, minimizing idle time.
- Maximizes **throughput** rather than reducing per-batch latency.

*If one stage is slower, it determines the pipeline speed.
Effective stage assignment and micro-batch sizing are critical for efficiency.*

167

Pipeline Parallelism

- Split the model into **stages** across multiple GPUs: $S_1, S_2, \dots, S_N$
- Forward/backward passes are executed in **pipeline fashion**:
 - While GPU1 processes batch B_2 at layer L_1 , GPU2 processes batch B_1 at layer L_2
- Overlaps computation and communication → improves throughput

Let:

- t_{S_i} = time to process stage S_i for one batch
- Pipeline efficiency factor = **efficiency** ≤ 1 (due to pipeline bubbles)

Then approximate total training time:

- $$T_{\text{total}} \approx \text{pipeline efficiency} \cdot \sum_{i=1}^N t_{S_i}$$
- The sum represents **stage-wise processing time**
 - Pipeline bubbles reduce efficiency at the **start and end** of training

166

Gradient Checkpointing

- Backpropagation stores **activations of all layers** for computing gradients.
- Problem: **Memory grows linearly with model depth** → can exceed GPU memory for large networks.
- Solution: Store **only selected activations** (checkpoints) and recompute the rest during backward pass.
- Modern models (Transformers, CNNs, ResNets) have **hundreds of layers**.
- GPU memory is **limited**, especially for large batch sizes.
- Naive backprop requires storing all intermediate activations:

$$M_{\text{total}} \approx O(L \cdot M_{\text{layer}})$$

Where:

L → number of layers

M_{layer} → memory per layer

Gradient checkpointing reduces this significantly.

168

Gradient Checkpointing

Forward Pass Memory Saving (Activation Checkpointing)

- Instead of storing **all activations** for backpropagation, store only a subset called **checkpoints**.
- Activations between checkpoints are **recomputed** during the backward pass.
- Reduces memory footprint significantly, allowing **larger models or batch sizes** on limited GPU memory.

$$\text{Memory Cost} \approx O(\text{number of checkpoints})$$

$$\text{Compute Cost} \approx O(\text{forward pass recomputation})$$

- More checkpoints $\rightarrow$ higher memory usage but lower recomputation cost.
- Fewer checkpoints $\rightarrow$ lower memory usage but higher compute overhead.
- Optimal checkpoint strategy balances memory and recomputation to fit GPU memory limits.

169

Gradient Checkpointing

Let:

- L = total number of layers
- C = number of checkpoints stored
- M_{layer} = memory required to store activations of one layer
- T_f = forward computation time per layer

Memory Usage

Without checkpointing:

$$M_{\text{full}} \approx L \cdot M_{\text{layer}}$$

With checkpointing:

$$M_{\text{checkpoint}} \approx C \cdot M_{\text{layer}}$$

Memory reduction factor:

$$\frac{M_{\text{checkpoint}}}{M_{\text{full}}} = \frac{C}{L}$$

So memory scales **linearly with number of checkpoints**, not total layers.

171

Gradient Checkpointing

Impact on Backward Pass

- During backpropagation, intermediate activations not stored are recomputed from the nearest checkpoint.
- Ensures gradient computation remains **exact**, but adds extra computation.
- Trade-off is **memory saved vs extra forward passes**.

Combined Memory-Compute Considerations

- Memory reduction factor $\approx$ **total layers / checkpoints stored**.
- Especially useful in **very deep networks** where storing all activations is prohibitive.
- Works well with **mixed-precision training** to further reduce memory usage.

170

Gradient Checkpointing

Additional Compute Cost

Activations between checkpoints must be recomputed.

Extra computation:

$$\text{Extra Compute} \approx (L - C) \cdot T_f$$

Total effective compute time becomes:

$$T_{\text{total}} \approx T_{\text{forward}} + T_{\text{backward}} + (L - C) T_f$$

This shows:

- Fewer checkpoints $\rightarrow$ lower memory
- But $\rightarrow$ higher recomputation cost

If $C=L \rightarrow$ No recomputation, full memory usage

If $C \ll L \rightarrow$ Large memory savings, moderate compute increase

Typically, compute overhead increases modestly (~20 – 40%) while memory drops dramatically

172

Forward and Backward Workflow with Checkpointing

Forward Pass

- Compute outputs sequentially:
 $a_i = f_i(a_{i-1}, \theta)$
- Store activations **only at checkpoint layers**

- Discard intermediate activations to save memory

If checkpoints are placed every k layers:

$$C = \frac{L}{k}$$

Only C activations remain in memory.

Backward Pass

For gradient computation:

$$\delta_l = \frac{\partial L}{\partial a_l}$$

For layers between two checkpoints:

1. Reload last checkpoint activation
2. Recompute forward activations up to layer l
3. Compute gradient:

$$\frac{\partial L}{\partial \theta_l} = \delta_l \cdot \frac{\partial a_l}{\partial \theta_l}$$

4. Move backward layer by layer

173

Why Mixed Precision is Needed

Modern models:

- Large parameter count
- Large batch sizes
- Deep architectures

Memory requirement:

$$M_{total} = M_{weights} + M_{activations} + M_{gradients} + M_{optimizer}$$

For very deep networks:

$$M_{total} \gg GPU$$

Mixed precision reduces:

- Activation memory
- Gradient memory
- Temporary tensor memory

175

Mixed Precision Training

Mixed Precision Training uses a combination of:

- **FP16 (half precision, 16-bit)** → for forward and backward computations
- **FP32 (single precision, 32-bit)** → for maintaining stable master weights

Instead of using full 32-bit precision everywhere, it strategically reduces precision where safe.

Most deep learning operations (matrix multiplications, convolutions) are tolerant to small numerical noise.

Thus:

Memory per parameter:

FP32 → 4 bytes

FP16 → 2 bytes

Memory reduction:

$$M_{new} \approx \frac{1}{2} M_{FP32}$$

This allows:

- Larger batch sizes
- Larger models
- Reduced memory bandwidth usage

174

Mixed Precision Training

Compute Acceleration

Modern GPUs (especially those with **Tensor Cores**) are optimized for FP16 operations.

If:

$$T_{FP32} = \text{compute time in full precision}$$

Then typically:

$$T_{FP16} < T_{FP32}$$

Speedup depends on hardware architecture.

Training Workflow

1. Forward pass → Computed in FP16
2. Loss computed in FP32
3. Backward pass → Gradients in FP16
4. Gradients converted to FP32
5. Master weights updated in FP32

Weight update:

$$\theta_{t+1}^{FP32} = \theta_t^{FP32} - \eta \nabla L^{FP32}$$

FP16 weights are refreshed from FP32 master copy.

176

Mixed Precision: Forward & Backward Computation Flow

Forward Pass (FP16 Compute)

$$y=f(x;\theta^{FP16})$$

- Model weights are cast from FP32 $\rightarrow$ FP16
- Activations and matrix multiplications computed in FP16
- Faster execution (Tensor Cores)
- Lower memory usage

Backward Pass (FP16 Gradients)

$$\nabla J^{FP16} = \frac{\partial L}{\partial \theta}$$

- Gradients computed in FP16
- Loss scaling may be applied to prevent underflow

Stable Weight Update (FP32 Master Copy)

Steps:

- Convert gradients:

$$\nabla J^{FP32} = \text{Cast}(\nabla J^{FP16})$$

- Update master weights:

$$\theta_{t+1}^{FP32} = \theta_t^{FP32} - \eta \nabla J^{FP32}$$

- Cast updated weights back:

$$\theta_{t+1}^{FP16} = \text{Cast}(\theta_{t+1}^{FP32})$$

177

Mixed Precision Training

Loss Scaling

- Scaling shifts gradients into representable FP16 range
- Division restores original magnitude
- No change to true optimization direction
- Prevents gradient underflow

Types of Loss Scaling

1. Static Scaling

- Fixed scale value S
- Simple but may overflow

2. Dynamic Scaling

- Automatically adjusts S
- Reduces overflow risk
- Used in modern deep learning frameworks

179

Mixed Precision Training

Problem: Gradient Underflow in FP16

FP16 has limited precision:

- Smallest normal positive value $\approx 2^{-14}$
- Smallest representable value $\approx 2^{-24}$

If:

$$\nabla J < 2^{-24}$$

Then it becomes:

$$\nabla J = 0$$

Small gradients vanish
No weight update
Training becomes unstable

Solution: Loss Scaling

Multiply the loss by a scale factor S:

$$L' = S \cdot L$$

Gradient Computation After Scaling

During backpropagation:

$$\nabla J' = S \cdot \nabla J$$

Gradients are now larger $\rightarrow$ safe from underflow in FP16.

Before Weight Update (Unscale Gradients)

Divide gradients by the same scale factor:

$$\nabla J = \frac{\nabla J'}{S}$$

Then update weights normally in FP32:

$$\theta_{t+1}^{FP32} = \theta_t^{FP32} - \eta \nabla J$$

178



Parallel Computing based on GPGPU using Compute Unified Device Architecture

Lifeng Xu

Master thesis
System-On-Chip Design
Royal Institute of Technology (KTH)
Supervisor: Xiufeng Wang
Examiner: Prof. Axel Jantsch

TRITA-ICT-EX-2011:226

Abstract

The demand of processing a huge amount of data within a limited time and the developing of computing capability of Graphic Process Unit (GPU) lead us to the world of parallel computing on General Purpose GPU (GPGPU). Because of the exposed parallelism, GPGPU could assign processing tasks to multiple threads and execute these threads simultaneously. This feature could speedup heavy data computation to a level, which we would never imagine in the past.

In this thesis, we use a very important algorithm in image processing(DCT/IDCT) to present parallel computing on GPGPU, and implemented it both on the single GPU and multi-GPU system. The parallel computing technology is based on Compute Unified Device Architecture (CUDA) and OpenMP. Furthermore, some optimization strategies are discussed in the thesis and results will be compared. The final results show that the parallel computing could accelerate the processing from dozens of times to almost five hundreds of times. The potential factors that might affect the performance are also discussed. Therefore, it is more effective to use GPGPU to do parallel computing, and it is more power efficient.

Acknowledgement

I would like to express my gratitude to all these people who I met in the last two years. Especially, I want to thank my supervisor Wang Xiufeng, who not just helped me with the thesis work, but also encouraged me how to face the difficulties on the way. Also I am thankful to Prof. Axel Jantsch and Prof. Ahmed Hemani for all your help and time. I offer my blessings to all my friends in Sweden. You make the life in Sweden so happy and meaningful. Lastly, I would like to thank my parents for all your support.

Contents

1	Introduction	1
1.1	Motivation	1
1.2	Related Work	3
1.3	Thesis organization	4
2	Background	5
2.1	BMP format	5
2.2	CUDA	5
2.2.1	Kernel, thread, block and grid	6
2.2.2	Memory Hierarchy	7
2.3	Platform	9
2.4	GPU and CPU	10
2.5	OpenMP	10
2.6	PCI-Express	11
3	Algorithm implementation on CPU	13
3.1	BMP file operation	13
3.1.1	Bytes align	14
3.1.2	Image save	14
3.2	DCT/IDCT	14
3.2.1	Properties of DCT/IDCT	14
3.2.2	1D-DCT	15
3.2.3	2D-DCT	16
3.2.4	Inverse DCT	16
3.3	Experiments and Results	17
4	Parallel Algorithm on GPU	21
4.1	CUDA Programming Model	21
4.1.1	Kernel	21
4.1.2	Memory Architecture	22
4.1.3	Hardware Implementation	23

4.2	GPU-Parallel Algorithm	24
4.2.1	Initialization	24
4.2.2	Kernel Function	24
4.2.3	Optimization	27
4.2.4	Timing	30
4.3	Experiment and Results	31
5	Algorithm on Multi-GPU System	35
5.1	OpenMP Execution Model	36
5.2	Muli-GPU System Implementation	37
5.3	Experiment and Results	40
6	Conclusions and future work	45
6.1	Conclusions	45
6.2	Future Work	46
	References	48

List of Figures

2.1	The kernel and the function	7
2.2	Grid,Block,Thread	8
2.3	Local memory, Global memory, Shared memory and their scopes . .	9
2.4	Schematic Difference between CPU and GPU	11
3.1	Process Period Result	18
3.2	Original Lena and processed Lena	19
4.1	CUDA Memory Architecture and Host-Device Data Transfer	23
4.2	Flow Chart of the Algorithm on GPU	25
4.3	Shared Memory Scheme	29
4.4	Speedup of GPU Algorithm	32
4.5	Speedup of GPU Algorithm	33
5.1	The Dual-GPU Schematic	35
5.2	Fork-joint Execution Model	37
5.3	Results for image size of 128×128	40
5.4	Results for image size of 256×256	41
5.5	Results for image size of 512×512	42
5.6	Results for image size of 1024×1024	43
5.7	Results for image size of 2048×2048	43
5.8	Speedup of Multi-GPU Compared with Single GPU	44

List of Tables

2.1	GTX580 Device Information	10
3.1	Results on CPU	17
4.1	Results on GPU	31
4.2	Speedup of Algorithm on GPU	33
5.1	Results on Multi-GPU	40
5.2	Speedup of Multi-GPU System	41

Chapter 1

Introduction

1.1 Motivation

This is a world of multimedia technology. A better experience of multimedia requires a more powerful capability of computation. However, the traditional way that using CPU as the processing unit has been proved inefficient. For example, a 1-minute video clip in 1080p will need 41 seconds to convert to portable format on a high-end CPU. How to speed up the data processing in the era of an information explosion is very important. In some specific area, DSP and FPGA could help with the issue. Due to the limitation of hardware, they could not be widely applied. So the problem is that do we have any other choice besides CPU to do fast computation? In this thesis project, we would propose a new method to do fast computation on a desktop computer. The DCT/IDCT algorithm would be implemented on both GPU and multi-GPU to demonstrate the advantages of parallel computing on GPGPU.

The discrete cosine transform (DCT)¹ was first introduced by N. Ahmed, T. Natarajan and K.R. Rao[1]. DCT has been the de-facto image transformation in most visual systems. It is widely applied in many video or image coding standards, for example, MPEG, JPEG[2, 3] etc.. However, DCT and inverse DCT are very computational expensive. Therefore, people have been trying very hard to speed them up in two ways. One is to design fast algorithms, and the other is to implement DCT/IDCT on different platforms, such as FPGA. Trainor, D.W. Heron implemented the 2D-DCT using Xilinx FPGA[4]. The reason that FPGA could obtain a better speedup is because hardware itself has inherent parallelism, while the software system is executed in serial. And the DCT/IDCT algorithm is highly parallel.

¹The details of DCT algorithm, includes the algorithm and the properties, will be further discussed in Chapter 3

Parallel structure computing in this case is much faster than serial structure. Also FPGA could be redesigned to optimize or add some features. Therefore, it is more feasible than other platforms. However, to use FPGA as the processing unit, matched peripheral hardware is compulsory and fixed, which means the usage of FPGA is also limited. The best solution for image processing like DCT/IDCT is still on computer, except CPU would be replaced with GPU. In [5], the performance of two-dimensional filters has been compared among four threads CPU, GPU and FPGA running at 100MHz. The final results show that the GPU has the best performance with any filter size, while the performance of FPGA is very stable compared to CPU, which has the poorest performance when the filter size is greater than 5. GPGPU has been proven to be one of the best options to realize heavy computation.

In the past, GPU was only used to accelerate the memory intensive work, such as texture mapping and rendering polygons. Basically, GPU was only related to graphic processing. With the developing of IC technology, especially when the developing speed of GPU is even higher than Moore's law, it is natural that people try to take fully advantage of GPU to do computation more graphic acceleration. Therefore, General Purpose Graphic Processing Units (GPGPU)² research becomes the rising star in the field of high-performance computing (HPC) due to their extremely power of parallel computing. The theoretical computations per second of GPGPUs have surpassed a teraflop years ago. The GPU vendors, AMD and NVIDIA, both released their GPGPU solutions. It also proves the importance of GPGPU. Within a very short time, GPGPU based applications have been developed in many areas, such as medical, deep oil exploration and video converting.

The key feature of GPU is that GPU has a tremendous number of threads, which could work together simultaneously. A certain number of threads could combine together as a block, which processes a specific part of data. This kind of multi-core strategy has been applied on GPU much earlier than CPU. This stream processing architecture[6] releases CPU from the heavy load of computation, and allows GPU to finish the parallel tasks. The GPU based computation usually could speed up the system from dozens of times to hundreds of times. Besides the capability of parallel computation, the power consumption per calculation capability is also higher than CPU, and the cost will be lower. For example, the computing system built on Tesla GPU could deliver the same supercomputing power at 1/10th the cost and 1/20th the power consumption³. According to [7], a reduction of the energy consumption could be up to 83 percent based on several real-world benchmarks. It would be a remarkable improvement for embedded devices to have a better performance and a

²more details about GPGPU could refer to <http://www.gpgpu.org/>

³http://www.advancedhpc.com/gpu_computing/tesla_gpu_s2050_s2070.html

longer duration if this technology could be applied on them.

Though the performance of GPU based computation system is already very satisfying, it is still possible to further improve the performance by building up multi-GPU system. NVIDIA introduced the first multi-GPU system, Scalable Link Interface (SLI), on 2004. In this SLI system, one graphic card will be the master card and controls the other slave card. However, this system is designed for graphic acceleration and also low efficiency. In GPGPU area, since the CUDA language does not support multi-GPU currently, the system would be built up with the help of OpenMP. The performance of the multi-GPU system is supposed to be at least as good as single GPU system. The ideal speedup of the dual-GPU system compared with single GPU should be close to 2X or even more.

1.2 Related Work

From the year when DCT/IDCT was first introduced, people have been trying very hard to find all possible solutions to do fast DCT/IDCT. In [8], a 8 points DCT algorithm was presented, which became the base for many graphic standards. In [9], researchers kept developing fast algorithm to process a single block in an image with smaller computational cost. All these algorithms have their own advantages and disadvantages. For the advantages, they are much faster than the original one. However, they would introduce the blurry effect into the image after decode since the processing is not base on the whole image but blocks by blocks. As mentioned above, many different platforms have been used to implement the algorithm, just like the FPGA platform in [4]. Use FPGA platform could be a better choice if the time is a critical factor in the system.

In GPGPU area, it was very complicated to use GPU for general computing in the past. Since the GPU's old fixed-function pipeline did not allow complex operation, the programmer needs to have a very good command of graphic programming skills[10]. To solve this problem, high-level programming languages have been introduced in recent years. Brook programming, offered by Stanford University, is one of the first programming languages for GPGPU[11]. Some years later, NVIDIA released a more advanced programming model—CUDA. The C based programming language allows the programmer to take advantages of CUDA-enabled GPU without having extensive knowledge of graphic programming. In the very few years after the release of CUDA, engineers and researchers noticed that the potential of this technology in high-performance computing area. Still, very limited research has been done under the multi-GPU system.

1.3 Thesis organization

The rest of this thesis is organized as follows: Chapter 2 describes the basic knowledge about some concepts we need in this thesis. Chapter 3 implement the algorithm on CPU and test it. Chapter 4 addresses the algorithm on GPU and optimize it. The test will also be done and results compared with the previous chapter. Chapter 5 introduces the multi-GPU system and also have the running test on this platform. Chapter 6 demonstrates the conclusions and future work.

Chapter 2

Background

2.1 BMP format

BMP¹ is short for Bitmap, which is introduced by Microsoft. The BMP File Format is capable of storing 2D digital images of arbitrary width, height, and resolution, both monochrome and color, in various color depths, and optionally with data compression, alpha channels, and color profiles. In this thesis report, our concerns are how the image data stored, and how to read important information about the image, such as height and width.

BMP file has four parts of data, bitmap file header, bitmap information header, color table and image data. In this report, what we need is the bitmap information header and the image data.

The bitmap information header is stored in a structure called BITMAPINFOHEADER. By reading BITMAPINFOHEADER, parameters like height and width can be accessed. The image data which we are going to process starts from the 118th Byte.

2.2 CUDA

In the traditional developing path, the GPU serves as an accelerator for 3D display. The performance of GPU has been increasing much faster than any sub-system in computer, including CPU. Now GPU has more powerful computing capability than CPU, especially in float point computing area. Besides the computing capability, GPU also has its own memory system, which offers a bandwidth that CPU system can not offer. It is natural that people try to use this power for calculation. In

¹more details could refer to http://en.wikipedia.org/wiki/BMP_file_format

the past, people have to use very complicated language and API to access GPU resources. Early GPU programming languages were based on shaders² in rendering systems. Later, GPU programming languages were divided into two directions: graphical purpose and general purpose. The early GPU programming languages for general-purpose programming depend on running codes on shaders. It is a very sophisticated way because it needs additional codes to cooperate with GPU. Extensive knowledge about GPU is compulsory to use those languages. This is the reason why people developed high level API for programmers. However, these improvements changed very little in general purpose computing area. When CUDA was launched in 2007 for the first time, a new world started.

CUDA is short for Compute Unified Device Architecture, which is a parallel computing architecture developed by NVIDIA. CUDA comes with a software environment that allows developers to use C as a high-level programming language. It also supports other application programming interfaces, such as CUDA FORTRAN, OpenCL and DirectCompute. CUDA simplifies the way that programmers access the GPU resources. It becomes easier to utilize on-chip memory, DRAM, Streaming Processors or some other hardware components. Since CUDA supports many languages and does not need to command complex knowledge of GPU, it becomes the most popular language in GPGPU area.

2.2.1 Kernel, thread, block and grid

In CUDA, programmer could define functions, which are called as kernels to run on GPU. However, not like C functions, kernels are executed by every thread with a unique given thread ID through the built-in variable `threadIdx`. For convenience, `threadIdx` is a 3-component vector, so it could define thread indices up to three-dimensional, forming a block with three-dimensional tops. Therefore, the algorithm will be divided into two parts, the kernel and the C function as the Figure 2.1 shows.

Most of the CUDA operations are conducted by stream processor (SP), which contains a fused-multiply-add (FMA) unit. This FMA could do one multiply operation, and an add operation. In the program, each SP is mapped to one thread. So, the thread is the most basic unit to execute the computation.

All these SP combined together will be a thread block. Every block has limited registers and shared memory. In most cases, the number of registers is 16KB (In our thesis, the GPU that we use has 64KB share memory). Block is the unit to

²A shader is a set of software instructions that is used primarily to calculate rendering effects on graphics hardware with a high degree of flexibility. Cited from wikipedia.org

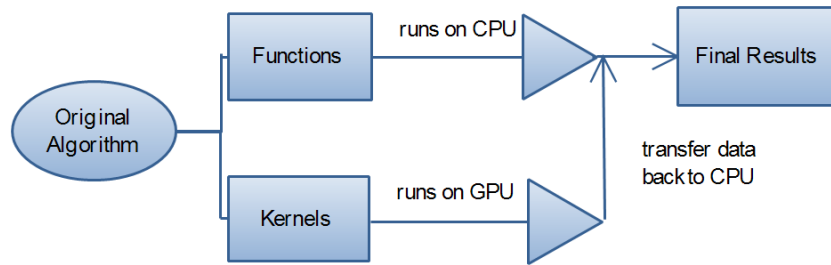


Figure 2.1: The kernel and the function

contain threads. On latest GPUs, a thread block could contain up to 1024 threads, while on most of GPUs, the number is 512. Blocks are organized into the grid, which has up to two dimensions. Figure 2.1 shows the relations between Grid, Block and Threads. Since the Thread, Block and Grid are basic elements to execute a CUDA program, they should be defined before the kernel is going to be executed. For example,

```

...
dim3 threadsPerBlock(16,16);
dim3 NumofBlocks(4,4);
kernelSample <<<threadsPerBlock, NumofBlocks >>> (A,B,C);
...

```

2.2.2 Memory Hierarchy

There are several types of memory in GPU, register, shared memory, global memory, local memory and constant memory.

Register is the fastest one. It is on-chip memory, just like the L1 memory on CPU. Usually, automatic variables without any qualifier will reside in registers.

For shared memory, its scope is block. It means all threads within the same block could access the shared memory belongs to this certain block, while threads from another block could not. Every block has 16KB(64KB on GTX 580) shared memory, and this 16 KB shared memory is divided into 16 banks. If different threads access the same bank at the same time, it will cause bank conflict. Bank conflict means the access has to be serialized, which decreases the throughput. Therefore, it is very important to understand how to address the memory requests to minimize bank conflicts.

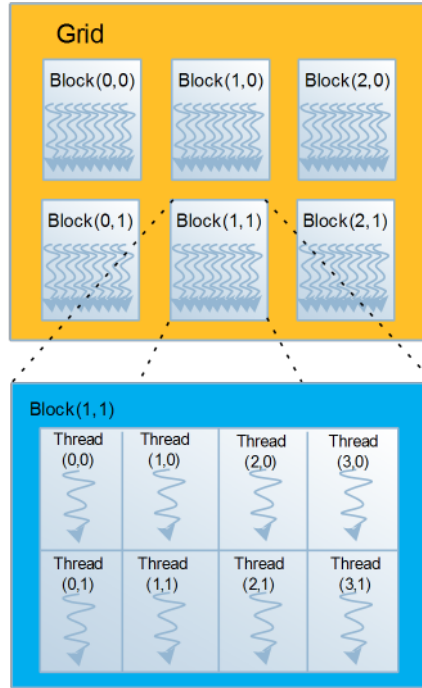


Figure 2.2: Grid,Block,Thread

Though registers and shared memory are on chip, and they are fast to read or write, global memory is the only channel to communicate with the host, which refers to CPU side. It is slow, but important. Any data which is going to be processed should be copied to global memory before it could be read by GPU. In addition, the memory space of global memory should be released by using specific CUDA function `cudaFree()` afterwards.

For constant memory, it resides in device memory, which means the constant memory is fast to access. However it is read-only memory, usually used for some constant parameters, which need to be accessed frequently.

The Figure 2.2.2 shows the relationship between local memory, shared memory and global memory.

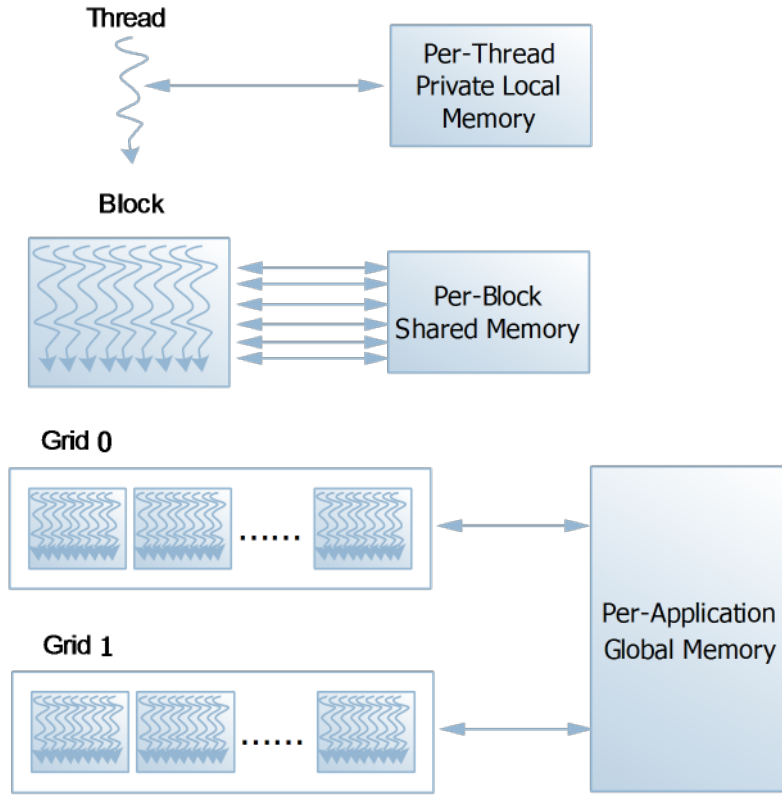


Figure 2.3: Local memory, Global memory, Shared memory and their scopes

2.3 Platform

CPU is i7 950 from Intel, two graphic cards are NVIDIA GTX 580, the operating system is Windows7 (64bit). Intel i7 950 is a 4 cores CPU with 8 threads, working at 3.06 GHz.

NVIDIA GTX 580 is manufactured in 40nm with 3000 million transistors. It runs 772 MHz clock speed, and has 512 CUDA cores also known as Streaming Processor in the past and a 1.5 GB GDDR5 memory running at 4000MHz with a 192.4GB/s memory bandwidth. The architecture for GTX 580 is called Fermi 2.0, which is the latest GPU core of NVIDIA. In Fermi, there are 16 Streaming Multiprocessors at most, and each Streaming Multiprocessor has 32 Streaming Processors. Besides SP, each SM has four Special Function Unit (SFU). In the earliest G80 architecture, the number is 2. Another improvement on Fermi is that each SM has 64KB shared memory while 16KB in the past. Because NVIDIA found that the bottle-neck for computation is the shared memory, instead of a number of threads. So in Fermi,

they cut down the number of threads from 30720 in GT200 to 24576 in Fermi 2.0. Each SM 16 load/store units, allowing source and destine addresses to be calculated for 16 threads per block. These could be addressed to cache or DRAM. This is a very important step to improve the performance while accessing global memory. The software environment is Microsoft Visual Studio 2008. Visual Studio 2008 sup-

Total amount of global memory	1.5GB
16 Multiprocessors 32 CUDA cores	512 CDUA cores
Total amount of constant memory	6.5KB
Total amount of shared memory per block	4.8KB
Clock rate	1.54GHz

Table 2.1: GTX580 Device Information

ports both CUDA and OpenMP 2.0, which is a programming API we will use in this thesis.

2.4 GPU and CPU

GPU has very obvious advantages on processing ability and memory bandwidth. GPU uses more transistors as execute units, while CPU has more flow control units and cache. Hence, some very simple instructions like "if...else..." are very resource consuming on GPU since GPU has very few transistors working on flow control. GPU is well-suited for data-parallel computations, especially those with high arithmetic intensity. GPU will map those parallel data elements on each Arithmetic Logic Unit or ALU, so the large set of data could be handled paralleled to speed up the computation. The following figure shows the difference between CPU and GPU.

2.5 OpenMP

OpenMP stands for Open Multi-Processing. It is an application programming interface (API) which supports multiprocessor programming in C/C++ and other language. The first OpenMP API was published for Fortan in 1997. And the first C/C++ version 2.0 was launched in 2002, then Version 2.5 released in 2005. It is usually applied for multi-thread computation. OpenMP has been implemented in many commercial compilers, such as Visual Studio, GCC above 4.2 and so on. In this thesis, the OpenMP Version 2.0 is the solution to support multi-GPU implementation and implemented under Visual Studio 2008 environment.

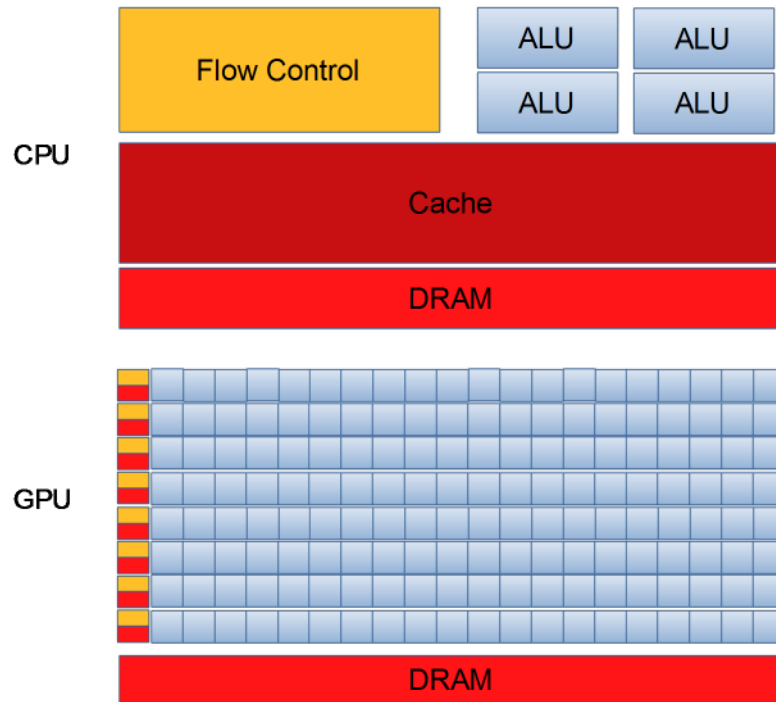


Figure 2.4: Schematic Difference between CPU and GPU

2.6 PCI-Express

PCI-Express stands for Peripheral Component Interconnect Express, which is used for motherboard level interconnection. In this thesis project, when we copy data from host side to device side, PCI-E is used. Since the PCI-E has very limited bandwidth compared to process ability of GPU, it is the bottle-neck if host and device need to communicate very often. In this thesis, GTX580 supports PCI-E 2.0, the peak bandwidth could be 16GB/s.

Chapter 3

Algorithm implementation on CPU

3.1 BMP file operation

Before we start to process an image, we need to read the image data and parameters about image first. The parameters we need are the width, the height, and the bits of every pixel. To do all these jobs, a header file, windows.h, needs to be included.

```
...
fseek ( fp ,  sizeof ( BITMAPFILEHEADER ) , 0 );

BITMAPINFOHEADER head ;
fread ( & head ,  sizeof ( BITMAPINFOHEADER ) ,  1 , fp );

bmpWidth = head . biWidth ;
bmpHeight = head . biHeight ;
biBitCount = head . biBitCount ;
...
```

As mentioned in Chapter 2.1, there are four parts of a BMP file, BITMAPFILEHEADER, BITMAPINFOHEADER, color table and the image data. The information we concern is stored in the latter three parts. So first we need to skip the BITMAPFILEHEADER, and access the information in BITMAPINFOHEADER. Read the returned value from BITMAPINFOHEADER and save them in bmpWidth, bmpHeight, biBitCount. Code fragment is as following. The biBitCount is a number of bits per pixel, which means the BMP could present $2^{biBitCount}$ kinds of color. If the biBitCount equals to 8, a color table needs to be created for gray image, which is the image we are going to process. In this report, all image's biBitCounts

equal to 8.

3.1.1 Bytes align

Since the number of bytes in a line must be multiples of 4, before to use the bmp-Width to read image data, we need to align it. Calculate the number of bytes and then add 3 to make sure the result is up rounded. In BMP file, if the image data in a line is not long enough to make it multiples of 4, the missing data will be replaced by 0. This will not affect on the results of our processing.

```
...
int lineByte=(bmpWidth * BitCount/8+3)/4*4;
...
```

3.1.2 Image save

Since for an 8-bit BMP file, the value of each pixel is supposed to be between 0~255, which means that if the processed data is out of this range, the data should be normalized before save it as an image file.

To implement this function, the processed data need to be an absolute value and the maximum value and the minimum value shall be found. The difference between the maximum value and the minimum value is the scale to normalize the data. And data type of the data should be changed from float type to unsigned char type. Otherwise the final image will be an invalid image.

3.2 DCT/IDCT

3.2.1 Properties of DCT/IDCT

The discrete cosine transform (DCT) was first proposed by N. Ahmed in 1974[3], it has become an extremely important tool for signal and image processing. DCT is a way to de-correlate the image data, just like other transforms. But DCT has its own special properties that it is widely applied to image processing applications. Firstly, DCT could remove the redundancy between neighbor pixels like other transforms could do. Secondly, it could pack data into very few coefficients so compression becomes easier afterwards without introducing visual distortion in the recovered image. Most of the energy is stored in the upper left corner of the DCT coefficients. And in the bottom right corner, usually there is some high-frequency noise, which could be neglected. The third property is that the DCT is separable. A 2D-DCT

could be separated into two 1D-DCT.

Because of these properties above, DCT becomes the base for standards such as Joint Photographic Expert Group (JPEG), Moving Pictures Expert Group (MPEG) and H.261. In many implications, the inverse of DCT (IDCT) is even more important compared to DCT, such as edge detection, object detection and segmentation in MPEG or JPEG. For example, However, these two algorithms are very time consuming. People developed many algorithms to reduce the complexity of computation for DCT/IDCT. In this chapter, we are going to implement the original DCT/IDCT and run them on CPU.

3.2.2 1D-DCT

The DCT of a data sequence $X(m)$, $m=0,1, \dots, (M-1)$ is defined as

$$G_x(0) = \frac{\sqrt{2}}{M} \sum_{m=0}^{M-1} X(m)G_x(k) = \frac{2}{M} \sum_{m=0}^{M-1} X(m) \cos \frac{(2m+1)k\pi}{2M} \quad (3.1)$$

$$G_x(k) = \frac{2}{M} \sum_{m=0}^{M-1} X(m) \cos \frac{(2m+1)k\pi}{2M}, k = 1, 2, \dots, (M-1) \quad (3.2)$$

We can merge these two equations to one equation as,

$$G_x(k) = \frac{\sqrt{2}}{M} c(k) \sum_{m=0}^{M-1} \cos \frac{(2m+1)k\pi}{2M} \quad (3.3)$$

where

$$c(k) = \begin{cases} \frac{1}{\sqrt{2}}, & k = 0 \\ 1, & k = 1, 2, \dots, N-1 \end{cases} \quad (3.4)$$

$$(3.5)$$

The coefficients and the cosine function transform the energy from a time domain to a frequency domain. Most of the energy will be stored in the low-frequency area and very little in the high frequency. As the equation shows, the algorithm itself does not compress any data. It just re-organizes the data so the processed data will be easier to be compressed.

3.2.3 2D-DCT

For a $N \times N$ image, $x(i, j)$ is value of image sample. The 2D-DCT equation will be,

$$G(u, v) = \frac{2}{\sqrt{NM}} e(u) e(v) \sum_{i=0}^{N-1} \sum_{j=0}^{M-1} x(i, j) \cos \frac{(2i+1)u\pi}{2N} \cos \frac{(2j+1)v\pi}{2N} \quad (3.6)$$

where

$$e(u), e(v) = \begin{cases} \frac{1}{\sqrt{2}}, & u, v = 0 \\ 1, & u, v \neq 0 \end{cases} \quad (3.7)$$

$$(3.8)$$

This equation is very complicated at the first sight. However, if we consider equation 3.6 as a combination of two 1D-DCT, we could reconstruct it as,

$$G(u, v) = \sqrt{\frac{2}{M}} e(v) \sum_{j=0}^{M-1} x(u, j) \cos \frac{(2j+1)v\pi}{2M} \quad (3.9)$$

where

$$x(u, j) = \sqrt{\frac{2}{N}} e(u) \sum_{i=0}^{N-1} x(i, j) \cos \frac{(2i+1)u\pi}{2N} \quad (3.10)$$

From eq 3.9 and eq 3.10, to calculate 2D-DCT, first we need to calculate the DCT of each column of the image data, which is a matrix. Then swap the inner loop and outside loop, so the DCT of the each row could be calculated. The final result will be the 2D-DCT. Code fragment will be as following

The coefficients here were calculated before this function was called. Since the coefficients could be reused, pre-calculated values would improve the performance. As the code illustrates, there are two phases in the process, and each phase has three layers of loops. Obviously, it is very computationally expensive if the size of the matrix goes up.

3.2.4 Inverse DCT

For 1D-IDCT, it is defined as

$$f(x) = \sqrt{\frac{2}{N}} \sum_{v=0}^{N-1} \cos \frac{(2x+1)v\pi}{2N} \quad (3.11)$$

and 2D-IDCT is defined as

$$f(x, y) = \frac{2}{\sqrt{NM}} \sum_{u=0}^{N-1} \sum_{v=0}^{M-1} C(u)C(v)F(u, v) \cos \frac{(2x+1)u\pi}{2N} \cos \frac{(2y+1)v\pi}{2M} \quad (3.12)$$

where x, y are the coordinates in the pixel domain, and u, v are the coordinates in the transform domain. The coefficients $C(u)$, $C(v)$ are defined as

$$C(u), C(v) = \begin{cases} \frac{1}{\sqrt{2}}, & u, v = 0 \\ 1, & \text{otherwise} \end{cases} \quad (3.13)$$

Compare equation 3.6 and equation 3.12, we could reform the equation 3.12 in a similar way as we did to equation 3.6.

$$f(x, y) = \sqrt{\frac{2}{M}} \sum_{v=0}^{M-1} C(v) \left\{ \sqrt{\frac{2}{N}} \sum_{u=0}^{N-1} C(u) F(u, v) \cos \frac{(2x+1)u\pi}{2N} \right\} \cos \frac{(2y+1)v\pi}{2M} \quad (3.15)$$

It is the same row-column decomposition technique to separate 2D-IDCT into 1D-IDCT.

3.3 Experiments and Results

We use the algorithm on 5 BMP pictures with different resolutions as 128×128 , 256×256 , 512×512 , 1024×1024 , 2048×2048 . The timer only covers the data processing parts. The result is shown in Table 3.1

Picture Size	128×128	256×256	512×512	1024×1024	2048×2048
Elapsed Time(s)	0.31	2.235	17.762	141.254	1301.66

Table 3.1: Results on CPU

The following pictures are the experiment results of 512×512 resolution. Figure 3.2(a) is the original picture and Figure 3.2(b) is the recovered one with $256^2/512^2$ compression rate.

From these pictures, it is obvious that the program works well and DCT has very good features in loss compression as we indicated. From a normal scope, it could be barely noticed that some information in the picture has been neglected. However, if we watch the picture very carefully, we would see that around the hair part of

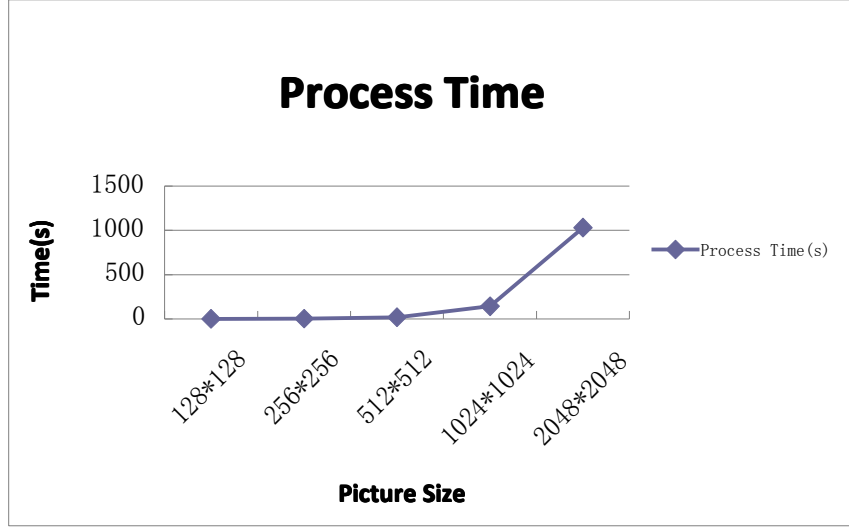


Figure 3.1: Process Period Result

the image, some details have lost and the hair does not seem to be as clear as the original one. This is because the high-frequency part of the data has been discarded and blurring effect is introduced since only one-fourth of the total number of coefficients are utilized to reconstruct the image. This is exactly the second property mentioned previously. And it is also the base for loss compression. Figure 3.1 shows the performance of the DCT and IDCT algorithm on CPU. The unit in this experiment is second.

The processing time increases exponentially with the size of image data. Therefore, if the image has very high resolution, the processing time would be unacceptable. This is also the reason why people try so hard to find solutions to solve this problem. Mentioned above, we assigned 256 (16×16) threads to a block. These 256 threads could access the data simultaneously so the latency could be hidden by them. The reason that 16×16 was chosen is the BMP format is 4-byte aligned, which means for a 8-bit BMP file. The number of image data in a row is multiples of 4. Therefore, the number of threads should also be multiples of 4. For those images, whose width or height is not multiples of 16. We have to fulfill the missing elements with 0 in the image data. So for image with any size, we could process it even it is not the standard size. A specific function called *size_check()* was designed to check if the data size meets the requirement or not.



(a) The original Lena



(b) The reconstructed Lena of a compression rate at $256^2/512^2$

Figure 3.2: Original Lena and processed Lena

Chapter 4

Parallel Algorithm on GPU

4.1 CUDA Programming Model

In this project, we use CUDA C to program parallel algorithm. CUDA C allows the programmer to define C functions called kernels. Once the kernels are called, they will be executed parallel by multiple different CUDA threads. Of course, the pre-request is the functions in the kernels could be executed in parallel.

4.1.1 Kernel

A kernel is defined by using `__global__` declaration. When the kernel is called, specified parameters about grid, block, and thread should be given by using `<<...>>` execution configuration syntax. As mentioned before, the *threads* are used to execute kernel function. Each thread has a unique thread ID that programmer can access in the kernel through a built-in variable *threadIdx*. Similar to thread, block has its own *blockIdx* variable to access kernel. The number of threads per block and the number of blocks per grid should be specified in the `<<...>>`. The type could be `int` or `dim3`. Usually, `dim3` will be used since the programmer could define up to 3-dimensional block or grid.

The block is defined as a 2D block with 16×16 threads, which means each column and row has 16 threads, 256 threads in total. Meanwhile, the grid will be covered by $(bmpWidth/16) \times (bmpHeight/16)$ blocks. Basically, it does not matter if the grid or the block is 1D, 2D or even 3D. Programmer could adjust the model according to the data going to be processed. If the data is an array, then use the 1D block and 1D grid; if the data is a matrix, then use the 2D block and 2D grid to cover the matrix. In our case, each block has 256 threads. Because each block could have 512 threads at most, and more threads in a block, it is more likely to hide latency and improve performance. However, in this case, it is more convenient

to set the block a square one. Meanwhile, the shape of the grid varies with the image data. The specific size is related to the width and height of the image.

In the CUDA programming model, the kernel is executed on a physically separate GPU called *device*, while the other C code runs on CPU, which is so-called *host*. Device and host maintain their own separate memory space in DRAM, which means the programmer has to allocate and de-allocate memory space on each side and transfer the data when it is necessary. For host memory, it is exactly the same as normal C program. For device memory, it could be allocated either linear memory or CUDA arrays. In this project, we used linear memory. And all the allocation and de-allocation are finished on the host side.

4.1.2 Memory Architecture

The CUDA-enabled GPU has several specific designed types of memory that have different latencies and different limitations. In the background, their types and properties have been discussed. The registers are fast but the very limited size read-write per-thread memory in each SP. The shared memory is a low-latency, limited size read-write per block memory in each block. It is usually used in data access within a block. Constant memory is initialized by the host and read-only for all threads in a kernel. It is almost as fast as registers if all the threads reading the constant memory are from the same particular warp. On the contrary, the global memory is a large size but very slow read-write per grid memory. It is the only way for GPU and CPU to communicate with each other[12]. Figure 4.1 shows the data transfer between shared memory, register, thread, global memory and host. If we compare the GPU with Master-Slave computation model, it is not hard to realize that GPU based computation is actually Master-Slave model. The CPU, which is called as host, allocates memory space and assigns computation tasks for GPU. On the device side, the GPU will finish the job parallel through different threads, which could be considered as slaves. The communication between them is via PCI-E interface. Even for the latest PCI-E 3.0 standard, it could only carry a bit rate of eight giga transfer per second. Obviously, the bottle-neck is the communication between host and device. In an experiment, we did in the thesis, the time spent on data transfer from the host to device cost 163.28ms; time spent on calculation was 30.32ms and time for transferring the data back to host cost 1015.4ms. In another word, most of the time, that GPGPU requires is spent on data transfer between host and device. Therefore, the key to design an algorithm with high performance is that reduce the data transfer between CPU and GPU.

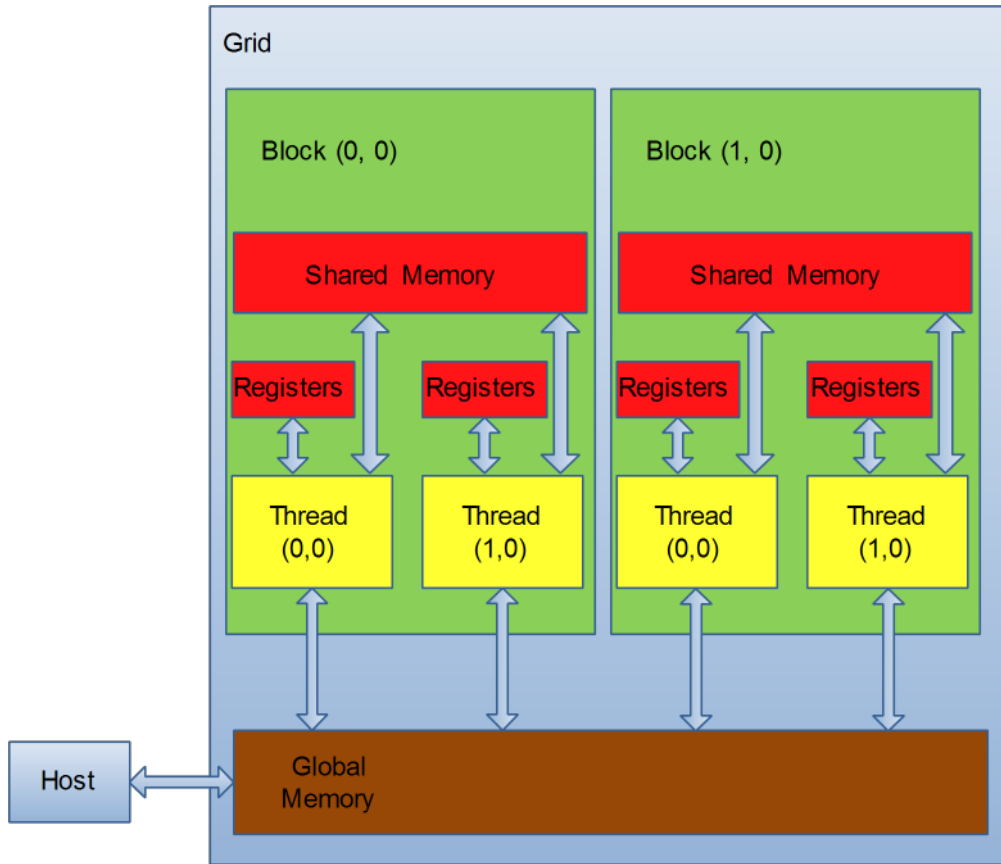


Figure 4.1: CUDA Memory Architecture and Host-Device Data Transfer

4.1.3 Hardware Implementation

The CUDA architecture is built around a scalable array of multithreaded *Streaming Multiprocessors* (SMs). When a CUDA program on the host CPU invokes a kernel grid, the blocks of the grid are enumerated and distributed to multiprocessors with available execution capacity. The threads of a thread block execute concurrently on one multiprocessor, and multiple thread blocks can execute concurrently on one multiprocessor. As thread blocks terminate, new blocks are launched on the vacated multiprocessors[12]. Multiprocessor creates, manages, threads in a group of 32 threads called warps. When one or more blocks are assigned to a multiprocessor, it partitions them into several warps by a warp scheduler. Each multiprocessor has 32-bit registers that are partitioned for the warps, as well as shared memory is partitioned for thread blocks. Therefore, in the view of multiprocessor, it owns all the registers and shared memory. In the view of blocks and threads, they have to share the resources with others, which means in some cases, if another thread or

block needs the resources more badly. They have to vacate the resources. This is a very important feature to analyze the performance.

4.2 GPU-Parallel Algorithm

The whole algorithm could be divided into two parts, CPU part and GPU part. GPU part is the core algorithm, and CPU is in charge of initialization and data transfer between host and device. It also calculates the time cost by GPU. The flow char of the program is as following.

4.2.1 Initialization

As mentioned above, we assigned 256 (16×16) threads to a block. These 256 threads could access the data simultaneously so the latency could be hidden by them. The reason that 16×16 was chosen is the BMP format is 4-byte aligned, which means for a 8-bit BMP file, the number of image data in a row is multiples of 4. Therefore, the number of threads should also be multiples of 4. For those images, whose width or height is not multiples of 16. We have to fulfill the missing elements with 0 in the image data. So for image with any size, we could process it even it is not the standard size. A specific function called *size_check()* was designed to check if the data size meets the requirement or not.

After the data size is verified, the program starts to allocate memory space on both *host* and *device*. We use a CUDA API function to allocate memory space on the device, which refers to GPU. This function is *cudaMalloc()*¹. Besides memory allocation, since the GPU could only process data of float type, and the data read from BMP is type char, we need to convert the data type before we transfer the data to GPU. Also we calculate the coefficients on CPU to save some time. A function called *uchar2float()* is designed for this. We transfer the image data and calculated coefficients by calling another CUDA API function *cudaMemcpy()*². All the data copied to GPU is stored in *global memory*, which is the only way to communicate between GPU and CPU as explained. Till now, all needed data has been allocated to the device.

4.2.2 Kernel Function

In the kernel function, since all the necessary data is stored in global memory, which is much slower compared to register and shared memory. So it is better to use reg-

¹refers to Nvidia, CUDA C Reference Manual Version 2.0, page 25

²refers to Nvidia, CUDA C Reference Manual Version 2.0, page 34

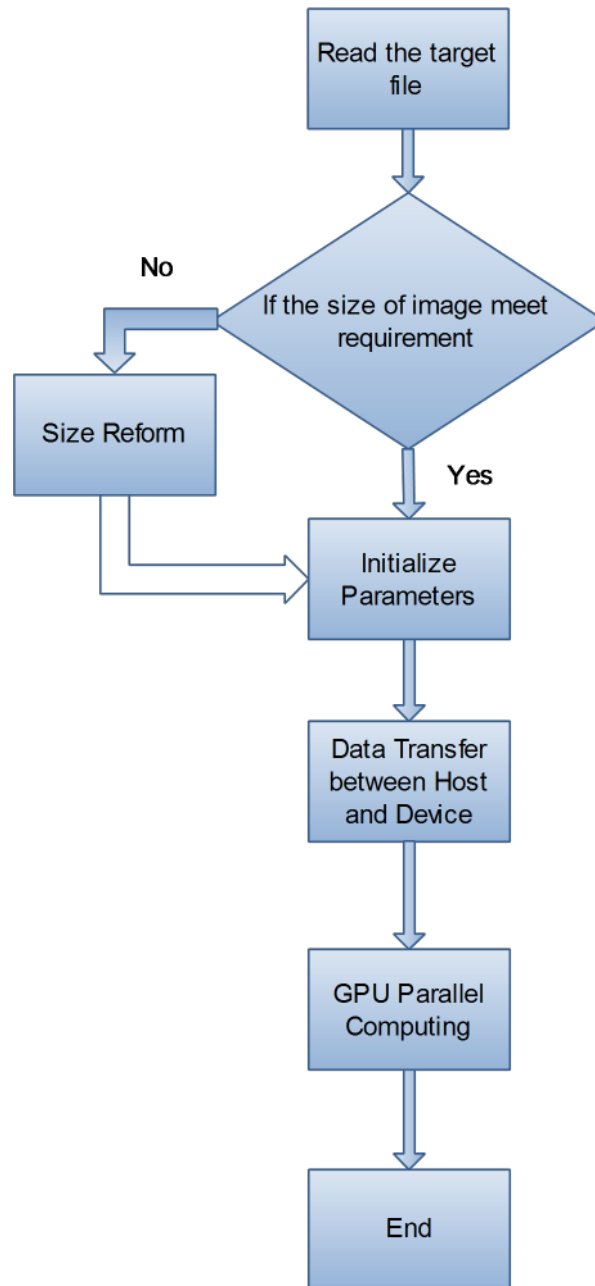


Figure 4.2: Flow Chart of the Algorithm on GPU

ister to keep those data which will be called frequently. However, since the register is limited, it is impossible to store the whole image data. Only coefficients will be copied to registers.

As mentioned in Section 4.1.2, the kernel function will contain a certain number of blocks, which has 256 threads in each one. In the kernel function, the blocks are used to cover all the data in order to make the data processing as parallel as possible. Since the size of block is fixed and the size of image data might vary, the number of blocks in a kernel varies with the size of image data. For example, in our code, we defined the block a 2D block with 16×16 threads, and the grid is a 2D grid with $\text{bmpWidth}/16 \times \text{bmpHeight}/16$ blocks.

```
...
dim3  grid( bmpWidth/16, bmpHeight/16, 1);
dim3  threads(16, 16, 1);
...
```

In the following parts, we will demonstrate the program by using a 512×512 size picture as an example. In this case, the whole image data will be covered by $\frac{512}{16} \times \frac{512}{16} = 1024$ blocks. We used six built-in variables to present the information about the blocks and threads in the kernel. They are *threadIdx.x*, *threadIdx.y*, *blockIdx.x*, *blockIdx.y*, *blockDim.x*, *blockDim.y*. The *threadIdx.x*, *threadIdx.y* present the coordinate of a thread in the block. Similarly, *blockIdx.x*, *blockIdx.y* are the coordinate of a block in a grid. And *blockDim.x*, *blockDim.y* indicate how many threads in each dimension of a block, in this case, they are 16. Therefore, we could use these six built-in variables to present any thread in any block.

```
...
int tx =threadIdx.x;
int ty =threadIdx.y;
int row=blockIdx.y*blockDim.y+threadIdx.y;
int col=blockIdx.x*blockDim.x+threadIdx.x;
...
```

With the *tx* and *ty*, we could determine the specific thread in a block, because they present the ID of each thread. With the *row* and *col*, every thread and its corresponding data element could be located by block ID and thread ID. Then we could reform the algorithm in a simpler way.

```
...
for (int f=0;f<size_t ;f++)
{
if (row==0)
y=d_temp[row*size_t+f]*coeff_[0]*
cosf((2.0*f+1)*col*PI/(2.0*bmpWidth));
else
y=d_temp[row*size_t+f]*coeff_[1]*
```

```

cosf((2.0*f+1)*col*PI/(2.0*bmpWidth));
    }
    ...

```

Then store the results in the allocated memory space. Since this is just 1D DCT, we have to call another kernel again to calculate the 2D DCT. However, this is just a raw version. The kernel has to access global memory every time it needs to read image data. As we already knew, global memory has very long latency to access. Also there is an *if* statement in every *for* loop. Due to the limited number of transistors dedicated for flow control in GPU, *if* statement consumes quite a number of clock circles every loop.

4.2.3 Optimization

In CUDA programming, how to optimize the program is a very important issue to solve. Generally speaking, the goal of optimization is to take fully advantage of hardware. The hardware resources in CUDA could be classified as memory resource, multiprocessor resource. Basically, these two resources are the foundation to calculate. For memory resource, we have registers, shared memory constant memory and global memory. Since the registers, shared memory and constant memory are much faster than global memory, one way to improve performance is to use the on-chip memory instead of global memory, which is off-chip memory. In our case, the original algorithm has to access the global memory every time it needs the image data. Since the global memory is slower than on-chip memory such as shared memory or registers, we need to use on-chip memory to replace global memory. It is not possible to read all the image data and store it in the registers because the size limitation of registers. Therefore, the only option is shared memory. Shared memory has a larger storage space than registers, and it is also on-chip memory. Shared memory likes the L-1 cache in CPU. It is not in GPU core but still on-chip. Figure 4.1 shows the relationship between registers, shared memory and global memory. The shared memory in each kernel is the same size as the size of each block, which is 16×16 . The shared memory is defined as,

```
__shared__ float stemp[16][16]
```

Shared memory is the most-used option to optimize performance since it reduces the data transfer between GPU and global memory. Compared with registers, shared memory is fast with a proper size of memory space. For CUDA compute capability of 2.X, the maximum amount of shared memory is 48KB while for compute capability of 1.X, the maximum amount of shared memory is 16KB. Despite the limited size of shared memory space, it is not programmer's concern to distribute

the shared memory among different multiprocessors. The CUDA system will allocate the shared memory to the multiprocessor who needs it most, and vacate the space for the next multiprocessor.

To use shared memory, the process should have three steps. First, the threads in the block copy the sub domain they are going to process from global memory to shared memory. Second, the computation will be done by all the threads using the data in the shared memory. At last, the results will be sent back to global memory before the kernel ends. In the whole process, the data transfer between shared memory and global memory will create overhead, as well as the creation and vocation of shared memory. Hence, the arithmetic intensity of the kernel should be sufficiently large to compensate for the overhead of data copying in order to benefit from the shared memory implementation[13].

If we check the 2D-DCT algorithm again, we would notice that in the 1D-DCT part, each column of the image data will be read from global memory and used in a for loop. And then, each row of the image data will be read for the next round of processing. Three new variables are defined to make sure needed block of data will be read and stored. *till4dian05fen*

```
int Begin = 16 * blockIdx.x;
int Step = 16 * size_t;
int End = Begin + (size_t - 1)*size_t
```

Since the block size is 16×16 , the *Begin* indicates the first element in the row in each block. The *size_t* here equals to the width of the image data, while in the next round of DCT process, *size_t* equals to the height of the image data. So a *Step* means the program jumps from the previous element to the first element in the next row of blocks. The *End* will be the last corresponding element in the last row of blocks. Then the original for loop will be modified into a new one. It is a 2-layer for loop. From the definition of *Begin*, *Step*, *End*, there will be $(Begin - End)/Step$ times to execute, which is obviously less than *size_t*. So another for loop is necessary to cover all image data in the global memory. As the Figure 4.3 demonstrates, the *Begin* was defined as a variable related to *blockIdx.x*, the function will fetch data one row after another. And one step is the distance between adjacent blocks in the same column. Therefore, it reads every element in a column first, then with the increasing of *blockIdx.x*, the data in the next column will be fetched. All the data fetching would be executed in parallel by threads in the block. Before the inner loop starts, a `__syncthreads()`³ function should be called. The function is used to set a synchronization points to coordinate memory access. It acts as a barrier where all

³CUDA C Programming Page, 22

threads in the block have to wait until they are allowed to proceed. Otherwise, some threads might finish their tasks earlier than others, and it would cause an error if expired data was taken as the updated one. If there is no `--syncthreads()` here, the fetched data in the inner for loop might be wrong. Because it is possible that the outer loop is being executed, and the data has not been updated yet for the inner loop. In the inner for loop, these 256 threads will each column in the shared memory and use the data to calculate. However, this inner for loop is done by one single thread. Meanwhile, the other threads are executing these for loops at the same time. When all these jobs are done, the kernel has to synchronize all threads again to make sure the updated data will be stored.

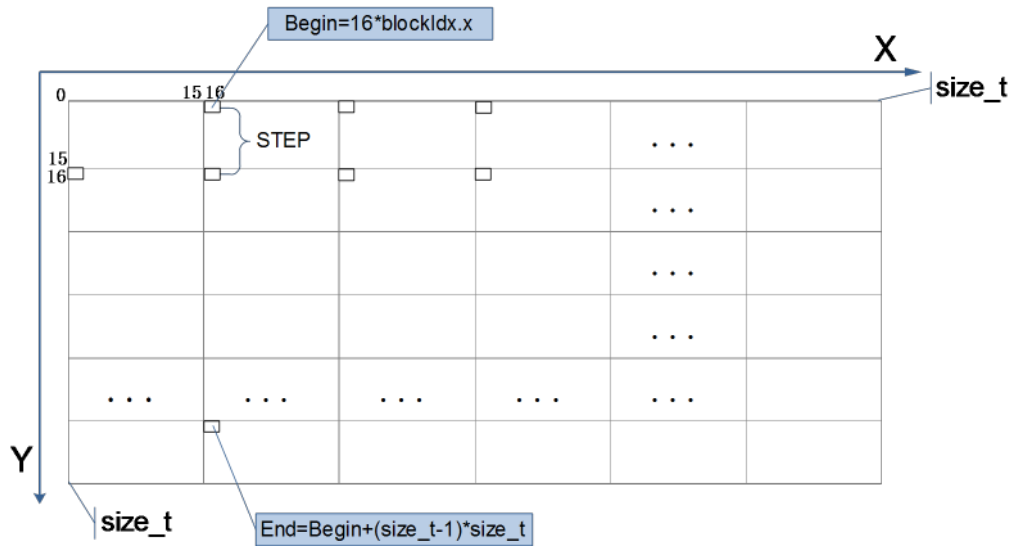


Figure 4.3: Shared Memory Scheme

```

for (a=Begin; a<=End; a+=Step)
{
    stemp[ty][tx] = d_temp[a + size_t * ty +tx];
    --syncthreads();
    for (k=0; k<16; k++)
    {
        if (row == 0)
        ...
        else
        ...
    }
    --syncthreads();
}

```

} }

Besides the shared memory, how to active more blocks in the processing is also important. For each Streaming Multiprocessor(SM), it should usually contain 6 8 blocks, which mean the total number of blocks should $8 \times \text{SM}$. On the GTX 580 graphic card, there are 16 SMs in total. From this angle, we could conclude that the less blocks the better. In another word, the size of block should be large. However, once the shared memory is introduced to the program, the number of blocks is defined by the size of shared memory. Since the shared memory is only available for the threads in the same block, how many pieces of shared memory you defined in the program determines the number of blocks. Meanwhile, shared memory is very limited but fast. If the size of block is very large, only very few blocks could have shared memory, which means the number of active blocks would drop down. Hence, from the angle of shared memory distribution, the size of block should not be too large. Also, another factor need to be considered is what we discussed in Section 4.1.1, the block is supposed to contain more threads to achieve a better performance.

From the above discussion, it is obvious that there is a trade-off between shared memory and the number of blocks. Optimization with inappropriate configurations would worsen the performance.

4.2.4 Timing

In CUDA, there are several ways to monitor the process. The most accurate way is *event*. CUDA events could record events at any point of the program and query when these events are completed. It is the best method to record each single event in CUDA. However, in this thesis the time we need to measure is whole CUDA kernel instead of some independent events. This is a better choice to measure kernel instead of the event. CUDA runtime API offers an internal soft timer. To use the run time API, `cuda_runtime.h` has to be included. This API is a C++ style interface built on top of the C API so it can be compiled by any C++ compiler. The soft timer measures the time in CPU. The timer could offer accuracy in milliseconds. It is also easier to implement. First, the timer should be created in the initialization phase. Then before the target process starts, the timer will be actived to monitor the process until it is stopped. `cutGetTimerValue()` could access the timer, and the return value is the time elapsed. After all measuring task is completed, the timers should be deleted.

As we mentioned in Section 4.2, before the data transfer, there are three phases

executed in CPU. The executing time on CPU is quite long. Therefore, the peripheral execution has a very limited effect on the total time. But for executing time on GPU, the calculating time is so short that we could not include the initialization phase into the timing zone. What we concern is the time spent on computing, hence our timer starts to work when data transfer between host and device starts. The timer ends when the processed data was copied back to host.

4.3 Experiment and Results

The same BMP files were used for the experiment. Since the speed of GPU algorithm is very fast compared to CPU algorithm. On CPU, time spent on calculation is in seconds or even thousands of seconds. But on GPU this number is in milliseconds. Also, the running period is related to the CPU usage and GPU usage. Any running program in the background might have effects on the final results. Therefore, it is necessary to run the test multiples of times to get the average data and also make sure that unnecessary processes were killed before the test. The Table 4.1 contains the results of non-optimized(NO), optimized without shared memory resize(O) and optimized with shared memory resized(OSMR)⁴. All the units in this experiment are in millisecond.

Picture Size	128×128	256×256	512×512	1024×1024	2048×2048
Elapsed Time NO(ms)	83.4012	110.4569	182.2122	726.9291	5330.0174
Elapsed Time O(ms)	155.4558	123.7503	199.8524	907.8359	6371.9727
Elapsed Time OSMR(ms)	84.6638	122.1142	210.5162	845.7852	5821.6899

Table 4.1: Results on GPU

From the above table, it is obvious that the optimized without shared memory could not improve the performance at all. The time spent on the 128×128 image is almost as twice as the non-optimized algorithm. As the discussion in Section 4.2.3, the size of shared memory defines the number of blocks, the number of blocks is related to the performance. Therefore, we decided to reduce the size of shared memory and see if the performance could be better. We changed the size of shared memory from 16×16 to 8×8 , as well as other parameters related to the size. And we have another series of results, marked as OSMR. Now the performance is better than the previous ones, but still not as good as the original algorithm. Some possible reasons to explain the phenomenon are

⁴the block size is set to 8×8

1. One of the main improvements of Fermi's architecture is the data transfer rate between GPU and global memory. And the reason we want to use shared memory is it could reduce the time to access global memory to improve performance. Now since the data-transfer speed is faster, the shared memory might not be able to improve the performance. If we consider the overhead caused by shared memory, it is possible that shared memory could worsen the performance.
2. The size of shared memory in Fermi is 48KB, which means it could handle only 12K floating points in total. If the block size is 16×16 , we will have 256 floating points in each block. Therefore, only $12K/256 \approx 47$ blocks would be activated, which means about 8 SM will be running considering that each SM would have 6 to eight blocks. As we mentioned in the Section 2.3, there are 16 SM in total, so the hardware resources could not be fully utilized.

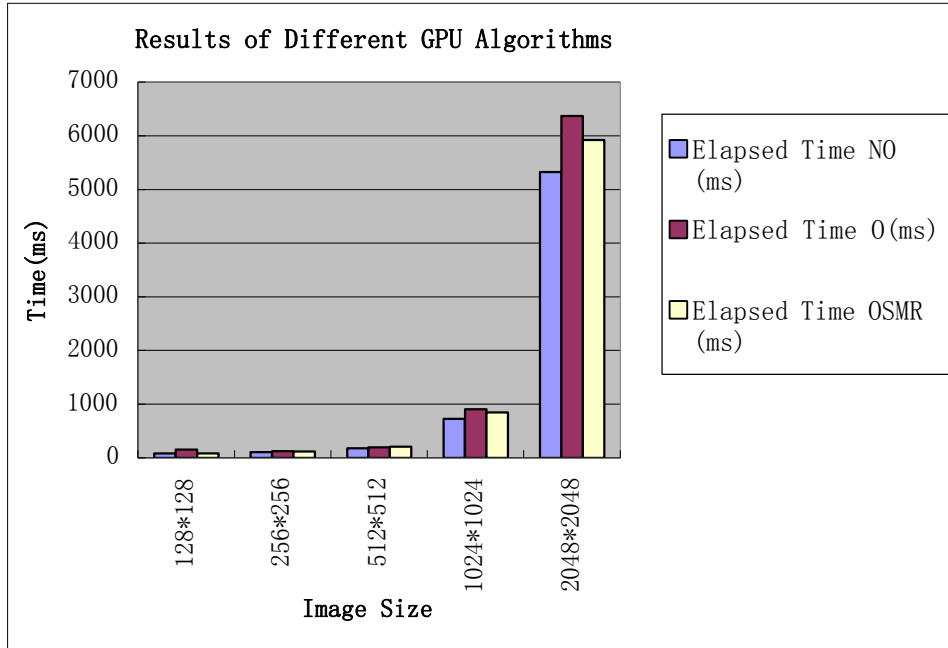


Figure 4.4: Speedup of GPU Algorithm

Even the optimized results could not meet our expectation, it is still good enough if we compare it with the algorithm running on CPU. Compare the results with the data in Table 4.1, the speedup for each resolution could be illustrated in the following Table 4.2 and Figure 4.5

Picture Size	128×128	256×256	512×512	1024×1024	2048×2048
Speedup	3.73	9.05	97.5	194.3	245.2

Table 4.2: Speedup of Algorithm on GPU

Since the non-optimized algorithm has the best performance, we will compare these results with the results from the algorithm running on CPU. As we already knew, the speedup should be related to the amount of data. If the size of image is not large enough to cover the overhead, the performance could be worsened. From Table 4.2 and Figure 4.5, we can find that when the data size is larger than 1024×1024 , the speedup of more than 200 times could be gained.

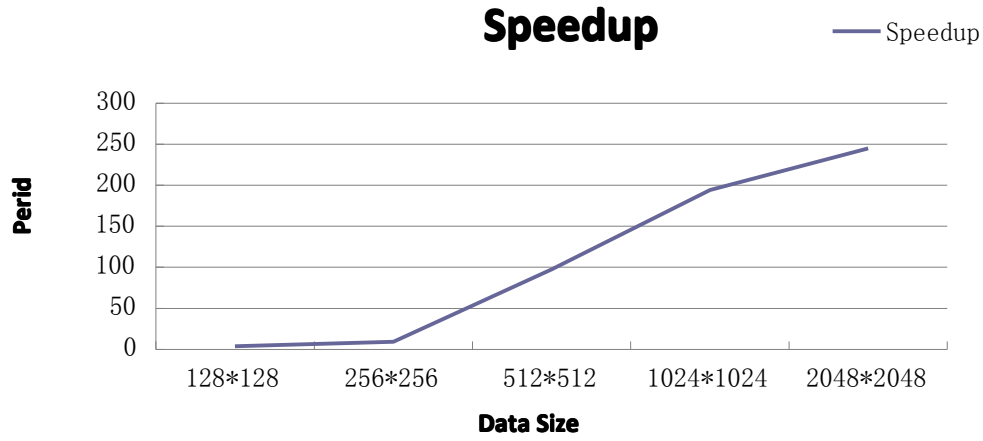


Figure 4.5: Speedup of GPU Algorithm

Chapter 5

Algorithm on Multi-GPU System

Even the GPGPU has already offered very powerful capacity of computing, people still keep looking forward to a better performance. Multi-GPU system is an improvement to have an even more powerful computing system. Since the CUDA itself does not support multi-GPU, we have to take advantage of other multiple threads language. OpenMP is a well-designed language for multi-thread execution. In this thesis, what we implemented is dual-GPU system as Figure 5.1 shows. We will divide the work into 2 halves and tag them with unique ID, then assign them to GPUs to have partial results. In the end, the CPU will combine the results and generate the final result.

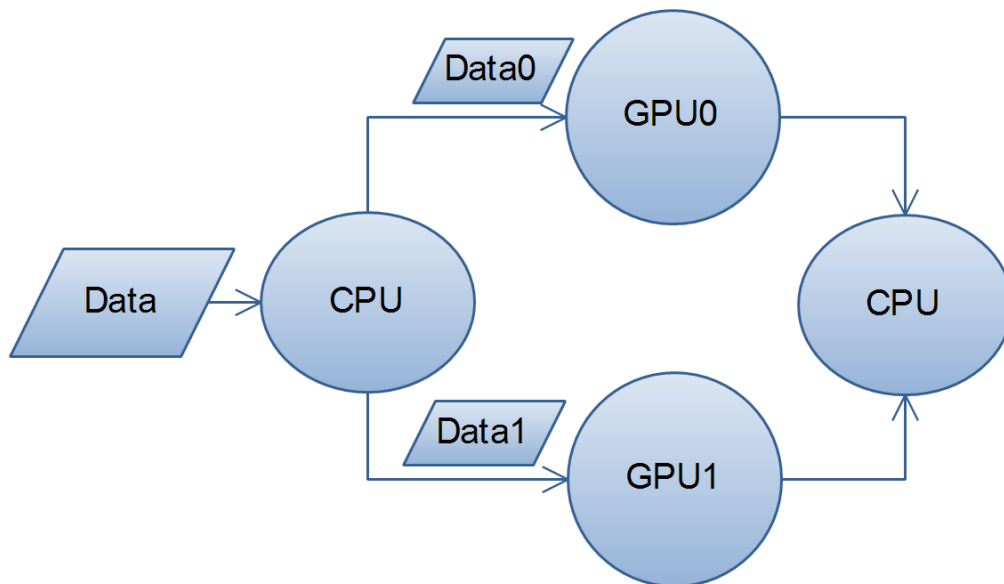


Figure 5.1: The Dual-GPU Schematic

5.1 OpenMP Execution Model

The execution base for OpenMP is the fork-join model. OpenMP supports programs that execute correctly both as parallel programs and as sequential programs. Usually, the OpenMP program is divided into three parts, single thread, multi-thread and single thread again. The program written with the OpenMP support starts to execute as a single thread called the master thread. The master thread initializes the program for multi-thread part. When a parallel construct is encountered, the master thread creates a team of threads[14]. In our thesis project, since there are only two GPUs in the system, only two threads are needed. Though it has no problem if more threads exist. The multi-thread part will be executed by all threads simultaneously. Those threads, called as worker threads, split the work and synchronize the results at the end of multi-thread part. Then the master thread will collect the processed pieces of data and combine them to be a final result. During the whole process, all threads have to access the globally shared memory. Data could be shared or private. Shared memory could be accessed by any thread, while the private memory is only available to the owner thread. Each thread will have a local copy and use it as a temporary variable. Either master thread or worker thread has its own thread ID. The master thread always has a thread ID 0. When the master thread encounters a parallel construct, which starts with `#pragma omp parallel`, a team of threads will be created¹. And the number of threads could be set by the programmer. These threads execute on different hardware threads with the corresponding thread ID until the end of parallel constructs. During the process, each thread won't communicate with each other. Threads only communicate with the master and do their own job. The following figure illustrates the fork-join model of OpenMP.

To support OpenMP, C/C++ programs need to include `<omp.h>` file. In `omp.h`, necessary API functions are included. Four critical API functions are used in this thesis project, they are `omp_get_num_procs()`, `omp_set_num_threads()`, `omp_get_thread_num()` and `omp_get_num_threads()`. The `omp_get_num_procs()` returns the number of processors that programmer could use. The `omp_set_num_threads()` sets the total number of threads that will run in the program, usually equals to the number of GPUs. The `omp_get_thread_num()` returns the thread ID in order to set the corresponding hardware. For example, CPU 0 manipulates thread 0. The `omp_get_num_threads()` returns the total number of CPU threads. All mentioned above is the general model of OpenMP programming. In our case, OpenMP is the way to manipulate two GPUs at the same time. The prerequisite is CPU should have at least two threads so each thread could control one GPU.

¹for more information, refers to [http : //www.openmp.org](http://www.openmp.org)

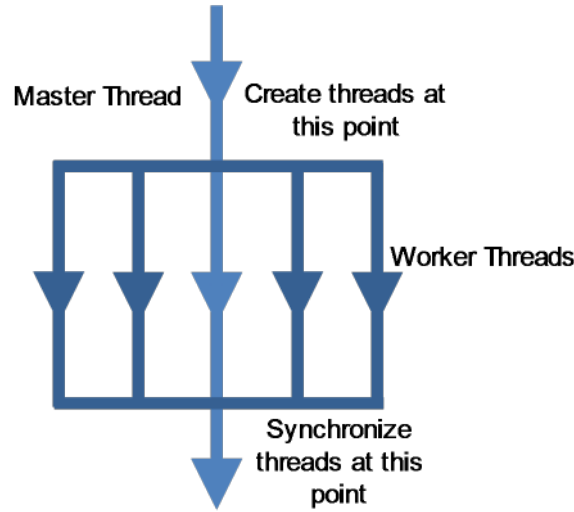


Figure 5.2: Fork-joint Execution Model

5.2 Multi-GPU System Implementation

At the end of last section, we mentioned that we will use OpenMP to manipulate two GPUs simultaneously. Usually multi-thread structure in OpenMP is used to do computation. But in this thesis, the OpenMP is mainly used to control. On our platform, there are a 4-core CPU with 8 threads and two graphic cards. So the hardware meets the requirements of OpenMP that at least one thread for one graphic card. It has no side effect if more CPU threads exist. Before we start to implement multi-GPU algorithm, we need to check if the algorithm could be split into 2 halves.

There are two methods to divide the whole computation, one is data division, and the other is *for* loop division. As the algorithm discussed in Section 3.2.3, every DCT or 2D-DCT coefficient has very strong relativity with the elements in a whole row or column. Therefore, in order to get any coefficient, a complete row of data or column of data is necessary. It is possible if the whole data matrix is divided into two parts, and each part has its own rows and columns to get the corresponding coefficients. However, it is impossible to implement data division for 2D-DCT. As the algorithm shows, the program will check if the data element is from the first row or not. If it is from the first row, the coefficient will be different from others. And both of the GPUs have the exactly the same program to run. This leads us to a serious problem. As the definition in Section 4.2.2,

```

...
int row=blockIdx.y*blockDim.y+threadIdx.y
...

```

The value of a row is only related to *blockIdx.y*, *blockDim.y* and *threadIdx.y*. These three variables are built-in variables of CUDA system. They always start from zero, which refers to the first row of threads. Therefore, if we divided the data matrix into two parts, the first row of the second half data would be considered as the first row instead of the middle of rows. And the coefficients the program chose would be incorrect. Therefore, it is impossible to implement data division for 2D-DCT on the multi-GPU system. Besides all mentioned above, if we separate the data matrix, each half only needs half of the blocks compared with the algorithm in Chapter 4. As the Section 4.1 indicates, if the amount of data is not enough, the advantage of hardware could not be fully utilized. Due to the lack of blocks, some of the multiprocessors might be in idle. The method left is to split the for loop and assign the split loops to GPUs so the work load could be assigned evenly to each GPU.

The second method is splitting the *for* loop. The complete *for* loop in the algorithm in Chapter 4 will be split into 2 halves. Each thread takes one-half of the for loop and the whole data matrix. The CPU thread ID will decide which half of the for loop should be done by which GPU. In our project, CPU0 controls GPU0 to calculate the first half of the *for* loop, while CPU1 and GPU1 for the other half. In this way, the if statement still works correctly. The reason is that the data matrix is still complete and every block covers the original data elements. The for loop will be modified as,

```

...
for (int f=0+CPUId*size_t/2; f<size_t/2+CPUId*size_t/2; f++)
...

```

One part of the program is serial part. Besides the content discussed in Section 4.2.1, the program will call CUDA API function *cudaGetDeviceCount()* to check the number of GPUs in the system. If the number is less than 2, hardware driver needs to be checked. If the hardware has been checked, the program starts to initialize threads and other parameters. The memory allocation will be modified differently. To store the temporary results and the final results, two different size arrays will be created on host memory due to two threads will have two temporary results. The size of the first array is twice as the second one, because this array will save the temporary results returned from both GPUs. The program will deal with

these results and save the final ones in the second array.

All above is about the serial part. For the parallel part, it starts from *#pragma omp parallel*. The ID of each will be assigned with the thread number obtained by the API. And the GPU will be set according to the CPU thread ID in order to build connections between CPU threads and GPUs. Then all the data will be copied to the kernel before the kernel starts. Since it is the kernel running in the parallel part, there are actually two kernels running on two GPUs controlled by two CPU threads. And the temporary results will be stored in the temporary array as mentioned before. The results from the first GPU will be copied to the first half of the array, and the results from the second one will be copied to the other half. This is the reason why this temporary array needs twice the space.

```

...
omp_set_num_threads(NGPUS);
float *firstOMP=(float*) malloc(sizeof(float) * size * 2);
float *secondOMP=(float*) malloc(sizeof(float) * size );
...
#pragma omp parallel
{
    unsigned int CPUThreadId = omp_get_thread_num();
    unsigned int CPUThreads = omp_get_num_threads();
    int GPUThreadId = -1;
    cutilSafeCall(cudaSetDevice(CPUThreadId % NGPUS));
    cutilSafeCall(cudaGetDevice(&GPUThreadId));
    ...
    cudaMemcpy(firstOMP+CPUThreadId*size, d_temp2,
        sizeof(float) * size, cudaMemcpyDeviceToHost);
    ...
}
...

```

Since the default value for the first thread is 0, the GPUThreadId is set to -1 in order to avoid errors. And the result address is determined by the address of the first temporary array, CPU thread ID and the size of results. This is similar to the address problem in CUDA. All the addresses are completely related to the thread ID no matter CPU ID or GPU ID. In parallel computing, the thread ID is the most important tag to find the correct address. It is the only connection between hardware thread, memory space and data. With the decomposition of for loop, the original loop has been divided into two even parts and assigned to each GPGPU.

But the GPU still has to send the results back to host, which is extra overhead of using multi-GPU. We also need to set a synchronization point before we start to put the results back together.

5.3 Experiment and Results

To get the more accurate results, we did the experiments for 5 times. The followings are the results table and the histograms from Figure 5.3 to Figure 5.7. All the units in this experiment are in millisecond.

Picture Size	128×128	256×256	512×512	1024×1024	2048×2048
Test 1 (ms)	79.56	80.44	124.01	451.72	2812.47
Test 2 (ms)	113.33	112.36	150.17	452.59	2841.11
Test 3 (ms)	68.76	124.05	167.82	462.65	2820.07
Test 4 (ms)	102.63	77.81	163.69	460.38	2811.48
Test 5 (ms)	84.56	119.79	156.14	455.16	2838.12
Average (ms)	89.76	102.89	152.35	456.60	2824.65

Table 5.1: Results on Multi-GPU

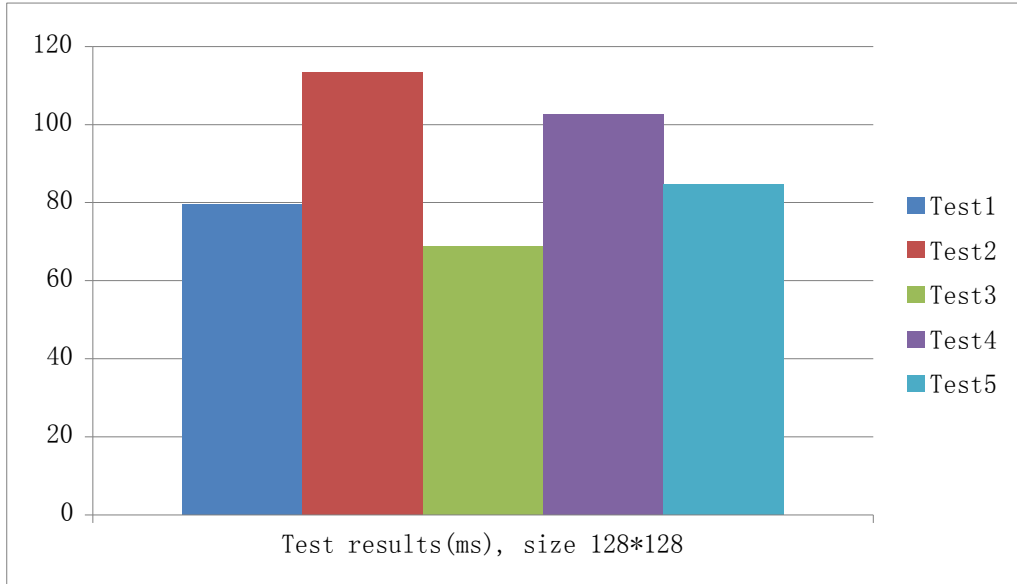


Figure 5.3: Results for image size of 128×128

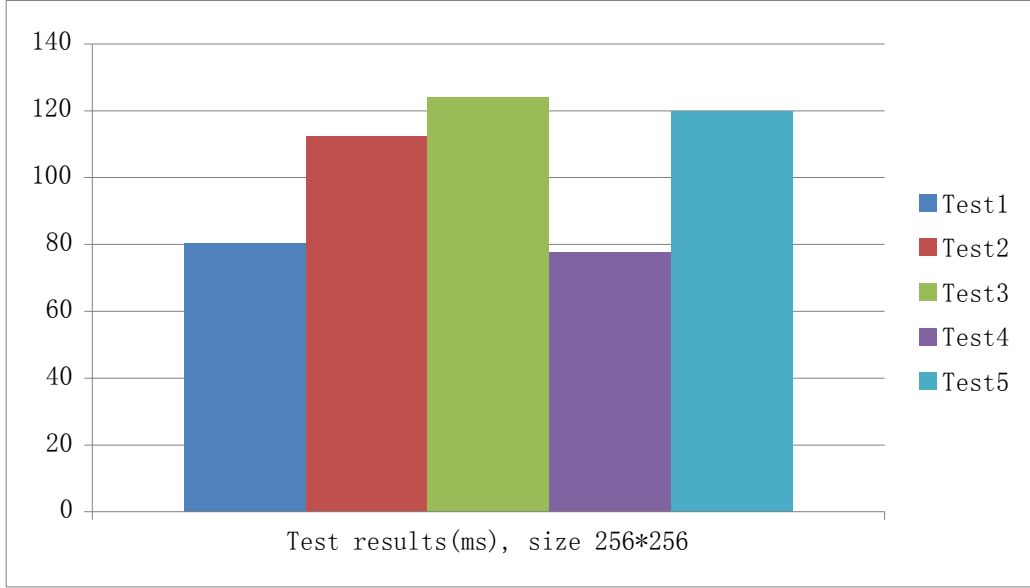


Figure 5.4: Results for image size of 256×256

From Table 5.1, we would notice the performance of the multi-GPU system is not very stable while the image size is small. While the data amount becomes larger, the performance becomes more stable. The histograms display the results in a more straightforward way. The main reason for the unstableness is the proportion of computing time is so small that the final result is highly effected by the rest rather than computing, such the initialization of OpenMP.

If we compare the results of multi-GPU with the results of single GPU, we would have the following table and the corresponding figure.

Picture Size	128×128	256×256	512×512	1024×1024	2048×2048
Time on single GPU(ms)	83.40	110.46	182.21	726.93	5330.02
Time on Multi-GPU(ms)	89.76	102.89	152.35	456.60	2824.65
Speedup of Multi-GPU(ms)	0.93	1.07	1.20	1.59	1.89

Table 5.2: Speedup of Multi-GPU System

From the above table, it is obvious that the speedup gets better when the image

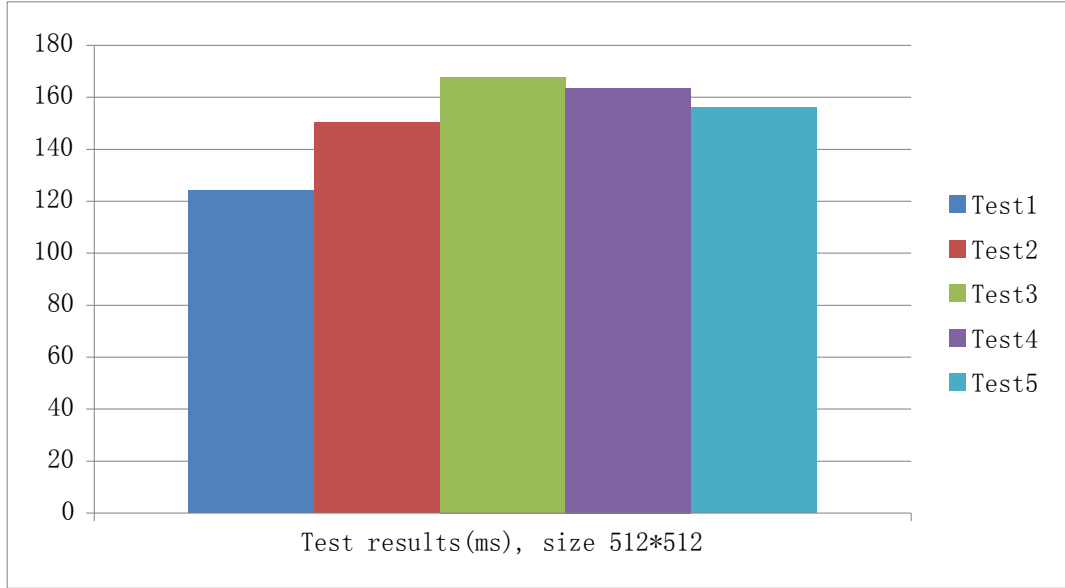


Figure 5.5: Results for image size of 512×512

size goes larger. It means that the multi-GPU is more suitable for the kind of computation that costs more than several seconds. And the best speedup in our experiments is close to 2X, which means the efficiency of the multi-GPU system is also acceptable.

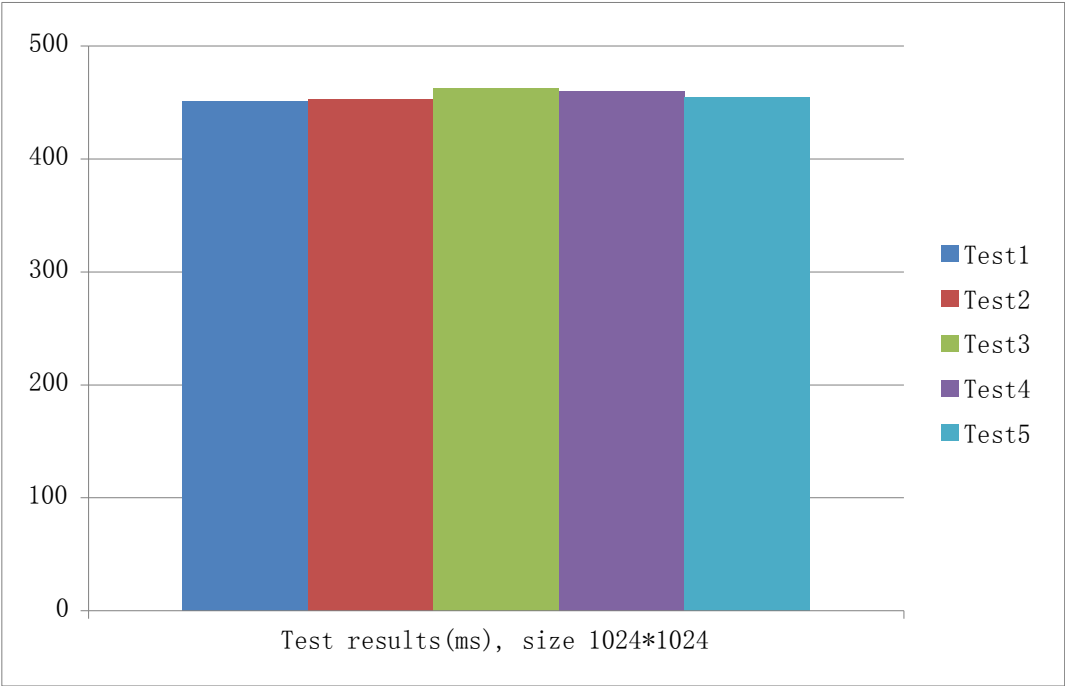


Figure 5.6: Results for image size of 1024×1024

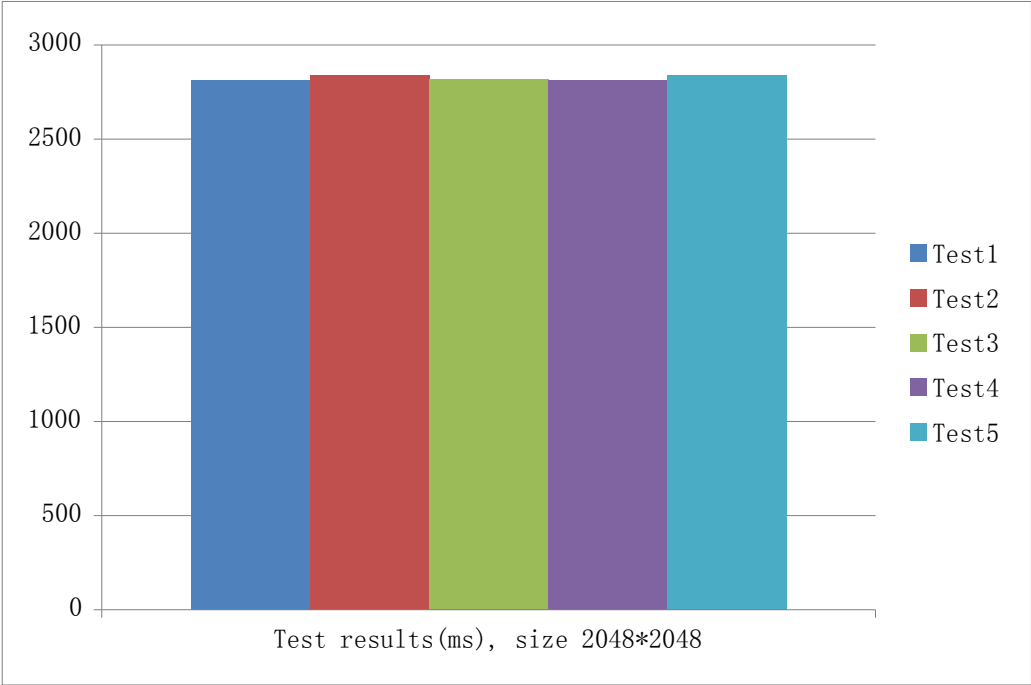


Figure 5.7: Results for image size of 2048×2048

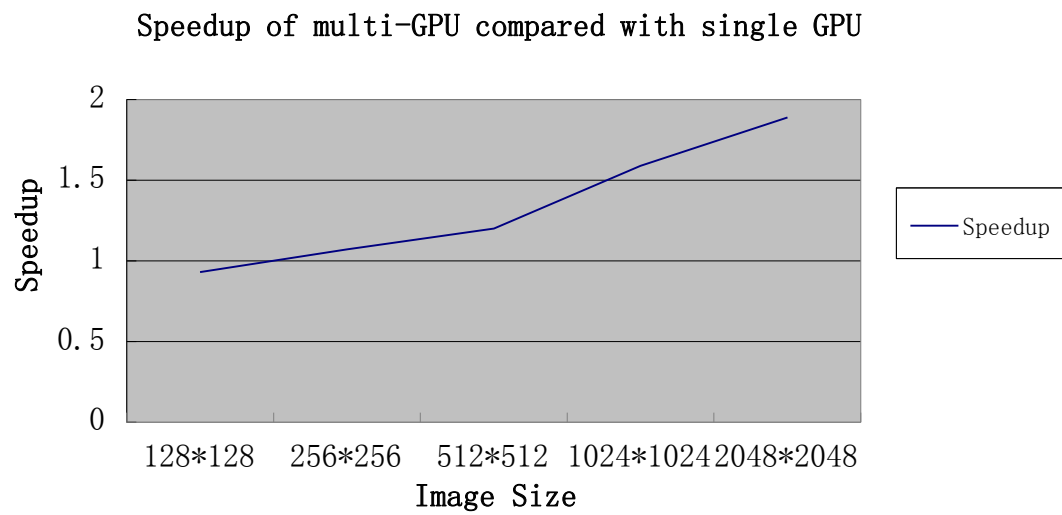


Figure 5.8: Speedup of Multi-GPU Compared with Single GPU

Chapter 6

Conclusions and future work

6.1 Conclusions

With great development on GPU technology, putting heavy load computation on GPU becomes possible and efficient. In this thesis, we implemented one of the most-used algorithms in image processing area on both single GPU and multi-GPU system.

From the experiment results from above chapters, what we can conclude are:

1. The algorithm on GPGPU is much faster than the algorithm on CPU. In the best case, the speedup could be over 245X. And within a certain limitation, the speedup is better if the data size is larger. Since the initialization of the first data transfer between host and device will take most of the time, if the data size is very small, the cost for data transfer will be too much compared to computational cost. In another word, if the speedup from parallel computing cannot compensate the overhead, the performance will be worsened. This is the reason why small data size could not take fully advantage of the computing capability of GPU.
2. The multi-GPU system has a better performance than single GPU system when the data size is large enough. In this case, when the data size is larger than 256×256 , the speedup could be a little lower than two, which is the number of GPUs in this system. It shows that the efficiency of multi-GPU system is very high, not many resources would be wasted on coordinate the two GPUs in the system. Besides that, the bandwidth for data transfer between host and device would be higher, which is also an important reason to have a better performance.
3. Optimization is critical for GPGPU program. If the program is not well op-

timized, the performance might be worsened instead of improved. Different optimization strategies should be applied for different devices, since the optimization is highly related to the device itself. For example, shared memory is supposed to improve the performance very much. However, in our case, our best results are just close to non-optimized results.

To summarize, data size is critical for the performance of GPU computing. Only when the data size is large enough, the power of GPGPU could be fully utilized. It is also possible to have an even better performance by building multi-GPU system. However, the cost to build such a system is usually very high and the performance price ratio is not acceptable for most users. Besides saving time for computing, GPGPU could also save power and money. The the same computing load, the working time could be saved so much that power consuming is also cut down. Meanwhile, the cost to build a computing system in other ways would be extremely high compared to GPGPU system.

6.2 Future Work

There is quite much work could be done in the future. The algorithm could be further optimized to improve better performance. The algorithm could also be applied for other more complex image processing applications. The study about the bottleneck during GPU computing should be studied so better hardware structure could be designed in the future. The internal chip architecture is still the base for all of these.

Also, since there are many strategies for optimization in CUDA and different strategies might conflict with each other, it caused much trouble during optimization. For example, if we used to much shared memory, which is obviously out of the amount of local memory, the system might just crash. So it is necessary to implement some computer-assisted software to help analyze and predict the performance after optimization. Besides performance analysis, the power consumption model could also be built. The multi-GPU system usually contains many GPUs and CPUs in the system, such as Nebulae supercomputer in China. This kind of supercomputer system consumes a large amount of power during computing. So it is very important to analyze the power consumption model to save energy for multi-GPU system. The research in this area is also important for introducing GPGPU to embedded devices, since low power consumption is critical for embedded device, such as smart phones.

GPGPU has proven its potential in computing. In the next a few years, it will not only be applied on high-end personal computer or supercomputer. It could also

be applied on the embedded system to improve the experience on the mobile device in the future.¹

¹All the work is performed at Institute of Computing Technology, CAS, Branch Taizhou, supervised by Xiufeng Wang

Bibliography

- [1] N. Ahmed, T. Natarajan, and K. Rao, “Discrete cosine transform,” *Computers, IEEE Transactions on*, vol. 100, no. 1, pp. 90–93, 1974.
- [2] G. Wallace, “The jpeg still picture compression standard,” *Communications of the ACM*, vol. 34, no. 4, pp. 30–44, 1991.
- [3] *MPEG-2 Video, Draft Int. Standard*.
- [4] D. Trainor, J. Heron, and R. Woods, “Implementation of the 2d dct using a xilinx xc6264 fpga,” in *Signal Processing Systems, 1997. SIPS 97-Design and Implementation., 1997 IEEE Workshop on*, pp. 541–550, IEEE, 1997.
- [5] S. Asano, T. Maruyama, and Y. Yamaguchi, “Performance comparison of fpga, gpu and cpu in image processing,” in *Field Programmable Logic and Applications, 2009. FPL 2009. International Conference on*, pp. 126–131, IEEE, 2009.
- [6] J. Owens, D. Luebke, N. Govindaraju, M. Harris, J. Krüger, A. Lefohn, and T. Purcell, “A survey of general-purpose computation on graphics hardware,” vol. 26, no. 1, pp. 80–113, 2007.
- [7] C. Timm, A. Gelenberg, F. Weichert, and P. Marwedel, *Reducing the Energy Consumption of Embedded Systems by Integrating General Purpose GPUs*. TU, Dep. of Computer Science, 2010.
- [8] Y. Arai, T. Agui, and M. Nakajima, “A fast dct-sq scheme for images,” *IEICE TRANSACTIONS*, vol. 1976, 1988.
- [9] Z. Alkachouh and M. Bellanger, “Fast dct-based spatial domain interpolation of blocks in images,” *Image Processing, IEEE Transactions on*, vol. 9, no. 4, pp. 729–732, 2000.
- [10] J. Owens, M. Houston, D. Luebke, S. Green, J. Stone, and J. Phillips, “Gpu computing,” *Proceedings of the IEEE*, vol. 96, no. 5, pp. 879–899, 2008.

- [11] I. Buck, T. Foley, D. Horn, J. Sugerman, K. Fatahalian, M. Houston, and P. Hanrahan, *Brook for GPUs: stream computing on graphics hardware*, vol. 23. 2004.
- [12] C. Nvidia, *CUDA C Programming Guide*. Version 3.2, 2010.
- [13] J. Thibault and I. Senocak, “Cuda implementation of a navier-stokes solver on multi-gpu desktop platforms for incompressible flows,” *Mechanical and Biomedical Engineering Faculty Publications and Presentations*, p. 4, 2009.
- [14] C. OpenMP, *OpenMP Application Program Interface Version 2.5*. <http://www.openmp.org>, 2005.

Mastering Apache spark architecture: A guide to optimizing data processing workflows

Quang Hai Khuat *

University of Rennes 1, France.

World Journal of Advanced Engineering Technology and Sciences, 2025, 15(01), 910-923

Publication history: Received on 01 March 2025; revised on 08 April 2025; accepted on 11 April 2025

Article DOI: <https://doi.org/10.30574/wjaets.2025.15.1.0294>

Abstract

This article provides a comprehensive guide to mastering Apache Spark architecture and optimizing data processing workflows. It begins by exploring the fundamental components of Spark's distributed computing model, including the driver program, cluster manager, and executors. The discussion then delves into advanced topics such as resource management, data locality enhancement, and fault tolerance mechanisms. Particular attention is given to performance optimization techniques, including memory management strategies, shuffle operation improvements, and Spark SQL tuning for complex queries. The article also covers the effective use of the Spark Web UI for monitoring and identifying performance bottlenecks. Real-world case studies and quantitative analyses demonstrate the practical impact of these optimization techniques across various industries. Finally, the article examines emerging trends in the Spark ecosystem, including integration with cloud-native technologies and the importance of continuous learning for data engineers. This guide serves as an essential resource for data professionals seeking to harness the full potential of Apache Spark in building scalable and efficient big data processing solutions.

Keywords: Apache Spark Architecture; Data Processing Optimization; Distributed Computing; Fault Tolerance; Performance Tuning

1. Introduction

In the era of big data, Apache Spark has emerged as a powerful and versatile distributed computing system, revolutionizing the way organizations process and analyze vast amounts of information. As data volumes continue to grow exponentially, the need for efficient and scalable data processing solutions has become paramount. Apache Spark, with its in-memory computing capabilities and support for diverse workloads, has positioned itself as a cornerstone technology in modern data engineering ecosystems.

Understanding the intricacies of Apache Spark's architecture is crucial for data engineers aiming to harness its full potential. The complexity of Spark's distributed nature, coupled with its dynamic resource allocation and execution model, presents both challenges and opportunities for optimization. By delving deep into Spark's architectural components and their interplay, engineers can unlock significant performance improvements and resource efficiencies.

This article aims to provide a comprehensive guide to mastering Apache Spark architecture, with a focus on optimizing data processing workflows. We will explore the fundamental building blocks of Spark, including the driver program, cluster manager, and executors, and how they collectively form the backbone of Spark applications. Moreover, we will examine strategies for enhancing data locality, leveraging Spark's fault tolerance mechanisms, and utilizing monitoring tools like the Spark Web UI to fine-tune performance.

* Corresponding author: Quang Hai Khuat.

The importance of this knowledge cannot be overstated in today's data-driven landscape. According to a recent survey by a leading provider of Apache Spark-based analytics platforms, many organizations reported that optimizing their Spark jobs led to significant cost savings in their data processing operations [1]. This statistic underscores the tangible benefits of mastering Spark architecture and optimization techniques.

As we navigate through the complexities of Spark's ecosystem, we will not only address current best practices but also look ahead to emerging trends and future developments. By the end of this article, readers will be equipped with the knowledge and insights necessary to design, implement, and optimize Spark applications that can handle the most demanding data processing tasks efficiently and effectively.

2. Apache spark architecture: key components

Apache Spark's distributed computing model is built upon three key components that work in concert to process data across a cluster of machines: driver program, cluster manager and executors (see Figure 1). Understanding these components is crucial for optimizing Spark applications and achieving peak performance.

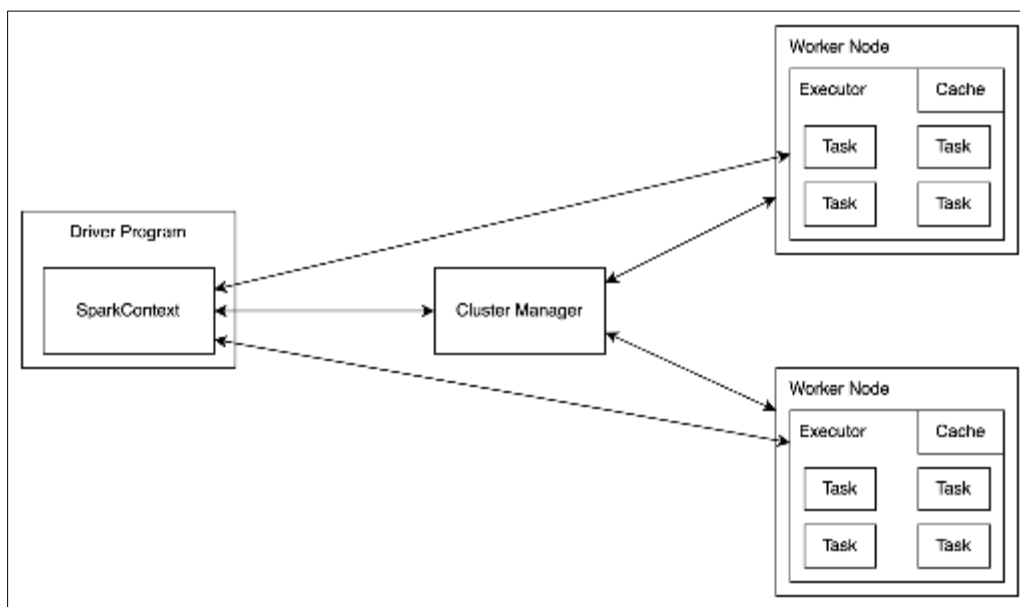


Figure 1 Apache Spark's distributed architecture [9]

2.1. Driver Program

The Driver Program is the heart of a Spark application. It runs the main function, which is responsible for creating the SparkContext - the entry point for any Spark functionality. The Driver Program performs several critical tasks:

- It translates user code into a Directed Acyclic Graph (DAG), which represents the sequence of operations to be performed on the data. The DAG is then divided into stages (group of tasks) that can be executed in parallel. For instance, a map operation followed by a reduce creates two stages: the first processes data locally, while the second requires shuffling data across nodes. Figure 2 illustrates a more complex DAG example with multiple stages/tasks.
- It communicates with the Cluster Manager to request resources such as CPU and memory. Once resources are allocated, it assigns tasks to executors running on worker nodes.
- It monitors task execution, manages data flow between stages, and handles retries in case of task failures.

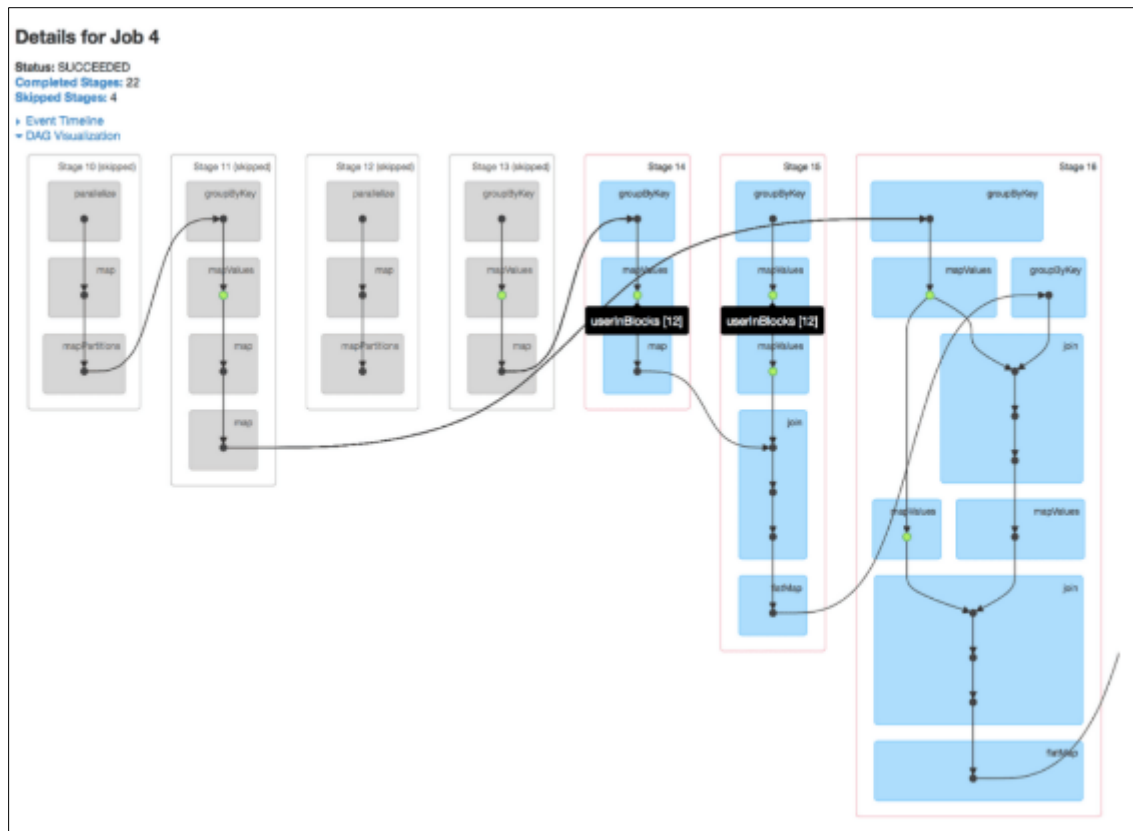


Figure 2 DAG visualization example [9]

The Driver Program typically runs on a separate machine from the worker nodes which are physical or virtual machines responsible for executing the assigned tasks. This separation enables the Driver Program to oversee and coordinate the entire distributed computation process efficiently.

2.2. Cluster Manager

The Cluster Manager acts as a resource allocator for Spark applications. It is responsible for managing and distributing the computational resources across the cluster. Spark supports several cluster managers, including:

- Standalone: Spark's built-in cluster manager
- Apache Mesos: A general-purpose cluster manager
- Hadoop YARN: The resource manager in Hadoop 2
- Kubernetes: A container orchestration platform

The Cluster Manager negotiates resources between Spark applications and ensures efficient utilization of the cluster's computing power. It plays a crucial role in scaling Spark applications and managing multi-tenancy scenarios where multiple applications share cluster resources.

2.3. Executors

Executors are processes that run on worker nodes in the cluster. They are responsible for executing the tasks assigned by the Driver Program. Key aspects of Executors include:

- They run tasks in separate threads, allowing for parallel execution.
- They store data in memory or on disk, facilitating data sharing across operations.
- They return results to the Driver Program upon task completion.

Executors are long-lived processes that persist throughout the lifetime of a Spark application, which helps to minimize the overhead of task startup.

In accordance with the key components, the Spark execution flow proceeds through the following steps, which are visually represented in the diagram in Figure 3 below:

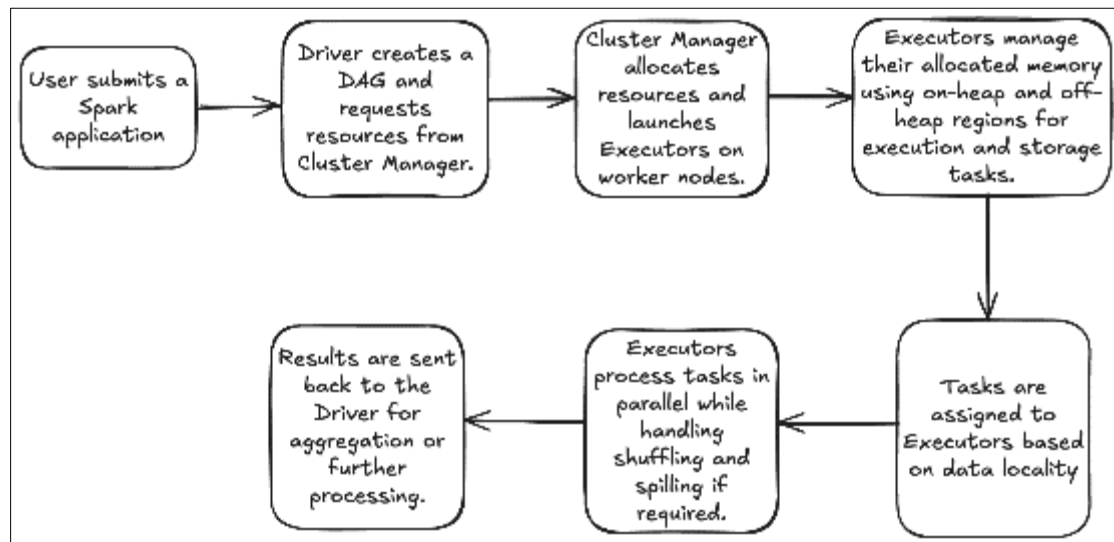


Figure 3 Spark execution flow

The interplay between these three components - Driver Program, Cluster Manager, and Executors - forms the foundation of Spark's distributed computing model. This architecture allows Spark to process vast amounts of data in parallel, providing both scalability and fault tolerance. A study demonstrated that Spark's architecture could outperform Hadoop MapReduce by up to 10 times in certain iterative machine learning tasks, highlighting the efficiency of its in-memory processing capabilities [2].

Understanding these key components and their interactions is essential for data engineers looking to optimize their Spark applications. By fine-tuning the configuration of each component and leveraging Spark's distributed nature effectively, engineers can significantly enhance the performance and resource utilization of their data processing workflows.

3. Apache Spark Essential Concepts and Terminologies

Spark architecture and execution model rely on several foundational concepts that make it scalable, fault-tolerant, and high-performing. Before diving deeper into Spark optimization strategies, it is crucial to understand the key concepts that underpin its functionality. These concepts form the building blocks of how Spark operates, from managing resources and processing data to handling failures and optimizing performance.

3.1. Core Concepts

- **Driver Program:** The main coordinator that manages tasks, stages, and execution of the Spark application.
- **SparkContext:** The entry point to Spark, connecting the Driver to the cluster.
- **Cluster Manager:** Allocates resources across the cluster (e.g., Standalone, YARN, Mesos, Kubernetes).
- **Worker Nodes:** Machines in the cluster where tasks are executed.
- **Executors:** Processes on worker nodes that run tasks and manage data partitions.

3.2. Data Concepts

- **RDD (Resilient Distributed Dataset):** Immutable distributed data collection with fault tolerance and parallel processing.
- **DataFrame:** A distributed table-like structure with named columns for structured data processing.
- **Dataset:** A strongly-typed version of DataFrame with compile-time type safety.
- **Partitions:** Data in RDDs is divided into smaller chunks called partitions. Each partition is processed independently by an executor, enabling parallelism.

3.3. Execution Concepts

- DAG (Directed Acyclic Graph): Logical execution plan representing operations and dependencies.
- Stages: Groups of tasks that can run in parallel, separated by shuffle boundaries.
- Tasks: Smallest unit of work, operating on a single partition.
- Jobs: A complete computation triggered by an action (e.g., collect, count).
- Dynamic Resource Allocation: Automatically adjusts the number of executors based on workload.

3.4. Operational Concepts

- Transformations: Lazy operations (e.g., map, filter) that define how data is processed but don't execute immediately.
- Actions: Operations (e.g., collect, count) that trigger execution and return results or write output.
- Lazy Evaluation: Transformations are recorded in a DAG and executed only when an action is called.
- Shuffling: Redistribution of data across partitions for wide transformations (e.g., reduceByKey).
- Spilling: Writing intermediate data to disk when memory is insufficient.

3.5. Performance Optimization Concepts

- Caching/Persistence: Storing intermediate results in memory or disk for reuse in iterative computations.
- Broadcast Variables: Shared read-only variables sent to all executors to reduce communication overhead.
- Accumulators: Aggregation variables used for counters or sums across tasks.
- Executor Cores: Number of threads per executor for running tasks in parallel.

3.6. Memory Management Concepts

- Execution Memory: Memory used for computations like shuffles and aggregations during task execution.
- Storage Memory: Memory used for caching RDDs or DataFrames and storing broadcast variables.
- On-Heap Memory: Managed within the JVM heap space and subject to garbage collection. Includes execution, storage, and user memory. Configured using `spark.executor.memory`
- Off-Heap Memory: Allocated outside the JVM heap to avoid garbage collection overhead. Enabled via `spark.memory.offHeap.enabled` and configured using `spark.memory.offHeap.size`.
- Unified Memory: A combined pool of execution and storage memory that allows dynamic sharing based on workload.
- `spark.memory.fraction`: Defines the fraction of JVM heap allocated for unified memory (default is 60%).
- `spark.memory.storage`: Determines the ratio of storage vs. execution memory within unified memory (default is 50%).
- Memory Overhead: Extra non-JVM memory allocated for tasks like shuffling and serialization. Configured using `spark.executor.memoryOverhead`.
- Garbage Collection (GC): JVM process to free up memory by clearing unused objects during execution.

3.7. Fault Tolerance Concepts

- Lineage Graphs: Tracks transformations applied to RDDs/DataFrames for recomputation during failures.
- Checkpointing: Saves RDDs/DataFrames to stable storage to avoid recomputation during failures or iterations.

Understanding Spark's core architecture and operational paradigms is essential for building robust, scalable, and high-performing data applications. From the way data is partitioned and processed in parallel to how memory and failures are handled, these foundational concepts provide the groundwork for effective Spark usage and optimization. Mastery of these terminologies enables developers to better analyze performance bottlenecks, fine-tune resource usage, and build fault-tolerant distributed systems.

4. Resource Management and Job Execution

Effective resource management and job execution are crucial for optimizing Apache Spark applications. This section explores key strategies for configuring Spark components, balancing resource allocation, and distributing jobs efficiently across a cluster.

4.1. Configuring Spark components for optimal performance

Configuring Spark components properly is essential for achieving optimal performance. Key areas to focus on include:

- Memory management: Properly setting executor and driver memory can significantly impact performance. This involves tuning parameters like `spark.executor.memory` and `spark.driver.memory`.
- Serialization: Choosing the right serialization format (e.g., Kryo serialization) can reduce data transfer overhead between nodes.
- Compression: Enabling compression for shuffle operations can reduce network I/O, but at the cost of increased CPU usage.
- Executor cores: Adjusting `spark.executor.cores` allows fine-tuning of parallelism within each executor.

4.2. Balancing resource allocation

Balancing resource allocation is a delicate process that requires understanding both your data and your cluster's capabilities:

- Right-sizing executors: Determining the optimal number and size of executors based on your cluster's resources and the nature of your workload.
- Dynamic allocation: Utilizing Spark's dynamic resource allocation feature to automatically adjust the number of executors based on workload.
- Fair scheduling: Implementing fair scheduling policies to ensure equitable resource distribution among multiple concurrent jobs.

4.3. Strategies for efficient job distribution

Efficient job distribution is key to maximizing cluster utilization and minimizing processing time:

- Data partitioning: Properly partitioning data can significantly improve parallel processing efficiency.
- Task scheduling: Understanding and optimizing Spark's task scheduling mechanisms to reduce data movement and improve locality.
- Speculative execution: Leveraging Spark's speculative execution feature to mitigate the impact of stragglers (slow tasks) on overall job completion time.
- Caching and persistence: Strategically caching frequently accessed data to reduce computation and I/O overhead.

Implementing effective resource management and job execution strategies in Apache Spark requires a solid grasp of the platform's architecture as well as a clear understanding of the unique demands of each data processing workflow. There is no universal solution—performance tuning is often an iterative process of experimentation and refinement. However, by thoughtfully configuring Spark components, allocating resources wisely, and optimizing job distribution, data engineers can build scalable, high-performing applications [3]. These efforts not only improve execution speed and system efficiency but also strengthen the resilience and flexibility of data pipelines, empowering organizations to extract insights from large-scale data more reliably and effectively.

5. Enhancing data locality

Data locality is a crucial concept in distributed computing systems like Apache Spark, significantly impacting the overall performance of data processing workflows. This section explores the concept of data locality in Spark, techniques to improve it, and its impact on performance.

5.1. Concept of data locality in Spark

Data locality refers to the practice of moving computation close to where the data resides, rather than moving large amounts of data to the computation. In Spark, this principle is applied at multiple levels:

- Node-local: Data and computation are on the same node.
- Rack-local: Data and computation are on different nodes but within the same rack.
- Off-rack: Data needs to be fetched from a different rack.

Spark's scheduler attempts to achieve the highest level of locality possible for each task, aiming to minimize data transfer across the network.

5.2. Techniques to improve data locality

Several techniques can be employed to enhance data locality in Spark applications:

- Proper data partitioning: Distributing data evenly across the cluster to ensure balanced processing.
- Caching strategies: Using Spark's `cache()` or `persist()` methods to keep frequently accessed data in memory.
- Broadcast variables: Utilizing broadcast variables for sharing read-only data across nodes efficiently.
- Coalesce and repartition operations: Adjusting the number of partitions to optimize data distribution.
- Custom partition placement: Implementing custom placement strategies for specific use cases.

5.3. Impact on overall performance

Enhancing data locality can have a significant positive impact on the overall performance of Spark applications:

- Reduced network traffic: Minimizing data movement across the network decreases latency and improves job execution speed.
- Improved resource utilization: Better data locality leads to more efficient use of CPU and memory resources.
- Faster processing times: Tasks can start immediately when data is locally available, reducing wait times.
- Scalability: Improved data locality allows Spark applications to scale more effectively as cluster size increases.

Data locality plays a vital role in the performance and scalability of Apache Spark applications, as evidenced by numerous research efforts [4][10]. By aligning computation with data placement, Spark minimizes unnecessary data transfers, leading to faster job execution and more efficient use of resources. Through techniques such as effective partitioning, caching, broadcasting, and custom data placement, data engineers can significantly enhance locality and, in turn, improve processing speed, reduce network overhead, and support scalable, cost-effective data pipelines. Prioritizing data locality is thus essential for optimizing distributed workflows and maximizing the potential of Spark in large-scale data environments.

6. Leveraging spark's fault tolerance

Fault tolerance is a critical feature of Apache Spark that ensures reliability and resilience in distributed computing environments. This section explores Spark's fault tolerance mechanisms, how to configure them, and the trade-offs between fault tolerance and performance.

6.1. Overview of Spark's fault tolerance mechanisms

Spark employs several mechanisms to achieve fault tolerance:

- Resilient Distributed Datasets (RDDs): The fundamental abstraction in Spark, RDDs are immutable, distributed collections of objects that can be rebuilt if a partition is lost.
- Lineage graphs: Spark maintains a record of all transformations applied to RDDs, allowing it to recompute lost data.
- Checkpointing: This mechanism allows saving RDD computation to stable storage, providing a fallback point for recovery.
- Speculative execution: Spark can run multiple copies of slower tasks to mitigate stragglers and node failures.

6.2. Configuring fault tolerance settings

Configuring Spark's fault tolerance involves several key settings:

- Storage level: Using `persist()` or `cache()` with appropriate storage levels (e.g., `MEMORY_AND_DISK`) to balance between memory usage and fault tolerance.
- Checkpointing interval: Setting appropriate checkpointing intervals based on the criticality of the data and the cost of recomputation.
- Speculative execution threshold: Adjusting `spark.speculation.multiplier` to control when Spark launches speculative tasks.
- Recovery mode: Configuring `spark.deploy.recoveryMode` for the Spark Standalone cluster manager to handle master node failures.

6.3. Trade-offs between fault tolerance and performance

While fault tolerance is crucial for reliability, it often comes with performance costs:

- Storage overhead: Persisting or caching RDDs consumes memory and disk space, which could otherwise be used for computation.
- Checkpointing overhead: Writing data to stable storage takes time and can slow down job execution.
- Speculative execution costs: Running duplicate tasks consumes additional resources and can impact overall cluster utilization.
- Lineage recomputation: While lineage allows for fault recovery, recomputing lost partitions can be time-consuming for long chains of transformations.

Fault tolerance is a foundational aspect of Apache Spark that ensures reliability in distributed data processing environments [5]. By leveraging mechanisms such as RDD lineage, checkpointing, and speculative execution, Spark can recover from failures and maintain data integrity during execution. However, these capabilities come with trade-offs in terms of performance, storage, and resource consumption. Effectively configuring fault tolerance requires a thoughtful balance between reliability and efficiency, based on the criticality of the data and the nature of the workload. Ultimately, understanding and tuning Spark's fault tolerance features is essential for building robust, resilient, and high-performing data applications.

7. Monitoring and Optimization with Spark Web UI

Effective monitoring and optimization are crucial for maintaining high-performance Apache Spark applications. The Spark Web UI is a powerful tool that provides insights into application behavior and performance. This section explores the Spark Web UI, key metrics for performance analysis, and strategies for identifying and addressing bottlenecks.

7.1. Introduction to Spark Web UI

The Spark Web UI (see Figure 4) is a built-in web interface that provides a comprehensive view of Spark application execution. It offers:

- Real-time monitoring of running applications
- Detailed information about completed jobs and stages
- Visualization of the DAG (Directed Acyclic Graph) for each job (see Fig 3)
- Access to executor and storage information
- SQL query execution details for Spark SQL applications

Stage ID	Pool Name	Description	Submitted	Duration	Tasks: Succeeded/Total	Input	Output	Shuffle Read	Shuffle Write
13	production	show all <console>26	2023/08/27 18:25:33	0.2 s	0/4 (4 running)				
14	default	show all <console>28	Unknown	Unknown	0/200				
12	production	show all <console>26	2023/08/27 18:25:33	0.3 s	4/4				1205.5 KiB
11	production	show all <console>28	2023/08/27 18:24:36	0.3 s	1/3			12.6 KiB	
10	production	show all <console>28	2023/08/27 18:24:35	0.3 s	4/4				1205.5 KiB
9	production	show all <console>30	2023/08/27 18:24:35	0.3 s	4/4				1205.5 KiB
8	default	touch all <console>27	2023/08/27 18:03:04	33 ms	4/4				
7	default	touch all <console>27	2023/08/27 17:59:31	0.6 s	4/4				
6	production	show all <console>26	2023/08/27 17:56:10	84 ms	1/3			12.6 KiB	
5	production	show all <console>28	2023/08/27 17:56:10	0.2 s	4/4				1205.5 KiB

Figure 4 Spark Web UI - Stage detail view [9]

The Web UI is automatically available for each Spark application and can be accessed through a web browser, typically on port 4040 of the driver node.

7.2. Key metrics for performance analysis

The Spark Web UI provides numerous metrics that are essential for performance analysis:

- Job duration and task skew
- Executor CPU and memory usage
- Shuffle read and write sizes
- GC (Garbage Collection) time
- Data spill to disk
- Stage input and output sizes
- Task serialization and deserialization times
- Executor idle time

These metrics offer valuable insights into the application's resource utilization, data flow, and potential bottlenecks.

7.3. Identifying and addressing bottlenecks

The Spark Web UI is instrumental in identifying performance bottlenecks:

- Data skew: Uneven distribution of data across partitions can be identified through task duration variability within stages.
- Resource contention: High executor CPU or memory usage may indicate the need for better resource allocation.
- Excessive shuffling: Large shuffle read/write sizes can point to opportunities for optimizing data partitioning or reducing wide transformations.
- Serialization issues: High serialization times may suggest the need for more efficient data serialization methods.
- Memory pressure: Frequent spilling to disk or high GC times can indicate memory-related issues.

Once bottlenecks are identified, various optimization strategies can be applied:

- Adjusting partition sizes to balance data distribution
- Tuning executor configurations for better resource utilization
- Caching frequently accessed data to reduce recomputation
- Optimizing shuffle operations through careful ordering of transformations
- Implementing custom partitioners for more efficient data distribution

The Spark Web UI is an invaluable tool for monitoring and optimizing Apache Spark applications, providing real-time insights into job execution, resource utilization, and performance metrics. By leveraging key metrics such as job duration, memory usage, and task skew, data engineers can quickly identify bottlenecks and areas for improvement. Addressing these issues through techniques like adjusting partition sizes, tuning executor configurations, and optimizing shuffle operations can significantly enhance application performance.

Studies have shown that effective use of the Spark Web UI can lead to considerable performance improvements, demonstrating its importance in optimizing Spark workflows [6]. Ultimately, the Spark Web UI enables data engineers to fine-tune their applications, resulting in better resource efficiency and faster processing times.

8. Advanced optimization techniques

As data engineers delve deeper into Apache Spark optimization, they encounter more sophisticated techniques that can significantly enhance performance. This section explores advanced strategies in memory management, shuffle optimization, and Spark SQL tuning for complex queries.

8.1. Memory management strategies

Effective memory management is crucial for Spark's performance, especially when dealing with large datasets:

- Off-heap memory: Utilizing off-heap storage for certain operations can reduce garbage collection overhead and improve performance for memory-intensive tasks.
- Memory fraction tuning: Adjusting `spark.memory.fraction` and `spark.memory.storage` to optimize the balance between execution and storage memory.
- Serialization: Using Kryo serialization instead of Java serialization can reduce memory footprint and improve overall performance.
- Caching strategies: Implementing intelligent caching policies, such as selective caching of frequently accessed data or using appropriate storage levels (e.g., `MEMORY_AND_DISK_SER`).

8.2. Optimizing shuffle operations

Shuffle operations are often a major bottleneck in Spark applications:

- Reduce shuffle partitions: Adjusting `spark.sql.shuffle.partitions` to an optimal value based on data size and cluster resources.
- Broadcast joins: Utilizing broadcast joins for smaller tables to avoid shuffling large datasets.
- Repartitioning strategies: Implementing custom partitioning to reduce data movement during shuffles.
- Avoid unnecessary shuffles: Analyzing the query execution plan to identify and eliminate redundant shuffle operations.

8.3. Tuning Spark SQL for complex queries

For applications heavily relying on Spark SQL, specific optimizations can yield substantial performance gains:

- Cost-based optimization: Leveraging Spark's cost-based optimizer by gathering table and column statistics.
- Query plan optimization: Analyzing and rewriting complex queries to leverage Spark's query optimizer effectively.
- Predicate pushdown: Ensuring filter operations are pushed down to data sources, when possible, to reduce data transfer.
- Columnar format: Using columnar storage formats like Parquet for analytical queries to improve I/O performance.
- Adaptive Query Execution: Enabling and fine-tuning Spark's Adaptive Query Execution feature for dynamic query optimization.

Advanced optimization techniques in Apache Spark—spanning memory management, shuffle efficiency, and Spark SQL tuning—offer powerful ways to dramatically boost performance for large-scale data processing. Table 1 provides a quick reference to understand how each technique contributes to improved efficiency and reliability. Techniques like off-heap memory use, Kryo serialization, custom partitioning, and Adaptive Query Execution can help overcome the limitations of default configurations.

While these strategies demand a deeper understanding of Spark's internals and workload-specific tuning, studies have shown they can yield substantial performance improvements for complex queries [7]. Ultimately, when applied thoughtfully and validated through benchmarking, these optimizations empower data engineers to build faster, more efficient, and scalable Spark applications.

Table 1 Advanced Optimization Techniques and Their Impact

Optimization Technique	Description	Potential Impact
Memory Management	Off-heap storage, memory fraction tuning, efficient serialization	Reduced GC overhead, improved memory utilization
Shuffle Optimization	Reducing shuffle partitions, broadcast joins, custom partitioning	Decreased data movement, faster job completion
Spark SQL Tuning	Cost-based optimization, query plan optimization, predicate pushdown	Improved query performance, especially for complex analytical queries

Data Enhancement	Locality	Proper data partitioning, caching strategies	Reduced network traffic, faster data access
Fault Configuration	Tolerance	Checkpointing, lineage graph optimization	Balanced reliability and performance

9. Practical Applications and Case Studies

The true value of mastering Apache Spark architecture and optimization techniques becomes evident when examining real-world applications and industry implementations. This section explores practical examples of optimized Spark workflows, lessons learned from various industry implementations, and quantitative analyses of performance improvements.

9.1. Real-world examples of optimized Spark workflows

- **E-commerce Recommendation Engine:** A large online retailer implemented a Spark-based recommendation engine processing terabytes of user interaction data daily. By optimizing data partitioning and caching strategies, they reduced processing time by 40% and improved recommendation accuracy.
- **Financial Risk Analysis:** A global bank utilized Spark for real-time risk assessment of financial transactions. Through careful tuning of executor configurations and implementation of custom partitioners, they achieved sub-second response times for complex risk calculations on massive datasets.
- **Genomic Data Processing:** A biotech company leveraged Spark for analyzing large-scale genomic sequencing data. By implementing advanced memory management techniques and optimizing shuffle operations, they reduced processing time for genome assembly tasks from days to hours.

9.2. Lessons learned from industry implementations

- **Start with proper data modeling:** Many organizations found that investing time in proper data modeling and schema design upfront led to significant performance gains in Spark applications.
- **Iterative optimization is key:** Successful implementations often involved continuous monitoring and iterative optimization, rather than one-time tuning efforts.
- **Balance between generalization and specialization:** While some companies benefited from highly specialized Spark configurations for specific workloads, others found value in more generalized setups that could handle diverse tasks efficiently.
- **Importance of data locality:** Organizations that focused on enhancing data locality, through techniques like intelligent data placement and caching, saw substantial improvements in processing speeds.
- **Collaboration between data engineers and domain experts:** The most successful optimizations often resulted from close collaboration between Spark experts and domain specialists who understood the nuances of the data and business requirements.

9.3. Quantitative analysis of performance improvements

Quantifying the impact of Spark optimizations provides valuable insights into the effectiveness of various techniques:

- **Processing Time Reduction:** Many organizations reported 30-60% reductions in job completion times after implementing advanced optimization techniques.
- **Resource Utilization:** Optimized Spark workflows often led to 20-40% improvements in cluster resource utilization, allowing companies to process more data with existing infrastructure.
- **Cost Savings:** By optimizing Spark applications, some companies achieved 25-50% reductions in cloud computing costs for their big data processing pipelines.
- **Scalability Improvements:** Properly optimized Spark applications demonstrated better scalability, with some organizations reporting linear performance improvements as they scaled from hundreds to thousands of nodes.
- **Query Response Time:** For interactive analytics use cases, optimizations often resulted in 5-10x improvements in query response times, enabling real-time data exploration on large datasets.

Real-world case studies underscore the transformative impact of mastering Apache Spark optimization, revealing how targeted strategies can significantly enhance performance, efficiency, and scalability across diverse industries. From accelerating genomic analysis and powering real-time financial risk assessment to refining e-commerce recommendation systems, organizations have achieved remarkable results through thoughtful data modeling, iterative

tuning, and a strong emphasis on data locality. These practical successes—often driven by close collaboration between data engineers and domain experts—translate into measurable gains such as faster job completion, improved resource utilization, and reduced operational costs. By studying these concepts, data engineers gain insights to build efficient, scalable, and cost-effective Spark solutions.

10. Future Trends and Emerging Best Practices

As Apache Spark continues to evolve and adapt to the changing landscape of big data processing, data engineers must stay abreast of emerging trends and best practices. This section explores the future direction of the Spark ecosystem, its integration with cloud-native technologies, and the importance of continuous learning for data professionals.

10.1. Evolving Spark ecosystem

The Spark ecosystem is rapidly evolving to meet the demands of modern data processing:

- Unified Analytics: Spark is moving towards a more unified platform for batch processing, stream processing, and machine learning, allowing seamless integration of different data processing paradigms.
- Enhanced GPU support: Increased focus on GPU acceleration for machine learning and deep learning workloads within Spark applications.
- Improved Python API: Continued development of Spark to provide more native Python support and better integration with popular data science libraries.
- Adaptive Query Execution: Further refinement of Spark's ability to dynamically optimize query plans based on runtime statistics.

10.2. Integration with cloud-native technologies

Spark is increasingly being integrated with cloud-native technologies to enhance scalability and ease of use:

- Kubernetes integration: Deeper integration with Kubernetes for more efficient resource management and deployment in containerized environments.
- Serverless Spark: Development of serverless Spark offerings that abstract away infrastructure management, allowing users to focus on data processing logic.
- Cloud storage optimization: Enhanced support for cloud-native storage solutions, improving data access patterns and reducing data movement costs.
- Multi-cloud and hybrid cloud support: Improved capabilities for running Spark applications across multiple cloud providers or in hybrid cloud-on-premises environments.

10.3. Continuous learning and adaptation for data engineers

To stay relevant in the rapidly evolving field of big data processing, data engineers must commit to continuous learning and adaptation:

- Keeping up with Spark updates: Regularly reviewing new Spark releases and understanding new features and optimizations.
- Cross-disciplinary knowledge: Expanding knowledge beyond Spark to related areas such as machine learning, data governance, and cloud architecture.
- Performance tuning skills: Continuously refining skills in performance analysis and optimization, adapting to new tools and techniques as they emerge.
- Embracing DataOps practices: Adopting DataOps methodologies to improve collaboration, automation, and continuous delivery of data pipelines.
- Ethical considerations: Developing awareness of ethical implications in data processing, including data privacy and bias in machine learning models.

The importance of staying current with these trends cannot be overstated. A study conducted by researchers at the University of California, Berkeley, in collaboration with industry partners, highlighted the rapid pace of innovation in the Spark ecosystem [8]. The study found that organizations that quickly adopted new Spark features and best practices saw an average of 30% improvement in processing efficiency and were able to tackle more complex data challenges compared to those that lagged in adoption (see Figure 5).

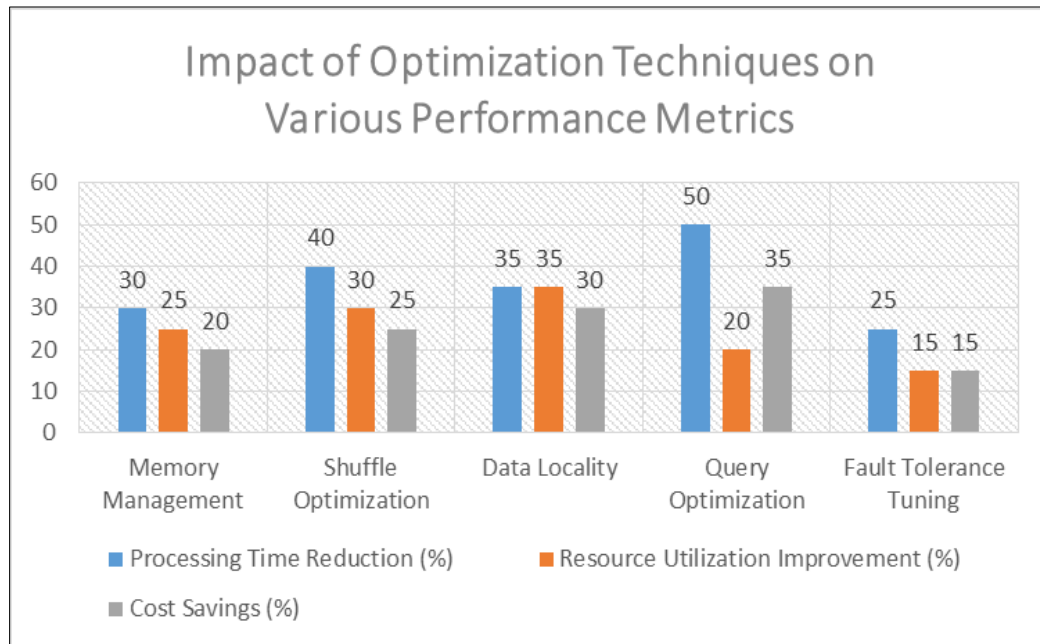


Figure 5 Impact of Optimization Techniques on Various Performance Metrics [8]

As the Spark ecosystem continues to evolve, data engineers who remain adaptable and committed to continuous learning will be best positioned to leverage its full potential. By staying informed about emerging trends, embracing cloud-native integrations, and continuously refining their skills, data engineers can ensure they remain at the forefront of big data processing capabilities, driving innovation and efficiency in their organizations.

11. Conclusion

In conclusion, mastering Apache Spark architecture and optimization techniques is crucial for data engineers seeking to build efficient and scalable data processing workflows. Throughout this article, we've explored the fundamental components of Spark's architecture, delved into advanced optimization strategies, and examined real-world applications and future trends. From understanding the interplay between the driver program, cluster manager, and executors to implementing sophisticated memory management and query optimization techniques, data engineers have a wealth of tools at their disposal to enhance Spark performance. The case studies and quantitative analyses presented underscore the significant impact that well-optimized Spark applications can have on processing times, resource utilization, and cost-efficiency. As the Spark ecosystem continues to evolve, integrating more closely with cloud-native technologies and adapting to emerging data processing paradigms, the importance of continuous learning and adaptation cannot be overstated. By staying informed about the latest developments, embracing best practices, and continuously refining their skills, data engineers can leverage Spark's full potential to drive innovation and tackle the most challenging big data problems in an increasingly data-driven world.

References

- [1] Databricks. (October 10, 2014). "Apache Spark Beats the World Record for Fastest Processing of Big Data" <https://www.databricks.com/company/newsroom/press-releases/apache-spark-beats-world-record-fastest-processing-big-data>
- [2] Matei Zaharia N. M. Mosharaf Chowdhury et al. (2010). Spark: Cluster Computing with Working Sets. In Proceedings of the 2nd USENIX Conference on Hot Topics in Cloud Computing (HotCloud'10). USENIX Association. https://www.usenix.org/event/hotcloud10/tech/full_papers/Zaharia.pdf
- [3] H. Karau and R. Warren. (2017). High-Performance Spark. O'Reilly Media, Inc. <https://www.oreilly.com/library/view/high-performance-spark/9781491943199/>

- [4] Kay Ousterhout, Ryan Rasti, et al. (2015). Making Sense of Performance in Data Analytics Frameworks. In 12th USENIX Symposium on Networked Systems Design and Implementation (NSDI 15) (pp. 293-307). USENIX Association. <https://www.usenix.org/system/files/conference/nsdi15/nsdi15-paper-ousterhout.pdf>
- [5] Paulo Vinicius Cardoso, Patricia Pitthan Barcelos. (2018). Definition of an Architecture for Dynamic and Automatic Checkpoints on Apache Spark. IEEE 37th Symposium on Reliable Distributed Systems (SRDS). <https://ieeexplore.ieee.org/document/8613979>
- [6] Diagnose cost and performance issues using the Spark UI. <https://docs.databricks.com/aws/en/optimizations/spark-ui-guide/>
- [7] S. Tang, B. He, C. Yu, Y. Li and K. Li. (2020). A Survey on Spark Ecosystem: Big Data Processing Infrastructure, Machine Learning, and Applications. IEEE Transactions on Knowledge and Data Engineering (vol. 34, no. 1, pp. 71-91). <https://ieeexplore.ieee.org/document/9007378>
- [8] Matei Zaharia, Reynold S. Xin et al. (2016). Apache Spark: A unified engine for big data processing. Communications of the ACM, 59(11), 56-65. <https://dl.acm.org/doi/10.1145/2934664>
- [9] Apache Spark - A Unified engine for large-scale data analytics. <https://spark.apache.org/docs/latest/>
- [10] Zhongming Fu, Mengsi He et al. (2022). Optimizing data locality by executor allocation in spark computing environment . IEEE Transactions on Cloud Computing. <https://ieeexplore.ieee.org/document/10540265>